Machine Learning Foundations

Every self-check item answered and explained, plus reference notes for each lab, for all 11 sessions. Use for grading and reteaching. Confidential instructor material.

11 sessions | 132 self-check items + lab solutions | Confidential · do not distribute

CONTENTS

S01 **Your ML Toolkit and the Estimator API**

- 0.1 Your ML Toolkit
- 1.1 The scikit-learn Estimator API

S02 **Splits, Leakage and Cross-Validation**

- 1.2 Splits, Baselines and Leakage
- 1.3 Cross-Validation

S03 **Metrics and Preprocessing**

- 1.4 Metrics for Classification and Regression
- 2.1 Impute, Scale and Encode

S04 **Pipelines and Creating Features**

- 2.2 Pipelines
- 2.3 Creating Features

S05 **Feature Selection and Imbalanced Classes**

- 2.4 Feature Selection
- 2.5 Imbalanced Classes

S06 **Linear Models and Regularization**

- 3.1 Linear and Logistic Regression
- 3.2 Regularization and the Bias-Variance Trade-off

S07 **Nearest Neighbors and Naive Bayes**

- 4.1 k-Nearest Neighbors
- 4.2 Naive Bayes

S08 **Support Vector Machines and Decision Trees**

- 4.3 Support Vector Machines
- 5.1 Decision Trees

S09 **Random Forests and Gradient Boosting**

- 5.2 Bagging and Random Forests
- 5.3 Gradient Boosting

S10 **Ensembles and Model Selection**

- 5.4 Voting and Stacking
- 6.1 Model Selection and Learning Curves

S11 **Hyperparameter Tuning and Final Project Studio**

- 6.2 Hyperparameter Tuning

INSTRUCTOR KIT · ANSWER KEY

Session 1: Your ML Toolkit and the Estimator API

Answers and reasoning for every Session 1 item: the diagnostic, both self-checks, the predict-then-run items, the lab reference outputs, the lesson pages' own quizzes and the exit ticket.

Session 1 | 12 self-check items | Confidential

START HERE · DIAGNOSTIC

Not graded. The reasoning for each answer also says where the skill comes back in this course.

D1

What does this print?

```
items = [5, 6, 7, 8, 9]
print(items[-3:], items[:2])
```

A) `[7, 8, 9] [5, 6]`

B) `[8, 9] [5, 6, 7]`

C) `[7, 8, 9] [5, 6, 7]`

D) `[9, 8, 7] [5, 6]`

> **Why:** A negative start counts from the end, and a slice leaves out its stop position. Selecting rows and columns of X uses the same rules. **Skill:** slicing. **Review if missed:** Intermediate Python: lists and slicing.

D2

What does this print?

```
counts = {}
for store in ["Bend", "Boise", "Bend"]:
    counts[store] = counts.get(store, 0) + 1
print(counts)
```

A) `{'Bend': 1, 'Boise': 1}`

B) `{'Bend': 2}`

C) `{'Bend': 2, 'Boise': 1}`

D) A `KeyError: 'Bend'`

> **Why:** `.get(store, 0)` returns 0 for a key not seen yet, so nothing raises. Lesson 1.1 keeps several models in a dictionary and loops over it. **Skill:** dictionaries. **Review if missed:** Intermediate Python: dictionaries and .get().

D3

What does this print?

```
def discounted(price, rate=0.10):
    return round(price * (1 - rate), 2)
print(discounted(80), discounted(80, rate=0.25))
```

A) `72.0 72.0`

B) `72.0 60.0`

C) `8.0 20.0`

D) A `TypeError`: rate is given twice

> **Why:** The default `rate=0.10` applies only when the caller leaves it out. Every scikit-learn model is created this way: `DecisionTreeClassifier(max_depth=4)` overrides one default and keeps the rest. **Skill:** functions and default arguments. **Review if missed:** Intermediate Python: functions.

D4

What does this print?

```
import pandas as pd
df = pd.DataFrame({"app_user": [1, 0, 1], "spend": [100.0, 50.0, 300.0]})
print(df.loc[df["app_user"] == 1, "spend"].mean())
```

A) `150.0`

B) `400.0`

C) `2`

D) `200.0`

> **Why:** The condition keeps the two app users (100 and 300); the mean of those is 200.0. Filtering rows by a condition is how you build X and check rates by group. **Skill:** boolean filtering. **Review if missed:** Python Data Science Lesson 2.1.

D5

What does this print?

```
import pandas as pd
df = pd.DataFrame({"tier": ["Gold", "Basic", "Basic"], "churned": [0, 1, 0]})
print(df.groupby("tier")["churned"].mean().to_dict())
```

A) `{'Basic': 0.5, 'Gold': 0.0}`

B) `{'Gold': 0.0, 'Basic': 1.0}`

C) `{'Basic': 1, 'Gold': 0}`

D) `{'Basic': 0.333, 'Gold': 0.0}`

> **Why:** Two Basic customers, one churned: 0.5. groupby sorts its keys, so Basic comes first. The churn rate by store in Lesson 0.1's Exercise 1 is exactly this. **Skill:** groupby. **Review if missed:** Python Data Science Lesson 2.4.

D6

What does this print?

```
import pandas as pd
s = pd.Series([1.0, None, 3.0, None])
print(s.isna().sum(), s.mean())
```

A) `2 1.0`

B) `0 nan`

C) `2 2.0`

D) `2 nan`

Why: `isna()` marks the two gaps; `mean()` skips them, so (1 + 3) / 2 = 2.0. Today's lab asks why values are missing; scikit-learn models refuse NaN outright (Lesson 1.1, Common Mistake 4). **Skill:** missing values. **Review if missed:** Python Data Science Lesson 2.3.

0.1 · YOUR ML TOOLKIT: SELF-CHECK

Q1

In JupyterLab, `import sklearn` fails with `ModuleNotFoundError`, yet in your activated terminal `python -c "import sklearn"` works. What is the most likely cause?

A) scikit-learn is only partly installed: reinstall it

B) The import name is wrong: use `import scikit_learn`

C) The notebook's kernel is a different Python from the one in `.venv`

D) JupyterLab cannot run scikit-learn; use a script instead

Why: The terminal proves the package is installed in the environment. A notebook runs on a kernel, and if that kernel is another Python it cannot see the environment's packages. Start `jupyter lab` from the activated terminal, or pick the `.venv` kernel in VS Code.
A is wrong: The terminal import works, so the install is fine.
B is wrong: `scikit_learn` is the install name with an underscore; the import name is `sklearn`, and the terminal shows it works.
D is wrong: JupyterLab runs any package the kernel can see.

Q2

Which of these requests is a regression problem?

A) Will this customer stop buying in the next six months?

B) Which customers shop in similar ways? (No column says.)

C) Is this product review positive or negative?

D) How many dollars will the Boise store take tomorrow?

> **Why:** The target is a quantity (a dollar amount) where values in between make sense: regression.
> **A** is wrong: Two categories (churned or stayed): classification.
> **B** is wrong: No answer column to learn from: unsupervised learning (clustering).
> **C** is wrong: Two categories: classification on text.

Q3

What does this print?

```
import pandas as pd
s = pd.Series([0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0])
print(round(s.mean(), 2), s.sum())
```

`0.25 3`

> **Why:** The mean of a 0/1 column is the share of 1s: 3 ones out of 12 values is 0.25. The sum counts the 1s. This is how every churn and fraud rate in the course is computed.

Q4

Which column of `customers_churn.csv` must never be a feature for predicting churn?

A) `email_opens_90d`, because it has 537 missing values

B) `cancellation_reason`, because it is filled in only after a customer has left

C) `home_store`, because it is text

D) `app_user`, because it only holds 0 and 1

> **Why:** On the day you make a prediction the reason is always empty, because nobody has cancelled yet. A model given it looks brilliant in testing and is useless in practice: target leakage, the subject of Lesson 1.2.
> **A** is wrong: Gaps can be filled (Lesson 2.1); the gap even carries a little signal.
> **C** is wrong: Text can be encoded as numbers (Lesson 2.1).
> **D** is wrong: A 0/1 column is a perfectly good feature.

Q5

Your model scores 0.840 accuracy on held-out customers, and a model that always predicts "stays" scores 0.835. Which workflow step are you in, and where does the loop send you if this is not good enough?

A) Evaluate; back to Prepare

B) Train; on to Explain

C) Explain; back to Frame

D) Deploy; on to Monitor

> **Why:** Comparing with a baseline on data the model never saw is Evaluate. The loop goes back to Prepare (better features, better framing) before anything is explained or deployed.
> **B** is wrong: Training is done; you are grading the result.
> **C** is wrong: You explain a model only after it has earned its place.
> **D** is wrong: A model that barely beats doing nothing is not ready to deploy.

Q6

What does this print?

```
import pandas as pd
customers = pd.read_csv("data/customers_churn.csv")
print(customers["signup_date"].dtype)
```

`str`

Why: `read_csv` leaves dates as text unless you pass `parse_dates=["signup_date"]`. In pandas 3 text columns have the `str` dtype. Until the column is a date, the `.dt` tools are not available.

0.1 · YOUR ML TOOLKIT: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
import pandas as pd
flags = pd.Series([1, 0, 0, 0, 0, 0, 0, 1, 0, 0])
print(flags.mean(), round(1 - flags.mean(), 1))
```

`0.2 0.8`

Why: The mean of a 0/1 Series is the share of 1s (2 of 10); one minus it is the share of 0s. Exactly what "always guessing 0" would score.

P2

Predict the output, then run it.

```
import pandas as pd
s = pd.Series([3.0, None, 5.0, None])
print(s.isna().sum(), s.mean())
```

`2 4.0`

Why: `isna().sum()` counts the two gaps; `mean()` skips gaps, so (3 + 5) / 2 = 4.0.

1.1 · THE SCIKIT-LEARN ESTIMATOR API: SELF-CHECK

Q7

What happens when this runs?

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
X = pd.DataFrame({"days": [10, 200, 30, 300]}); y = [0, 1, 0, 1]
model = LogisticRegression()
model.fit(X, y)
model = LogisticRegression()
print(model.predict(X))
```

A) It prints `[0 1 0 1]`

B) It prints `[0 0 0 0]`: a new model knows nothing

C) `ValueError`: the features must be scaled first

D) `NotFittedError`: the last assignment replaced the fitted model with a fresh one

> **Why:** Creating an estimator only stores its settings. The second `LogisticRegression()` throws away the fitted model, and predicting with an unfitted model raises `NotFittedError`. In a notebook this is what happens when you re-run the cell that creates the model.
> **A** is wrong: That would be the fitted model's answer, but it is gone.
> **B** is wrong: An unfitted model does not guess; it refuses.
> **C** is wrong: Logistic regression runs on unscaled data (it may warn about convergence, but it does not raise this).

Q8

What does this print?

```
from sklearn.preprocessing import StandardScaler
s = StandardScaler().fit([[2], [4], [6]])
print(s.mean_, s.transform([[4], [8]]).ravel().round(3))
```

`[4.] [0. 2.449]`

> **Why:** `fit` learns the mean (4) and the standard deviation (about 1.633) of the training values. `transform` applies them: (4 - 4) / 1.633 = 0 and (8 - 4) / 1.633 = 2.449. Nothing is re-learned from the new rows.

Q9

A fitted churn model has `classes_` equal to `[0 1]`. What is `model.predict_proba(X_test)[:, 1]`?

A) Each test customer's probability of churning

B) Each test customer's predicted label, 0 or 1

C) Each test customer's probability of staying

D) Both probabilities for the second test customer

> **Why:** `predict_proba` returns one row per customer and one column per class, in the order of `classes_`. Column 1 is class 1, churn.
> **B** is wrong: That is `predict`.
> **C** is wrong: That is column 0.
> **D** is wrong: That would be row 1, `[1]` or `[1, :]`.

Q10

Why does the scaler learn its means and standard deviations from the training rows only?

A) Fitting on fewer rows is faster

B) StandardScaler cannot fit a table that has the target in it

C) The test rows stand in for future customers, whose numbers could not have been seen when the model was built

D) `transform` raises an error on rows the scaler was fitted on

> **Why:** A score only means something if the test rows are treated exactly like new data. Fitting the scaler on them lets the test set shape the preprocessing: a silent leak.
> **A** is wrong: Speed is not the reason; the difference is negligible.
> **B** is wrong: The target is never passed to the scaler in the first place.
> **D** is wrong: `transform` works on any rows, including the training rows (the lesson transforms both).

Q11

What happens when this runs?

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
customers = pd.read_csv("data/customers_churn.csv")
LogisticRegression().fit(customers["days_since_last_order"], customers["churned"])
```

A) It trains a one-feature model

B) `ValueError`: X must be a table; `customers[["days_since_last_order"]]` fixes it

C) `KeyError`: the column does not exist

D) `ConvergenceWarning`: the feature is not scaled

> **Why:** Single brackets give a Series (one dimension). scikit-learn always wants X as a two-dimensional table, even with one feature; double brackets give a one-column DataFrame. The message: `ValueError: Expected a 2-dimensional container but got <clas...`
> **A** is wrong: It would, with double brackets.
> **C** is wrong: The column exists; the shape is the problem.
> **D** is wrong: It never gets as far as fitting.

Q12

What does this print?

```
from sklearn.linear_model import LogisticRegression
print(sorted(LogisticRegression().get_params())[:3])
```

`['C', 'class_weight', 'dual']`

> **Why:** `get_params()` lists the settings you can choose (hyperparameters), with no trailing underscore, before anything is fitted. Learned attributes such as `coef_` appear only after `fit`.

1.1 · THE SCIKIT-LEARN ESTIMATOR API: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
from sklearn.dummy import DummyClassifier
d = DummyClassifier(strategy="most_frequent").fit([[0], [1], [2]], [1, 1, 0])
print(d.predict([[5], [9]]))
```

```
[1 1]
```

> **Why:** The most frequent training label is 1 (two of three). The dummy ignores the features completely, so every new row gets 1.

P4

Predict the output, then run it.

```
from sklearn.preprocessing import MinMaxScaler
mm = MinMaxScaler().fit([[10], [20], [30]])
print(mm.transform([[20], [40]]).ravel())
```

```
[0.5 1.5]
```

> **Why:** The scaler learned min 10 and max 30, so x becomes (x - 10) / 20: 20 gives 0.5 and 40 gives 1.5. New values can fall outside 0 to 1, as in Lesson 1.1's Exercise 4.

LAB REFERENCE SOLUTIONS

◆ ENVIRONMENT CHECK: LABS/MODULE-01/L0_1-ORIENTATION/CHECK_ENVIRONMENT.PY

A tool, not an exercise. On a ready machine it prints the versions and ends with READY (paths shortened here; the first run takes up to a minute):

```
Python Machine Learning: environment check
============================================================
1. Python
  [ok]   Python 3.12.14         Linux x86_64
       running from: /path/to/pinecrest-ml/.venv/bin/python
2. Virtual environment
  [ok]   environment active     .venv
3. Course packages
  [ok]   numpy                  2.5.4
  [ok]   pandas                 3.0.6
  [ok]   matplotlib             3.11.2
  [ok]   scikit-learn           1.9.1
  [ok]   scipy                  1.18.1
  [ok]   imbalanced-learn       0.14.2
  [ok]   joblib                 1.6.0
  [ok]   skops                  0.16.0
  [ok]   xgboost                3.4.1
  [ok]   lightgbm               4.7.0
  [ok]   optuna                 5.0.0
  [ok]   shap                   0.53.0
  [ok]   lime                   0.2.0.1
  [ok]   umap-learn             0.5.12
  [ok]   torch                  2.14.1+cpu
  [ok]   torchvision            0.29.1+cpu
  [ok]   mlflow                 3.17.1
  [ok]   fastapi                0.143.2
  [ok]   uvicorn                0.54.0
  [ok]   httpx                  0.28.1
  [ok]   seaborn                0.13.2
  [ok]   pyarrow                25.0.1
  [ok]   jupyterlab             4.6.4
  [ok]   PyTorch device         CPU only (expected)
4. Course data
  [ok]   data folder            /path/to/pinecrest-ml/data
  [ok]   7 data files           all present
  [ok]   row counts             customers_churn.csv 6,000 customers, and the rest as ex
============================================================
Note: the deep-learning lessons (Module 10) download Fashion-MNIST, EuroSAT and pretra
      the first time they need them (about 175 MB). Nothing to do now.
READY for Python Machine Learning
```

On a machine without the environment it prints `[FIX]` lines and a numbered list (Python 3.12, activate, `python -m pip install -r requirements.txt` with the CPU PyTorch line first on Linux, unzip the data). seaborn, pyarrow and jupyterlab are notes, not fixes: no lesson needs them.

- DIAGNOSTIC PRACTICE: LABS/MODULE-01/L0_1-ORIENTATION/DIAGNOSTIC_STARTER.PY → DIAGNOSTIC_SOLUTION.PY

Six functions, one per diagnostic question; the checker at the bottom compares each with a tiny example, then the pandas ones run on the real customers. The solution prints:

```
PASS     D1 slicing           got [7, 8, 9]
PASS     D2 dictionaries      got {'Bend': 2, 'Boise': 1}
PASS     D3 default argument  got (72.0, 60.0)
PASS     D4 boolean filter    got 200.0
PASS     D5 groupby           got {'Basic': 0.667, 'Gold': 0.0}
PASS     D6 missing values    got 0.25
6 of 6 checks pass.

Mean 12-month spend of app users: $663.21
Churn rate by loyalty tier:
loyalty_tier
Basic     0.217
Silver    0.129
Gold      0.065
Share of customers with no age_group: 8.1%
```

Common sticking points: `items[-3:]` written as `items[len(items)-3:len(items)-1]` (drops the last item); `counts[v] += 1` without a default (KeyError on the first value); returning the filtered DataFrame instead of the column's mean in D4.

- LAB 0.1-A: LABS/MODULE-01/L0_1-ORIENTATION/MISSING_VALUES_STARTER.PY → MISSING_VALUES_SOLUTION.PY

The solution prints:

```
age_group               487
email_opens_90d         537
cancellation_reason    5375
dtype: int64

age_group missing: online 12.7%, store shoppers 4.4%
churn rate: email count missing 26.1%, email count present 15.6%

churned        0    1
has reason
False       5008  367
True           0  625

interactions without a star rating: 19,970 of 32,178 (62.1%)

Verdicts
  age_group           missing far more often online: depends on WHERE the customer sho
  email_opens_90d     missing customers churn more (26.1% vs 15.6%): the gap itself ca
  cancellation_reason filled only for churners (625 of 625): written AFTER the outcome
  rating              most buyers never rate: an empty rating means "no review", not "
```

What each TODO teaches: (1) the mean of `isna()` by group is a missing-share, and here missingness depends on where a customer shops (12.7% online vs 4.4% in stores); (2) a missing value can itself predict the target: 26.1% vs 15.6% churn, so Lesson 2.1's imputers can add a "was missing" flag; (3) the crosstab has a zero in the stayers' column: a column that exists only for one outcome is a leak; (4) a gap that means "no review". The four verdicts are the debrief.

- LAB 0.1-B: LABS/MODULE-01/L0_1-ORIENTATION/FRAME_TARGETS_STARTER.PY → FRAME_TARGETS_SOLUTION.PY

The solution prints:

```
1. reviews.csv  target: recommended  kind: classification
   distinct values: 2, share recommended: 65.5%
   always guessing 'recommends' would be right 65.5% of the time

2. daily_store_sales.csv (Boise rows)  target: sales  kind: regression (forecasting)
   days: 1,096, distinct sales values: 1,091
   mean $7,664.57, median $7,140.00, closed days (sales 0): 6

3. transactions.csv  target: is_fraud  kind: classification (a very rare class)
       payments  fraud  fraud_rate  always_legit_accuracy
month
1          8586     86      0.0100                 0.9900
2          7758     77      0.0099                 0.9901
3          8537     74      0.0087                 0.9913
4          8301     72      0.0087                 0.9913
5          8556     65      0.0076                 0.9924
6          8262     64      0.0077                 0.9923

Reading the numbers
  1. Two values, about two in three reviews recommend: a classification target, not too
  2. More than a thousand distinct amounts and 6 closed days: a quantity, so regressior
     real zeros the model must expect (holidays), not missing data.
  3. Fraud falls from 1.00% of payments in January to 0.77% in June, so "always legitir
     accurate every month while catching nobody. A score that rises for a useless model
```

Framing answers: `recommended` is a two-class target (not too unbalanced); Boise's `sales` is a quantity (regression, forecasting), with closed days as real zeros; `is_fraud` is a very rare class whose rate drifts down over time. The drift is the point to discuss: a do-nothing model's accuracy rises from 0.9900 to 0.9923 while it catches nobody.

- LAB 1.1-A: LABS/MODULE-01/L1_1-ESTIMATOR-API/FRAUD_BASELINE_STARTER.PY → FRAUD_BASELINE_SOLUTION.PY

The solution prints:

```
(50000, 9) fraud rate 0.876%
(35000, 9) (15000, 9) frauds: train 307, test 131
baseline accuracy: 0.9913  (frauds caught: 0 of 131)
logistic regression accuracy: 0.9937
flagged at the 0.5 cut-off: 40 payments, 38 of them fraud, 93 frauds missed

frauds among the 100 highest-risk payments: 53 (a random 100 would hold 0.9)

account_age_days            -0.47
hour                        -0.31
items                        0.11
country_mismatch             0.23
billing_shipping_mismatch    0.33
amount                       0.35
card_attempts                0.41
device_new                   0.58
velocity_24h                 0.80

random forest accuracy: 0.9935; frauds among its 100 riskiest: 58
```

Reading it: the model is 0.0024 more accurate than doing nothing, yet the 100 payments it ranks riskiest hold 53 frauds where a random 100 would hold 0.9. At the 0.5 cut-off it is very precise (38 of 40) but misses 93 of 131 frauds: the threshold trade-off Lesson 1.4 and Lesson 2.5 take apart. The negative `hour` weight is not "late payments are safe": fraud clusters at 0:00 to 5:59, and a single straight-line weight on hour 0 to 23 cannot express "night". A night flag is a feature (Lesson 2.3). The stretch forest (58 in its top 100) ranks better still.

- LAB 1.1-B: LABS/MODULE-01/L1_1-ESTIMATOR-API/NEW_CUSTOMERS_STARTER.PY → NEW_CUSTOMERS_SOLUTION.PY

The solution prints:

```
(4500, 10) (1500, 10)
mean_ equals X_train.mean():         True
scale_ equals X_train.std(ddof=0):   True
scale_ equals X_train.std() (ddof=1): False

C = 1.0, max_iter = 100, test accuracy 0.848

Avery    0.808
Casey    0.550
Blake    0.010

ValueError: The feature names should match those that were passed during fit.
fixed by selecting the training columns again: (3, 10)

C=0.01: test accuracy 0.853
new customers: {'Avery': 0.719, 'Blake': 0.018, 'Casey': 0.481}
```

Reading it: the scaler stores exactly the training means and the POPULATION standard deviations (`ddof=0`), which is why pandas' default `std()` (sample, `ddof=1`) does not match. New rows go through `transform` with the training numbers; `fit_transform` on three rows would scale them against each other and make the ranking meaningless. Since scikit-learn 1.0, a model fitted on a DataFrame remembers the column names and refuses another order rather than silently mixing features up. A smaller `C` means stronger regularization (Lesson 3.2): the probabilities move toward the middle, and here accuracy happens to rise slightly.

HANDOUT: LABS/MODULE-01/L1_1-ESTIMATOR-API/DEBUG_ESTIMATOR.PY (DELIBERATELY BROKEN)

1. `KeyError: 'churn'` → the column is `churned`.
2. `ValueError: Expected a 2-dimensional container but got <class 'pandas.Series'>...` → `X_train[["days_since_last_order"]]` (double brackets).
3. `NotFittedError` → delete the second `model = LogisticRegression()`, which threw the fitted model away.
4. No error: `scaler.fit_transform(X_test)` → `scaler.transform(X_test)`. With the bug the full model prints 0.847; fixed, 0.848. One feature: 0.841.

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 0.1 · YOUR ML TOOLKIT

1. **C** Supervised regression: the target (units) is a quantity, and past weeks give the right answers.
2. **A** Unsupervised learning: no column says which group anyone is in (clustering, Module 8).
3. **B** 0.25: the sum is 2 and there are 8 values; the mean of a 0/1 column is the share of 1s.
4. **C** Activate `.venv` in the new terminal: activation lasts for one window only.
5. **B** Evaluate: grading on held-out data against a do-nothing baseline.

LESSON 1.1 · THE SCIKIT-LEARN ESTIMATOR API

1. **A** `[[3.674]]`: fit learned mean 2 and standard deviation 0.816; (5 - 2) / 0.816 = 3.674.
2. **C** Accuracy hides most of the story: check whether the model ranks customers usefully (the top decile churned at 57%).
3. **B** `fit_transform` on X_train, then `transform` on X_test: the test data must be scaled with training numbers.
4. **A** A trailing underscore marks attributes learned from the data during `fit`.
5. **C** `(1500, 2)`: one row per customer, one column per class.

OPENING: READING THE ROOM

There is no retrieval warm-up in Session 1. The opening's two checks have their answers above: the environment check output (what READY looks like) and the diagnostic D1 to D6. Learners who end the opening NOT READY should leave with the numbered fix list in their workbook's Start Here box.

EXIT TICKET

1. **No `(.venv)`:** Python will not find the course packages (`ModuleNotFoundError`). Activate from the project folder: `.venv\Scripts\Activate.ps1` (Windows PowerShell) or `source .venv/bin/activate` (macOS, Linux).
2. **Estimator steps:** create with settings, `fit`, then `predict` (or `predict_proba`) or `score`. A transformer uses `fit` and `transform`, and is fitted on the training rows only, then used to transform both sets.
3. **0.848 vs 0.835:** accuracy alone cannot tell, because doing nothing already scores 0.835. Check the ranking: the riskiest tenth churned at 57% (vs 16.5% on average), and a top-50 list holds several times as many real churners as a random 50 (33 or 41 against 8.3 in the project). Useful.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 B ("use `import scikit_learn`").** Confuses install name and import name. Show `pip` installs `scikit-learn`, Python imports `sklearn`, and that the terminal import already worked.
- **Q2 A or C (churn / review sentiment as regression).** Thinks "stored as a number" means quantity. Repeat the halfway test.
- **Q4 A ("email_opens_90d, because of its gaps").** Treats missing data as the danger and misses the timing. Ask: "On the day you predict, is this column filled in?"
- **Q7 B ("prints [0 0 0 0]").** Expects an unfitted model to guess. scikit-learn refuses instead, loudly, which is a kindness.
- **Q8 written as `[0. 2.]` or `[0. 1.]`.** Re-fitted on the new rows in their head, or used min-max scaling. Write out mean 4 and standard deviation 1.633 from the training values.
- **Q10 A ("faster").** Has not yet connected the test set with the future. Use the new-hire picture: the manager's grading files must be unseen.
- **Exit ticket 3: "not useful, only 1.3 points better".** The most common answer in the room, and the reason Lesson 1.4 exists. Point back to the decile chart and the shortlist: this is the habit the course builds.

INSTRUCTOR KIT · ANSWER KEY

Session 2: Splits, Leakage and Cross-Validation

Answers and reasoning for every Session 2 item: the warm-up, both self-checks, the predict-then-run items, the lab reference outputs and debug handouts, the lesson pages' own quizzes and the exit ticket.

Session 2 | 12 self-check items | Confidential

WARM-UP: RETRIEVAL FROM SESSION 1

W1

On the churn table, what does always predicting "stays" score? And "never fraud" on the payments?

0.835 on churn (16.5% churn) and 0.9912 on fraud (0.88% fraud). Every score today is read against a do-nothing baseline.

W2

Which column of `customers_churn.csv` must never be a feature, and why?

`cancellation_reason`: it is filled in only after a customer has left, so on prediction day it is always empty. Today Lesson 1.2 measures what it does to a model (0.848 becomes 0.941).

W3

`train_test_split(X, y, ...)` returns four things. In which order?

`X_train, X_test, y_train, y_test`: the X parts, then the y parts. Today's Common Mistake 1 is the wrong order.

W4

Why is the scaler fitted on the training rows only, and then used to transform both sets?

The test rows stand in for future customers, whose numbers could not have been seen when the model was built. Today the same rule moves inside every cross-validation fold.

W5

Homework: what did `DummyClassifier(strategy="constant", constant=1)` score on the 1.1 test set?

0.165 (0.165): it calls everyone a churner, so it is right exactly for the 16.5% who churn. Accuracy rewards the majority class.

Homework check, the 1.1 shortlist project rerun with `random_state=7` in the split:

```
random_state=42  baseline 0.835  forest 0.873  top-50 hits 41
random_state= 7  baseline 0.835  forest 0.876  top-50 hits 45
```

The baseline is pinned by stratification; the forest's accuracy and its shortlist move with the split. That is Lesson 1.2's first experiment.

1.2 · SPLITS, BASELINES AND LEAKAGE: SELF-CHECK

Q1

Pinecrest wants to predict how a product it has never sold will sell, from `weekly_demand.csv`. Which evaluation matches how the model will be used?

A) A random 75/25 split of the rows with `stratify=demand["units_sold"]`

B) A time split: 2024 for training, 2025 for testing

C) A group split, `GroupShuffleSplit(..., groups=demand["product_id"])`, so that no test product appears in training

D) Cross-validate on all the rows, then confirm on the same rows

Why: The dress rehearsal must hold products the model has never seen, so every row of a product goes to one side. In the lesson the forest's error jumps from 10.23 units (random rows) to 17.84 (new products): the honest number for a launch.

A is wrong: Stratifying on a number crashes (Common Mistake 2), and a random split puts weeks of every product on both sides.

B is wrong: That answers a different question: next year's sales of the products Pinecrest already sells.

D is wrong: Grading on rows the model trained on is no evaluation at all.

Q2

What does this print?

```
from sklearn.model_selection import train_test_split
rows = list(range(1000))
rest, test = train_test_split(rows, test_size=0.20, random_state=42)
train, val = train_test_split(rest, test_size=0.25, random_state=42)
print(len(train), len(val), len(test))
```

`600 200 200`

> **Why:** The first call keeps 20% (200 rows) as the test set. The second takes 25% of the remaining 800 as validation (200) and leaves 600 for training: a 60/20/20 split from two calls, exactly the lesson's recipe.

Q3

You forecast tomorrow's sales for each store. Which candidate feature leaks the future?

A) Yesterday's sales, `shift(1)` within each store

B) A 7-day average built with `rolling(7, center=True)`

C) Sales on the same day last week, `shift(7)`

D) The day of the week

> **Why:** A centered window averages the three days AFTER each date too. On opening night those days do not exist. In Common Mistake 5 it makes an honest time split look like a $504 forecast instead of $854.
> **A** is wrong: Yesterday is known the evening before: a legitimate lag (and a cheap baseline).
> **C** is wrong: Last week is known: the seasonal-naive baseline is built from exactly this column.
> **D** is wrong: The calendar is known in advance.

Q4

What happens when this runs?

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
customers = pd.read_csv("data/customers_churn.csv")
X, y = customers[["tenure_months", "orders_12m"]], customers["churned"]
X_train, y_train, X_test, y_test = train_test_split(X, y, test_size=0.25, random_st
LogisticRegression().fit(X_train, y_train)
```

A) It fits a model on 4,500 customers

B) `KeyError` : `'churned'`

C) `NotFittedError` : the model was never fitted

D) `ValueError` : `y` should be one-dimensional; the second result is the 1,500 test rows of `X`

> **Why:** `train_test_split` returns the X parts first, then the y parts: `X_train, X_test, y_train, y_test`. Unpacked in the wrong order, `y_train` holds a 1,500 × 2 table. The message: `ValueError: y should be a 1d array, got an array of shape (1500, 2) instead.`
>
> **A** is wrong: It never gets that far.
>
> **B** is wrong: The column exists; the order of the results is the problem.
>
> **C** is wrong: The error comes from inside fit, before any predict.

Q5

Depths 3 and 4 tie on validation (0.864). You refit depth 3 on training plus validation rows and it scores 0.858 on the test set. What now?

A) Report 0.858 and keep depth 3: the test set has spoken once

B) Report 0.864: it is the best number you measured

C) Try depth 4 on the test set too and report the better of the two

D) Report the average of 0.864 and 0.858

> **Why:** The validation winner is partly the one that got lucky on those 1,200 rows, so a slightly lower test score is normal. The test score is the unbiased number, and only if nothing was chosen with it.
>
> **B** is wrong: The best of several validation scores is optimistic: you picked the maximum of noisy numbers.
>
> **C** is wrong: Choosing with the test set turns it into a second validation set; its score stops being unbiased.
>
> **D** is wrong: Averaging an optimistic number with an honest one gives a number that is neither.

Q6

What does this print? (The "yesterday" forecast's mean absolute error.)

```
import pandas as pd
sales = pd.Series([100, 120, 90, 130])
yesterday = sales.shift(1)
print((sales - yesterday).abs().mean())
```

```
30.0
```

> **Why:** `shift(1)` moves every value down a row, so the first day has no forecast (NaN) and is skipped by `mean`. The misses are 20, 30 and 40: their mean is 30.0. The naive baseline is that cheap.

1.2 · SPLITS, BASELINES AND LEAKAGE: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
from sklearn.model_selection import train_test_split
a, b = train_test_split(list(range(20)), test_size=5, random_state=0)
print(len(a), len(b))
```

```
15 5
```

> **Why:** An integer `test_size` is a COUNT, not a share: 5 rows test, 15 train (1.2 Common Mistake 3).

P2

Predict the output, then run it.

```
from sklearn.model_selection import train_test_split
y = [0] * 16 + [1] * 4
y_tr, y_te = train_test_split(y, test_size=0.25, stratify=y, random_state=0)
print(len(y_te), sum(y_te))
```

```
5 1
```

> **Why:** A quarter of 20 is 5 test rows; stratifying keeps the 20% share of ones, so exactly 1 of the 5 is a one.

1.3 · CROSS-VALIDATION: SELF-CHECK

Q7

What does this print?

```
from sklearn.model_selection import RepeatedStratifiedKFold
rcv = RepeatedStratifiedKFold(n_splits=4, n_repeats=2, random_state=42)
print(rcv.get_n_splits())
```

`8`

> **Why:** Four folds, repeated twice with different shuffles: eight rounds, so eight fits of every model you cross-validate with it (the lesson's 5 × 3 gives 15).

Q8

What happens when this runs?

```
from sklearn.model_selection import KFold
cv = KFold(n_splits=5, random_state=42)
```

A) It makes five shuffled folds with seed 42

B) `ValueError`: setting a `random_state` has no effect since `shuffle` is False

C) It makes five unshuffled folds and silently ignores the seed

D) `TypeError`: `KFold` has no `random_state` argument

> **Why:** An unshuffled splitter is not random, so scikit-learn refuses a seed rather than let you believe the folds are shuffled. Add `shuffle=True` (and for classification, use `StratifiedKFold`).

Q9

A colleague fills the gaps in `email_opens_90d` with the median of all 4,500 training rows, then runs `cross_val_score` on the filled table. What is wrong?

A) Nothing: the test set was never used

B) `cross_val_score` cannot handle a column that once had gaps

C) The median should come from all 6,000 rows to be more accurate

D) Each validation fold helped compute the median that filled it: put the filling step inside a pipeline so it is refitted on the training folds of every round

> **Why:** Same leak as pre-scaling: a step that learns from data saw the validation rows. With a median it is tiny; with steps that learn more (feature selection, encoding by churn rate) it can turn noise into 0.825 accuracy, as in Common Mistake 1.
> **A** is wrong: The validation folds play the test set's role inside cross-validation; they were seen.
> **B** is wrong: After filling, the column has no gaps; scikit-learn runs happily, which is the danger.
> **C** is wrong: That would let the real test set leak in as well.

Q10

`interactions.csv` has many rows per customer. You want to know how well a model will predict ratings for customers it has never seen. Which splitter?

A) `StratifiedKFold(5, shuffle=True, random_state=42)` on the rating

B) `TimeSeriesSplit(n_splits=5)` on the rows in file order

C) `GroupKFold(n_splits=5)` with `groups=customer_id`

D) `KFold(5)` without shuffling

> **Why:** Ask what will be new to the model: a customer. Group folds keep all of a customer's rows on one side, so every validation customer is a stranger. With row folds, nearly every validation customer also has rows in training (the workbook's worked example: 2,641 of 2,649).
> **A** is wrong: Shuffled row folds put each customer on both sides: an easier exam than reality.
> **B** is wrong: Time order is a different question (next month), and the file is sorted by customer, not by time.
> **D** is wrong: Unshuffled blocks of a file sorted by customer happen to keep most customers together, but by accident; and they mix it with whatever order the file has.

Q11

What does this print?

```
from sklearn.dummy import DummyRegressor
from sklearn.model_selection import KFold, cross_val_score
X = [[0]] * 6
y = [1, 2, 3, 4, 5, 6]
print(cross_val_score(DummyRegressor(), X, y, cv=KFold(3), scoring="neg_mean_absolu
```

`[-3. -0.5 -3. ]`

> **Why:** Unshuffled `KFold(3)` validates on [1, 2], [3, 4], [5, 6]. The dummy predicts the training mean each time: 4.5, 3.5, 2.5, so the misses average 3.0, 0.5 and 3.0. Scorers are "higher is better", so the errors come back negative: put a minus sign in front to report them.

Q12

On 15 paired folds the forest beats the depth-4 tree in all 15 (CV AUC 0.867 +/- 0.018). Fitted on all training rows, it then scores 0.865 on the test set. What do you conclude?

A) The estimate held: report 0.865 and keep the forest

B) The forest got worse on the test set; switch to the tree

C) Cross-validation was optimistic; rerun it with the test rows included

D) Report 0.867, because it is based on more folds

> **Why:** 0.865 lies well inside 0.867 +/- 0.018, and the lead over the tree was consistent fold by fold. A trustworthy estimate looks exactly like this: the single locked test number holds no surprise.
>
> **B** is wrong: A 0.002 dip is far smaller than the fold-to-fold spread, and switching would be choosing with the test set.
>
> **C** is wrong: Putting the test rows into cross-validation destroys the only unbiased check (Common Mistake 5).
>
> **D** is wrong: The CV mean is the estimate you chose with; the test score is the one you report.

1.3 · CROSS-VALIDATION: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
import numpy as np
from sklearn.model_selection import StratifiedKFold
y = np.array([0] * 8 + [1] * 4)
cv = StratifiedKFold(n_splits=4, shuffle=True, random_state=42)
print([int(y[val].sum()) for _, val in cv.split(np.zeros((12, 1)), y)])
```

```
[1, 1, 1, 1]
```

Why: Stratified folds keep the 1-in-3 share in every fold: each validation fold of 3 rows holds one 1.

P4

Predict the output, then run it.

```
import numpy as np
from sklearn.model_selection import TimeSeriesSplit
for train, val in TimeSeriesSplit(n_splits=3).split(np.zeros((10, 1))):
    print(len(train), val.tolist())
```

```
4 [4, 5] 6 [6, 7] 8 [8, 9]
```

Why: `TimeSeriesSplit(3)` on 10 rows validates on the last 6 in blocks of 2 and always trains on everything before: 4, 6, 8 training rows.

LAB REFERENCE SOLUTIONS

- LAB 1.2-A: LABS/MODULE-02/L1_2-SPLITS-AND-LEAKAGE/DEMAND_FORECAST_STARTER.PY → DEMAND_FORECAST_SOLUTION.PY

The solution prints:

```
(9984, 15) 2024-01-01 to 2025-12-22
time split: 5,088 train rows (last week 2024-12-30), 4,896 test rows (first week 2025-0
weeks on each side: 53 51
random split: 5,088 train rows, 50% of them from 2025 (the 'future')

forest MAE, random split: 10.45 units per week
forest MAE, time split:   10.27 units per week
the random split does not flatter the forest here (difference +0.18)
2025 rows without a value 52 weeks back: 0

MAE on 2025, units per week (lower is better):
forest (time split)            10.27
each product's 2024 average    12.95
same week last year            13.81
training mean                  17.13

strongest free baseline: each product's 2024 average (12.95); the forest beats it

forest MAE by category, 2025:
category
Camp Kitchen    13.2
Footwear        12.7
Snow            11.0
```

What each TODO teaches: (1) a time split needs a look at where the line falls: 2024-12-30 is a Monday of 2024, so 2024 holds 53 weeks and 2025 only 51; (2) a random split puts half of the training rows in the "future"; (3) on this data that does NOT flatter the forest (10.45 vs 10.27), because weekly demand carries little week-to-week memory once season and price are known, unlike daily sales in the lesson ($613 vs $854): measure, do not assume; (4) and (5) the strongest free baseline is each product's usual level (12.95), not the training mean (17.13) and not the same week last year (13.81, a single noisy week); the forest beats it, so it earns its place. The stretch shows where it misses most (Camp Kitchen, Footwear, Snow).

- LAB 1.2-B: LABS/MODULE-02/L1_2-SPLITS-AND-LEAKAGE/FRAUD_VALIDATION_STARTER.PY → FRAUD_VALIDATION_SOLUTION.PY

The solution prints:

```
train      30,000 rows  263 frauds
validation 10,000 rows   87 frauds
test       10,000 rows   88 frauds

random 100 validation payments hold 0.9 frauds on average
logistic regression  frauds in the validation top 100: 45
tree (depth 3)       frauds in the validation top 100: 30
tree (depth 8)       frauds in the validation top 100: 34
random forest        frauds in the validation top 100: 47
chosen on validation: random forest

random forest: 41 frauds in the test top 100 (validation said 47; a random 100 holds 0.
for comparison, 'never fraud' is 0.9912 accurate and fills the queue with nobody

Fraud review queue: random forest, chosen on 10,000 validation payments; test top 100 h
```

Reading it: stratifying twice gives each part its share of the 438 frauds (263 / 87 / 88). Accuracy is useless for a review queue ("never fraud" is 0.9912 accurate), so the candidates are judged by frauds in the 100 riskiest validation payments. The forest wins validation (47) narrowly over logistic regression (45), and the test set then says 41: the validation winner is partly the lucky one, exactly as the depth-3 tree went from 0.864 to 0.858 in the lesson. Going back now to try logistic regression on the test set would turn it into a second validation set. Note for questions: a shallow tree gives many payments the same score, so its top 100 contains ties broken by row order; that is one reason it ranks worse. Expected TODO 4 sentences: the test number is lower because the winner on validation was partly lucky on those 10,000 payments; picking again with the test set would make its score optimistic.

- HANDOUT: LABS/MODULE-02/L1_2-SPLITS-AND-LEAKAGE/DEBUG_LEAKY_SPLIT.PY (DELIBERATELY BROKEN)

1. `ValueError: The least populated classes in y have only 1 member ...` → stratify on the target: `stratify=y` (not `customers["spend_12m"]`).
2. `ValueError: Found input variables with inconsistent numbers of samples: [5975, 25]` → unpack in the standard order: `X_train, X_test, y_train, y_test`.
3. No error: `test_size=25` is 25 customers, not 25%. With the other silent bug still in, it reports a perfect 1.000; fix: `test_size=0.25`.
4. No error: `has_reason` in the features is the target leak (0.941 on 1,500 customers). Drop it.

The two silent bugs, measured:

```
25 test customers, with the leak: 1.000
1,500 test customers, with the leak: 0.941
```

Fixed, the script prints:

```
test customers: 1500 (churn rate 0.165)
baseline accuracy 0.835
logistic regression accuracy 0.848
```

- LAB 1.3-A: LABS/MODULE-02/L1_3-CROSS-VALIDATION/FRAUD_FOLDS_STARTER.PY → FRAUD_FOLDS_SOLUTION.PY

The solution prints:

```
train (Jan-May): 41,738 payments, 374 frauds; June locked: 8,262
KFold, no shuffle  frauds per validation fold [83, 84, 71, 75, 61]
StratifiedKFold    frauds per validation fold [75, 75, 75, 74, 75]

baseline             validation AUC 0.500 +/- 0.000   train AUC 0.500
logistic regression  validation AUC 0.854 +/- 0.023   train AUC 0.857
tree (depth 5)       validation AUC 0.776 +/- 0.020   train AUC 0.791

tree ahead of logistic regression in 0 of 5 folds, mean difference -0.078

logistic regression, next-month AUC (Feb, Mar, Apr, May): [0.864, 0.844, 0.86, 0.881] r
June AUC 0.874   (shuffled CV said 0.854, time-ordered CV said 0.862)

out-of-fold top 400 of 41,738: 186 frauds (a random 400 would hold 3.6)
```

Reading it: (1) the rows are in time order and the fraud rate drifts down, so unshuffled `KFold` puts 83 frauds in the January block and 61 in May's, while stratified folds hold 74 or 75 each; (2) the pipeline keeps the scaler inside the folds; train and validation AUC are close for logistic regression (0.857 vs 0.854), the depth-5 tree is weaker (0.776); (3) the tree loses all 5 paired folds by 0.078 on average: a gap far larger than the spread, so this one is real; (4) next-month folds (train on all earlier months, validate on the next) average 0.862; (5) June scores 0.874, inside one spread of both estimates, a little closer to the time-ordered one. The stretch: 186 frauds among the 400 riskiest out-of-fold payments, against 3.6 for a random 400.

- LAB 1.3-B: LABS/MODULE-02/L1_3-CROSS-VALIDATION/REGULARIZATION_CHECK_STARTER.PY → REGULARIZATION_CHECK_SOLUTION.PY

The solution prints:

```
(4500, 10)
C=0.01 accuracy, pre-scaled: 0.8493   in a pipeline: 0.8491
C=1.0   accuracy 0.852 +/- 0.008   AUC 0.830 +/- 0.020
C=0.01  accuracy 0.849 +/- 0.007   AUC 0.830 +/- 0.019
accuracy  C=0.01 ahead in  2 of 15 folds, behind in 11; mean difference -0.0026
roc_auc   C=0.01 ahead in  9 of 15 folds, behind in  6; mean difference +0.0006

C=0.001  train AUC 0.827   validation AUC 0.825
C=0.01   train AUC 0.833   validation AUC 0.830
C=0.1    train AUC 0.833   validation AUC 0.830
C=1.0    train AUC 0.833   validation AUC 0.829
C=10.0   train AUC 0.833   validation AUC 0.829

Verdict: on accuracy C=0.01 is behind C=1.0 by 0.0026 on average (ahead in 2 of 15 fol
on AUC the gap is +0.0006, against a fold-to-fold spread of 0.020: a tie. Session 1's
```

Reading it: pre-scaling leaks, but for a scaler on 4,500 rows the leak is in the fourth decimal place (0.8493 vs 0.8491); the habit matters for steps that learn more. On 15 identical folds C=0.01 is BEHIND C=1.0 on accuracy in 11 of 15 folds (mean -0.0026), the opposite of Session 1's single test split (0.853 vs 0.848), and tied on AUC (+0.0006 against a spread of 0.020). Only C=0.001 is clearly worse (it underfits); no value of C overfits, since train and validation AUC stay within 0.003. The lesson: one split's opinion, even on the test set, is inside the wobble.

- HANDOUT: LABS/MODULE-02/L1_3-CROSS-VALIDATION/DEBUG_CROSS_VALIDATION.PY (DELIBERATELY BROKEN)

1. `ValueError: Setting a random_state has no effect since shuffle is False...` → `StratifiedKFold(n_splits=5, shuffle=True, random_state=42)` (or at least `shuffle=True`).
2. `InvalidParameterError: The 'scoring' parameter of cross_val_score must be a str among ...` → `scoring="roc_auc"` (underscore, not hyphen).
3. No error: the scaler is fitted on all of `X` before cross-validation → `make_pipeline(StandardScaler(), LogisticRegression())` on the raw features.
4. No error: cross-validating `X` and `y` (6,000 rows, test set included) → `X_train`, `y_train` only; the message even says "training customers".
5. No error: `scores.max()` reports the luckiest fold → `scores.mean()`.

With the two loud bugs fixed (`KFold(5, shuffle=True, random_state=42)`, `"roc_auc"`) but the three silent ones left in, it reports:

```
cross-validated on 6000 training customers, 5 folds
ROC AUC 0.850 +/- 0.013
```

Fully fixed with the course splitter:

```
cross-validated on 4500 training customers, 5 folds
ROC AUC 0.829 +/- 0.025
```

A learner who fixes bug 1 with `KFold(n_splits=5, shuffle=True, random_state=42)` instead of `StratifiedKFold` gets slightly different folds (AUC about 0.830 ± 0.015 on the training rows); accept it, then ask which splitter the course uses for classification and why.

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 1.2 · SPLITS, BASELINES AND LEAKAGE

1. **B** `7 3`: `test_size=0.3` of 10 items puts 3 in the second result; the first holds the other 7.
2. **C** Train on 2024, test on 2025: the model will forecast the future of products Pinecrest already sells, so the dress rehearsal is a time split.
3. **A** Target leakage: a refund is recorded after the return happens, so it will not exist when the prediction is made (0.97 alone is the alarm).
4. **B** 0.866, the test score: the best of 12 validation scores is optimistic; the test set was not used to choose.
5. **C** When many rows share an entity and the model must work for new entities: every row of a group goes to one side.

LESSON 1.3 · CROSS-VALIDATION

1. **B** Five times: one fit per fold; `cross_val_score` does not refit on all rows at the end.
2. **A** `[3, 3, 2, 2]`: 10 rows in 4 folds; the first 10 mod 4 = 2 folds get one extra row.
3. **C** The scaler saw the validation folds: scale inside a pipeline so it is refitted on the training folds of each round.
4. **C** Time-ordered folds: train on earlier weeks, validate on the next quarter, as the forecast will be used.
5. **A** The 0.004 gap is within the noise (spread about 0.015): check the paired folds and prefer the simpler model; never break a tie with the test set.

EXIT TICKET

1. **Training** fits every candidate; **validation** compares candidates and chooses; **test** grades the final choice, opened once at the very end (or cross-validation on the training rows replaces the fixed validation set).
2. **$854**, the time split: it is how the forecast will be used (trained on the past, asked about the future). The random split is lower because almost every test day has its neighbors, the day before and the day after, in training, and sales have momentum; tomorrow's neighbors are never available.

3. **So the scaler is refitted inside every round on the training folds only.** Pre-scaling lets every validation fold help compute the mean and standard deviation it is scaled with: a leak, tiny for a scaler and large for steps that learn more (selecting features, encoding categories).

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 B (a time split for never-sold products).** Knows "time split" is the honest choice and applies it everywhere. Ask: "What will be new to the model on opening night: a date, or a product?"
- **Q3 A or C (yesterday or last week as the leak).** Thinks any use of past sales is cheating. Ask: "Is yesterday known the evening before?" Shifted columns that only look backward are legal; the centered window looks forward.
- **Q4 A (it fits).** Has not internalized the order of the four results. Have them say "X, X, y, y" out loud and print shapes after every split.
- **Q5 C (try depth 4 on the test set too).** The single most tempting move in the session. Show what the test set becomes: a second validation set with no unbiased number left.
- **Q6 written as 22.5 or NaN.** Counted the first row (NaN) or divided by 4. `mean` skips NaN: three misses.
- **Q9 A ("the test set was not used").** Thinks leakage only means the test set. Inside cross-validation, the validation folds play the test set's role.
- **Q11 written as positive numbers.** Forgot that scorers are "higher is better": error metrics come back negated.
- **Q12 B (switch to the tree).** Reads a 0.002 dip as a verdict and would choose with the test set. Point at the spread, 0.018.

INSTRUCTOR KIT · ANSWER KEY

Session 3: Metrics and Preprocessing

Answers and reasoning for every Session 3 item: the warm-up, both self-checks, the predict-then-run items, the lab reference outputs, the lesson pages' own quizzes and the exit ticket.

Session 3 | 12 self-check items | Confidential

WARM-UP: SESSION 2 IN FIVE QUESTIONS

Retrieval, not graded. The fifth question checks the 1.3 "Before the Next Lesson" task.

W1

Why does every churn split in this course pass `stratify=y`?

It keeps the churn share (16.5%) the same in the training and test parts. Without it the class balance wobbles from split to split, and with it the baseline and every score wobble too.

> **From:** Lesson 1.2.

W2

In the honest-evaluation project, a depth-2 tree on the single column `has_reason` scored 0.942 on validation against a baseline of 0.835. What do you do, and what do you check first?

Treat it as a leak suspect: far too good for one column. Check when and how it is recorded (it is filled only when an account is closed, after the outcome), then drop it. A high score alone is not proof; the recording process is.

> **From:** Lesson 1.2.

W3

Why cross-validate `make_pipeline(StandardScaler(), LogisticRegression())` instead of scaling `X_train` once and cross-validating the model on the scaled table?

The pipeline refits the scaler on the training folds of every round. Scaling first lets each validation fold shape the mean and standard deviation: a small, hidden leak, and a big one when the preprocessing learns more (today's imputers and encoders).

> **From:** Lesson 1.3.

W4

What does `cross_val_predict(model, X_train, y_train, cv=cv, method="predict_proba")[:, 1]` give you, and why is it honest?

One churn probability per training customer, each from the fold model that did NOT train on that customer. Today's project chooses its decision threshold from exactly these out-of-fold probabilities, so the test set stays sealed.

> **From:** Lesson 1.3.

W5

You reran the bake-off with `scoring=["accuracy", "roc_auc", "f1"]`. What did the baseline score on accuracy and on F1, and why so different?

Accuracy 0.835, ROC AUC 0.5, F1 0.0. The baseline never predicts churn, so it has no true positives: recall is 0 and precision is 0/0 (scikit-learn reports 0 with a warning), so F1 is 0. Accuracy rewards the 83.5% it gets right by saying "stays". That gap is today's first topic.

> **From:** Homework check (1.3, Before the Next Lesson).

W5, from a real run (the bake-off's baseline, 5 stratified folds, three scorers):

```
{'test_accuracy': 0.835, 'test_roc_auc': 0.5, 'test_f1': 0.0}
```

1.4 · METRICS FOR CLASSIFICATION AND REGRESSION: SELF-CHECK

Q1

A churn model's confusion matrix on a test set: TN 880, FP 60, FN 20, TP 40. What are its precision and recall?

A) precision 0.667, recall 0.400

B) precision 0.400, recall 0.667

C) precision 0.978, recall 0.936

D) precision 0.920, recall 0.400

Why: Precision = TP / (TP + FP) = 40 / 100 = 0.400: of the customers flagged, four in ten were really leaving. Recall = TP / (TP + FN) = 40 / 60 = 0.667: two thirds of the real churners were found.

A is wrong: Precision and recall swapped: exactly what swapping the arguments of a metric function does, silently.

C is wrong: Those treat "stays" as the positive class: 880 / 900 = 0.978 and 880 / 940 = 0.936.

D is wrong: 0.920 is the accuracy (920 of 1,000), which counts the 880 easy true negatives.

Q2

What does this print?

```
from sklearn.metrics import precision_score, recall_score
y_true = [1, 0, 1, 1, 0, 0, 0, 1]
y_pred = [1, 1, 0, 1, 0, 0, 0, 0]
print(round(precision_score(y_true, y_pred), 3), recall_score(y_true, y_pred))
```

`0.667 0.5`

Why: Truth first, prediction second. Three customers are flagged (positions 0, 1, 3); two of them really churned, so precision is 2 / 3 = 0.667. Four customers really churned (0, 2, 3, 7) and two of them were flagged, so recall is 2 / 4 = 0.5. scikit-learn 1.9 returns plain floats, so `round()` works and `.round()` would not.

Q3

You want the decision threshold that maximizes the retention campaign's profit. Where should the probabilities you search over come from?

A) The test set: it is the most realistic data you have

B) The training rows, from the fitted model's own `predict_proba(X_train)`

C) The training rows, out of fold: `cross_val_predict(..., method="predict_proba")` or `TunedThresholdClassifierCV`

D) Nowhere: always use 0.5, because the model was trained for it

Why: Each training customer gets a probability from a model that never saw them, so the profit curve is honest, and the test set stays untouched for one final check. TunedThresholdClassifierCV does exactly this internally.
A is wrong: Choosing the threshold on the test set is tuning on the test set: the final profit is then flattered.
B is wrong: A model scored on the rows it learned from is over-confident (a forest especially), so the curve and its peak are wrong.
D is wrong: 0.5 is a default, not a decision. The lesson's tuned threshold (0.245) earns $3,555 against $2,280 at 0.5 with the same model.

Q4

What does this print?

```
from sklearn.metrics import roc_auc_score
y = [0, 0, 1, 1]
print(roc_auc_score(y, [0, 1, 1, 1]), roc_auc_score(y, [0.2, 0.6, 0.7, 0.9]))
```

A) `0.75 1.0`

B) `1.0 1.0`

C) `0.5 1.0`

D) `ValueError`: ROC AUC needs probabilities

Why: ROC AUC is the share of (positive, negative) pairs in which the positive scores higher, ties counting one half. With hard labels the second negative ties with both positives: (1 + 1 + 0.5 + 0.5) / 4 = 0.75. With probabilities every positive beats every negative: 1.0. Labels throw ranking information away, without any error.
B is wrong: The labels cannot separate the negative scored 1 from the two positives.
C is wrong: 0.5 would be a coin flip; the labels still separate one negative correctly.
D is wrong: Nothing checks that the scores are probabilities, which is why this mistake is silent.

Q5

A retention offer costs $20 and reaching a real churner is worth $80. If the model's probabilities were perfectly calibrated, whom should Pinecrest contact?

A) Everyone predicted to churn at the default threshold of 0.5

B) Nobody: each offer costs money

C) The customers with a churn probability of at least 0.80

D) The customers with a churn probability of at least 0.25

Why: An offer to a customer with probability p is worth 80 × p on average and costs 20, so it pays when p ≥ 20 / 80 = 0.25, the break-even probability. Real probabilities are only roughly calibrated, which is why the lesson measures profit on out-of-fold probabilities instead of trusting the formula.

A is wrong: 0.5 ignores the costs: everyone between 0.25 and 0.5 is a profitable call left unmade.

B is wrong: Contacting nobody earns 0; contacting the right customers earns more.

C is wrong: 0.80 is the value, not the break-even; it would skip most profitable calls.

Q6

What does this print?

```
from sklearn.metrics import mean_absolute_error, root_mean_squared_error
y_true = [10, 20, 30, 40]
y_pred = [12, 18, 30, 50]
print(mean_absolute_error(y_true, y_pred), round(root_mean_squared_error(y_true, y_
```

`3.5 5.2`

Why: The errors are 2, 2, 0 and 10. MAE = 14 / 4 = 3.5 units. RMSE = √((4 + 4 + 0 + 100) / 4) = √27 ≈ 5.2: the one big miss counts far more once squared, so RMSE is above MAE.

1.4 · METRICS FOR CLASSIFICATION AND REGRESSION: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
import numpy as np
proba = np.array([0.9, 0.7, 0.4, 0.3, 0.2, 0.1])
truth = np.array([1, 0, 1, 1, 0, 0])
for t in [0.5, 0.25]:
    flagged = (proba >= t).astype(int)
    print(t, flagged.sum(), (flagged & truth).sum())
```

```
0.5 2 1 0.25 4 3
```

Why: At 0.5 two customers are flagged (0.9 and 0.7) and one of them really churned. At 0.25 four are flagged and three of them churned. Recall rises from 1/3 to 3/3, and here precision rises too (1/2 to 3/4), because the two extra flags happen to be churners: "lower threshold, lower precision" is the usual direction, not a law, and six customers are a tiny sample.

P2

Predict the output, then run it.

```
from sklearn.metrics import mean_absolute_error, r2_score
y = [2, 4, 6, 8]
print(r2_score(y, [5, 5, 5, 5]), mean_absolute_error(y, [5, 5, 5, 5]))
```

```
0.0 2.0
```

Why: Predicting the mean (5) explains none of the variation: R^2 is exactly 0.0. The misses are 3, 1, 1 and 3, so the MAE is 2.0.

2.1 · IMPUTE, SCALE AND ENCODE: SELF-CHECK

Q7

What does this print?

```
import pandas as pd
from sklearn.preprocessing import OneHotEncoder
enc = OneHotEncoder(sparse_output=False, handle_unknown="ignore")
enc.fit(pd.DataFrame({"store": ["Bend", "Boise", "Bend"]}))
print(enc.transform(pd.DataFrame({"store": ["Boise", "Reno"]})))
```

A) `[[0. 1. 0.] [0. 0. 1.]]` (a new Reno column)

B) `ValueError`: Found unknown categories ['Reno']

C) `[[0. 1.] [0. 0.]]`

D) `[[1. 0.] [0. 0.]]`

> **Why:** The encoder learned two categories, Bend and Boise, in sorted order. Boise gets a 1 in the second column. Reno was never seen, and with `handle_unknown="ignore"` it becomes all zeros: "none of the stores I know".
> **A** is wrong: An encoder never adds columns after fit; the output always has the training columns.
> **B** is wrong: That is the default, `handle_unknown="error"`.
> **D** is wrong: Columns follow the sorted categories: Bend first, Boise second.

Q8

What does this print?

```
import numpy as np
from sklearn.impute import SimpleImputer
imp = SimpleImputer(strategy="mean", add_indicator=True).fit([[2.0], [np.nan], [4.0
print(imp.transform([[np.nan], [1.0]]))
```

`[[5. 1.] [1. 0.]]`

> **Why:** The imputer learns the mean of the three known values, (2 + 4 + 9) / 3 = 5, and adds an indicator column because the training column had a gap. The first new row was missing: filled with 5 and flagged 1. The second was present: kept (1.0) and flagged 0.

Q9

After `StandardScaler`, `days_since_last_order` still has a skew of 1.71, the same as the raw column. Why?

A) Scaling changes the units, not the shape; a log transform such as `np.log1p` changes the shape

B) The scaler was fitted on the test rows instead of the training rows

C) StandardScaler only removes skew when there are no outliers

D) Skew cannot be measured on scaled data

> **Why:** (x − mean) / sd moves and stretches every value by the same amounts, so the histogram keeps its shape and only the numbers on the axis change. `np.log1p` squeezes the long tail: skew −0.19.
> **B** is wrong: Fitting on other rows would change the center, not the shape.
> **C** is wrong: No scaler removes skew; RobustScaler only limits the pull of outliers on the center and spread.
> **D** is wrong: Skew is unit-free, which is exactly why it comes out identical.

Q10

What does this print?

```
from sklearn.preprocessing import OrdinalEncoder
enc = OrdinalEncoder().fit([["Small"], ["Medium"], ["Large"]])
print(enc.transform([["Small"], ["Large"]]).ravel())
```

A) `[0. 2.]`

B) `[2. 1.]`

C) `[1. 0.]`

D) `[2. 0.]`

> **Why:** Without `categories=`, OrdinalEncoder sorts alphabetically: Large 0, Medium 1, Small 2. So Small is 2 and Large is 0, the reverse of the size order, and nothing warns you. Spell the order out: `OrdinalEncoder(categories=[["Small", "Medium", "Large"]])`.
> **A** is wrong: That is the order you meant (Small 0, Large 2), which the encoder does not know.
> **B** is wrong: Medium is 1, but it is not asked for.
> **C** is wrong: Neither code matches the alphabetical order.

Q11

In a `ColumnTransformer`, the `email_opens_90d` column needs its gaps filled AND then scaling. How do you do it?

A) Two branches on the same column: one with the imputer, one with the scaler

B) One branch whose transformer is `make_pipeline(SimpleImputer(...), StandardScaler())`

C) Put the scaler in the branch and fill the gaps in pandas on the whole table before splitting

D) Set `remainder="passthrough"` so the scaler runs after the imputer

> **Why:** Branches run side by side, each on the raw columns it is given. Steps that must run one after another go into a small pipeline inside one branch: fit fits the imputer, passes its output to the scaler, and fits that too.
> **A** is wrong: Two branches would produce two separate copies of the column (one filled but unscaled, one scaled but still with gaps, which fails).
> **C** is wrong: Filling before the split lets the test rows shape the fill value: a leak, and no answer for next month's customers.
> **D** is wrong: remainder only decides what happens to unlisted columns.

Q12

What does this print?

```
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
df = pd.DataFrame({"spend": [10.0, 20.0, 30.0], "store": ["Bend", "Boise", "Bend"],
                   "customer_id": ["C1", "C2", "C3"]})
ct = ColumnTransformer([("num", StandardScaler(), ["spend"]),
                        ("cat", OneHotEncoder(sparse_output=False), ["store"])])
print(ct.fit_transform(df).shape, ct.get_feature_names_out().tolist())
```

`(3, 3) ['num__spend', 'cat__store_Bend', 'cat__store_Boise']`

> **Why:** Each branch fits on its own columns and the outputs sit side by side: one scaled number plus one column per store (Bend, Boise), named branch__column. `customer_id` is in no branch, so the default `remainder="drop"` leaves it out: a safety feature, since an unlisted column (an ID, or a leak) cannot sneak into the model.

2.1 · IMPUTE, SCALE AND ENCODE: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
import numpy as np
from sklearn.impute import SimpleImputer
imp = SimpleImputer(strategy="most_frequent").fit([["25-34"], [np.nan], ["25-34"],
print(imp.statistics_, imp.transform([[np.nan]]).ravel())
```

```
['25-34'] ['25-34']
```

> **Why:** `most_frequent` on text is fine (the median would raise). Two of the three known values are 25-34, so it is learned and used for the gap. `statistics_` keeps it.

P4

Predict the output, then run it.

```
from sklearn.preprocessing import RobustScaler
print(RobustScaler().fit_transform([[1], [2], [3], [4], [100]]).ravel())
```

```
[-1. -0.5 0. 0.5 48.5]
```

> **Why:** RobustScaler subtracts the median (3) and divides by the interquartile range (4 − 2 = 2). The ordinary values land between −1 and 0.5; the outlier 100 lands at 48.5, far out, but it did not move the center or the spread.

LAB REFERENCE SOLUTIONS

- LAB 1.4-A: LABS/MODULE-03/L1_4-METRICS/FRAUD_REVIEW_STARTER.PY → FRAUD_REVIEW_SOLUTION.PY

The solution prints:

```
test payments 15,000, frauds 131

TN 14867  FP 2  FN 93  TP 38
precision 0.950  recall 0.290  F1 0.444
In words: of 40 payments sent to review, 38 are fraud; 93 of 131 frauds slip through.

ROC AUC 0.848   average precision 0.444   fraud rate (AP of a random ranking) 0.0087

break-even probability 0.027
cheapest threshold on the training payments (out-of-fold): 0.04, cost 52,320 (threshold

threshold 0.04    reviews    390   precision 0.195   recall 0.580   cost  24,525
threshold 0.50    reviews     40   precision 0.950   recall 0.290   cost  35,275
review everything reviews 15,000   precision 0.009   recall 1.000   cost 150,000
review nothing    reviews      0   precision 0.000   recall 0.000   cost  49,125

FixedThresholdClassifier sends 390 test payments to review
TunedThresholdClassifierCV threshold 0.040, test cost 24,485
```

What each TODO teaches: (1) at 0.5 the model is very precise (38 of 40) and misses most fraud (93 of 131): the confusion matrix makes the trade visible where accuracy (0.9937) hid it; (2) ROC AUC 0.848 looks comfortable, average precision 0.444 is fifty times the random baseline of 0.0087 and still says that a typical alert list mixes fraud with legitimate orders; (3) a COST is minimized, a profit maximized: same idea, opposite sign; (4) out-of-fold probabilities keep the test set sealed; the break-even is 10 / 375 = 0.027 and the cheapest training threshold, 0.04, sits next to it; (5) the chosen threshold sends ten times as many payments to review as 0.5 (390 against 40) and is still $10,750 cheaper on the test set, because a missed fraud costs as much as 37 reviews; (6) `FixedThresholdClassifier` carries the threshold with the model. The stretch needs `greater_is_better=False`, because scikit-learn's search always maximizes: the scorer then returns the negative cost.

- LAB 1.4-B: LABS/MODULE-03/L1_4-METRICS/STORE_FORECAST_STARTER.PY → STORE_FORECAST_SOLUTION.PY

The solution prints:

```
train 2,924 rows (2023-2024), test 1,460 rows (2025), closed days in 2025: 8

overall mean        MAE  1,967   RMSE  2,704   R2 -0.048   median error 1,520
store-weekday mean  MAE  1,292   RMSE  1,782   R2  0.545   median error   942
yesterday           MAE  1,350   RMSE  1,882   R2  0.493   median error   926
same day last week  MAE  1,072   RMSE  1,628   R2  0.620   median error   692

MAPE, all 2025 days : 1.33e+17
MAPE, open days only: 0.168
MAE / mean sales    : 0.173

                mae  mean_sales  mae_share
store
Boise      1355.612    8341.766      0.163
Flagstaff  1189.481    6560.560      0.181
Bend        946.052    5527.230      0.171
Asheville   798.322    4380.470      0.182

      date     store    sales  last_week        event
2025-11-29 Flagstaff 15847.80    6078.79 Black Friday
2025-11-30     Boise 17267.74    8023.04 Black Friday
2025-12-25 Flagstaff     0.00    8368.45       Closed
```

Reading it: copying the same weekday last week (seasonal naive) beats the other three by every metric, and beats copying yesterday because the weekly cycle (Saturday high, Tuesday low) is strong. The overall mean has a NEGATIVE R^2 (−0.048): sales grew from 2023-2024 to 2025, so last years' mean is worse than this year's mean. MAPE over all 2025 days is 1.33e+17 because eight closed days have true sales of 0; on open days it is a sensible 0.168, and MAE divided by mean sales (0.173) is the scale-free number that survives zeros. Boise has the largest error in dollars and the smallest relative error: report both when stores differ in size. The three biggest misses (two Black Friday weekend days and a closed Christmas Day) are why RMSE (1,628) is so much larger than MAE (1,072).

HANDOUT: LABS/MODULE-03/L1_4-METRICS/DEBUG_METRICS.PY (DELIBERATELY BROKEN)

1. `ValueError: y should be a 1d array, got an array of shape (1500, 2) instead.` → `logreg.predict_proba(X_test)[:, 1]` (one score per row: the positive class).
2. `ValueError: pos_label=1 is not a valid label. It should be one of ['churned' 'stayed']` → `precision_score(text_true, text_pred, pos_label="churned")`.
3. No error: `recall_score(forest.predict(X_test), y_test)` has the arguments swapped and prints 0.705, which is the PRECISION. Truth first: `recall_score(y_test, forest.predict(X_test))` gives 0.395.
4. No error: `roc_auc_score(y_test, forest.predict(X_test))` uses hard labels and prints 0.681. With `forest.predict_proba(X_test)[:, 1]` it is 0.865. Fixed, the file prints 0.823, 0.705, 0.395, 0.865.

LAB 2.1-A: LABS/MODULE-03/L2_1-PREPROCESSING/FRAUD_COLUMNS_STARTER.PY → FRAUD_COLUMNS_SOLUTION.PY

The solution prints:

```
(35000, 10) (15000, 10) | text columns: ['category']

fraud rate by category (training rows):
category
Gift Cards      0.0546
Tents           0.0109
Climbing        0.0095
Snow            0.0092
Accessories     0.0077
Apparel         0.0072
Sleeping        0.0064
Packs           0.0064
Camp Kitchen    0.0061
Footwear        0.0057

(35000, 19) (15000, 19)
['num__card_attempts', 'num__velocity_24h', 'cat__category_Accessories', 'cat__category

9 numeric columns        ROC AUC 0.848   average precision 0.444
+ category (19 columns)  ROC AUC 0.843   average precision 0.463
weight of cat__category_Gift Cards: 0.95 (largest of all: cat__category_Gift Cards)

fill value for account_age_days: 447 days
categories learned: 10

P2's category columns (Bikes was never seen): 0.0
P2's account age after imputation (scaled): -0.311
P1    1.000
P2    0.001
P3    0.000
```

What each TODO teaches: (1) look before you encode: Gift Cards carry a fraud rate five to ten times the others, so the column deserves a place; (2) one ColumnTransformer, impute-then-scale chained in the numeric branch, one-hot for the text, pandas output; 9 numbers + 10 categories = 19 columns; (3) the honest verdict depends on the metric: ROC AUC dips (0.848 to 0.843) while average precision rises (0.444 to 0.463). For a rare class whose top of the list is what reviewers see, average precision is the number that matches the job; on one split, both differences are small enough to check with cross-validation (Lesson 2.2 makes that a one-liner); (4) the Gift Cards weight (0.95) is the largest of all 19; (5) the imputer learned a fill value (447 days) although the training data had no gaps: that is what makes it ready for the guest checkout; (6) Bikes becomes all zeros, the missing account age becomes the training median (scaled −0.311), and P1 (a gift card at 3 a.m. on a new device, three card attempts, four payments in 24 hours) scores 1.000.

LAB 2.1-B: LABS/MODULE-03/L2_1-PREPROCESSING/DEMAND_CATEGORIES_STARTER.PY → DEMAND_CATEGORIES_SOLUTION.PY

The solution prints:

```
train 5,088 rows, test 4,896 rows
predict the 2024 mean: MAE 17.13 units per week

numbers only           6 columns   MAE 14.71
+ store               10 columns   MAE 14.26
+ store + category    19 columns   MAE 13.26
+ store + product_id  34 columns   MAE 11.31

week 27, Boise, Timberline 2-Person Tent: sold 131
+ store + category    TNT-2001  52.6   new TNT-2004  52.6
+ store + product_id  TNT-2001  45.7   new TNT-2004  87.4

+ store + category + product_id: MAE 11.31   TNT-2001 45.7   new TNT-2004 59.6
```

Reading it: knowing the store helps a little (14.71 to 14.26), the category more (13.26), the product most (11.31, a third below the baseline's 17.13): socks and tents sell at different volumes, and no price or temperature column can say which a row is about. The catch is the new tent. The category model cannot tell TNT-2004 from TNT-2001 (52.6 for both: it knows "a tent in Boise in week 27"). The product model encodes the unknown product as all zeros, and in an unregularized linear model "no product column set" is not "an average product": its 87.4 comes from the intercept and means nothing. Keeping both columns (the stretch) gives the unknown product at least its category, 59.6. Lesson 3.1 builds a far better demand model; Lesson 3.2's regularization makes "all zeros" mean something closer to average.

HANDOUT: LABS/MODULE-03/L2_1-PREPROCESSING/DEBUG_PREPROCESS.PY (DELIBERATELY BROKEN)

1. `ValueError: Cannot use median strategy with non-numeric data: could not convert string to float: '45-54'` → `age_group` is text: take it out of the median branch and add it to the one-hot branch (its gaps become an `age_group_nan` column). A `most_frequent` or `constant` imputer in its own branch also works, with a different column count.
2. `ValueError: Pandas output does not support sparse data...` → `OneHotEncoder(sparse_output=False, handle_unknown="ignore")`.
3. No error: `OrdinalEncoder()` sorts the tiers alphabetically and prints `Basic 0.0, Silver 2.0, Gold 1.0`. Fix: `OrdinalEncoder(categories=[["Basic", "Silver", "Gold"]])`.
4. No error: `preprocess.fit_transform(X_test)` re-learns every median, mean and category list from the test rows. The clue is the printed test-set mean of exactly 0.0. Fix: `preprocess.transform(X_test)`. Fixed, the file prints (4500, 31) (1500, 31), 0.055, the tiers in order, and ROC AUC 0.824 (0.823 with the two silent bugs in: small here, and nonsense the day the test batch is three new customers).

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 1.4 · METRICS FOR CLASSIFICATION AND REGRESSION

1. **C** 0.400: recall = TP / (TP + FN) = 100 / 250; 0.667 is the precision and 0.867 the accuracy.
2. **B** Recall rises and precision falls: more customers are flagged; the probabilities and ROC AUC do not change.
3. **A** Compare average precision with 0.009, the fraud rate (its random baseline); here it is the more realistic summary.
4. **B** 0.75: of the 4 positive-negative pairs the positive wins 3 (only 0.35 against 0.4 is lost).
5. **A** N: RMSE is dominated by big misses, and big misses are what hurt Pinecrest most.

LESSON 2.1 · IMPUTE, SCALE AND ENCODE

1. **B** [[3.]]: the median of the known values 1, 3 and 100 is 3 (the mean, 34.67, is dragged up by 100).
2. **A** A row of five zeros: `handle_unknown="ignore"` encodes an unknown store as none of the known ones; no column is added.
3. **C** So that the test rows cannot influence the learned fill value, exactly as future customers cannot.
4. **C** The codes impose an order, so a linear model can only learn an effect that grows steadily from Asheville to Online; use one-hot.
5. **A** They are dropped: the default `remainder="drop"` keeps only listed columns (a safety feature against IDs and leaks).

EXIT TICKET

1. **TN 900, FP 100, FN 30, TP 70:** precision 0.412 recall 0.700. Precision: of the customers we flag, about four in ten really churn (six of ten calls are wasted). Recall: we reach seven in ten of the customers who are leaving. Accept any correct business sentence that names who is counted.
2. **Out-of-fold, not test:** the test set exists to give one honest final number about data the model and every choice never saw. Choosing the threshold on it tunes on it and flatters the result. Out-of-fold probabilities (`cross_val_predict` or `TunedThresholdClassifierCV`) come from models that never saw each training customer, so the threshold is chosen honestly and the test set is used once.
3. **Reno and no age:** the fitted one-hot branch encodes `home_store="Reno"` as all zeros (`handle_unknown="ignore"`) and the missing `age_group` as a 1 in `age_group_nan`; the numeric branches use the training medians, means and standard deviations; nothing is refitted on this customer. Full marks need "nothing is refitted" or "uses what was learned from the training rows".

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 A (precision 0.667, recall 0.400).** Precision and recall swapped in their head, the same thing a swapped `(y_pred, y_true)` does in code. Use the questions: "of those we called" (precision), "of those who left" (recall).
- **Q3 A or B (test set, or in-sample training probabilities).** Has the idea of tuning a threshold but not of where it may be tuned. In-sample is the subtle one: a forest scored on its own training rows is far too confident.
- **Q4 C (0.5 1.0).** Thinks labels give a coin flip. Count the pairs with the learner, ties as halves.
- **Q5 A or C.** Either ignores the costs (0.5) or confuses value with break-even (0.80). Write the expected value of one call: $80 \times p - 20$.
- **Q7 A (a new Reno column).** Expects the encoder to grow. The output always has the training columns, which is the whole point of a fitted encoder.
- **Q10 A (`[0. 2.]`).** Assumes the encoder knows the intended order. It never does: spell it out.
- **Q11 A (two branches).** Thinks branches run in sequence. Draw the ColumnTransformer diagram: branches side by side, each on the raw columns.
- **Exit ticket 2: "because the test set is smaller".** Misses the point about unseen data. Return to the dress-rehearsal rule from Lesson 1.2.

INSTRUCTOR KIT · ANSWER KEY

Session 4: Pipelines and Creating Features

Answers and reasoning for every Session 4 item: the warm-up, both self-checks, the predict-then-run items, the worked examples, the lab reference outputs, the lesson pages' own quizzes and the exit ticket.

Session 4 | 12 self-check items | Confidential

WARM-UP

Retrieval from Session 3 (Lessons 1.4 and 2.1). Answers and what each checks:

W1

Pinecrest's care team calls 150 customers and 103 of them really were about to churn. Which metric is 103 / 150, and what does it answer? (Lesson 1.4)

Precision (at 150): of the customers we contacted, how many were real churners? Recall would divide by all real churners instead.

W2

What does this print? (Lesson 1.4)

```
from sklearn.metrics import precision_score, recall_score
y_true = [1, 1, 1, 0, 0, 0, 0, 0]
y_pred = [1, 1, 0, 1, 0, 0, 0, 0]
print(round(precision_score(y_true, y_pred), 3), round(recall_score(y_true, y_pred)
```

`0.667 0.667` : two of the three flagged are churners (precision 2/3) and two of the three churners were flagged (recall 2/3).

W3

Lesson 1.4's project chose a threshold of 0.26. Why from out-of-fold probabilities on the training rows, and never from the test set? (Lesson 1.4)

The threshold is a setting learned from data. Choosing it on the test set lets the test set help build the model, so its score is no longer an honest preview. Out-of-fold probabilities give every training customer a score from a model that did not see them.

W4

What does this print? (Lesson 2.1)

```
from sklearn.preprocessing import OneHotEncoder
enc = OneHotEncoder(sparse_output=False, handle_unknown="ignore").fit([["Bend"], ["
print(enc.transform([["Reno"], ["Boise"]]))
```

Reno is unknown, so with `handle_unknown="ignore"` it becomes a row of zeros; Boise is the second learned category. The category list was learned in `fit`.

W5

Lesson 2.1 asked: to cross-validate its churn model, which objects must be refitted inside every fold, and in which order? (Lesson 2.1)

The whole `ColumnTransformer` (imputer, scalers, ordinal and one-hot encoders), then the model, on the training folds only; then transform-and-predict on the validation fold. Doing that by hand is error-prone: a `Pipeline` does exactly this, which is today's first lesson.

2.2 · PIPELINES: SELF-CHECK

Q1

A fitted pipeline `make_pipeline(StandardScaler(), LogisticRegression())` is asked to `predict_proba(X_new)`. What does the scaler step do with `X_new`?

A) `fit_transform`: the new rows are centered on their own mean

B) `transform` only, with the means and standard deviations it learned during `fit`

C) Nothing: scaling only happens during fit

D) It refits on the training rows and the new rows together

> **Why:** During prediction every step except the last calls `transform` with what it learned in `fit`; nothing is refitted on the data being predicted. That is the whole reason a pipeline cannot leak.
> **A** is wrong: Then a batch of ten new customers would be scaled against each other, and the same customer would get a different score in a different batch.
> **C** is wrong: Without scaling, the model would receive raw numbers it was never trained on.
> **D** is wrong: The pipeline does not keep the training rows; it keeps only what each step learned.

Q2

Which name sets the imputer's strategy in this pipeline?

```
pipe = Pipeline([
    ("prep", ColumnTransformer([
        ("num", make_pipeline(SimpleImputer(), StandardScaler()), ["email_opens_90d
    ])),
    ("model", LogisticRegression()),
])
```

A) `prep__simpleimputer__strategy`

B) `num__simpleimputer__strategy`

C) `prep_num_simpleimputer_strategy`

D) `prep__num__simpleimputer__strategy`

Why: Read it as a path from the top: the step named prep, its branch named num, that branch's step simpleimputer (make_pipeline's automatic name), then the setting. Each level is joined by two underscores.

A is wrong: Skips the branch name num.

B is wrong: Starts below the top level: the pipeline itself has no step called num.

C is wrong: Single underscores are part of ordinary names; the separator is a double underscore.

Q3

What happens when this runs?

```
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
pipe = make_pipeline(StandardScaler(), LogisticRegression())
pipe.set_params(model__C=0.1)
```

A) `ValueError`: `Invalid parameter` `'model'`: make_pipeline named the step `logisticregression`

B) It sets C on the logistic regression

C) `TypeError`: C can only be set when the model is created

D) Nothing: the unknown setting is silently ignored

Why: `make_pipeline` names steps after their classes in lowercase, so the model step is `logisticregression` and the setting is `logisticregression__C`. The step is called `model` only when you name it yourself with `Pipeline([...])`. The message: `ValueError: Invalid parameter 'model' for estimator Pipeline(s...`

B is wrong: It would, with the right step name.

C is wrong: `set_params` can change any setting after creation.

D is wrong: scikit-learn refuses unknown settings loudly, and lists the valid ones.

Q4

Which of these steps could be applied to all training rows BEFORE cross-validation without changing the cross-validated score at all?

A) `SimpleImputer(strategy="median")`

B) `TargetEncoder()` on a high-cardinality column

C) `FunctionTransformer(np.log1p)`

D) `SelectKBest(k=10)` (keeps the columns most related to y)

Why: A fixed function learns nothing from the data (it is stateless), so applying it early or inside the folds gives the identical result. Lab 2.2 B measures it: a difference of +0.00000.

A is wrong: Learns a median from the rows it sees, so the validation rows would shape it (a tiny leak, but a leak).

B is wrong: Learns from the target: the biggest leaks of all (0.913 vs 0.583 in the lab).

D is wrong: Selection looks at y, so choosing columns on all rows lets the validation rows vote (Lesson 2.4).

Q5

What does this print?

```
import numpy as np
from sklearn.preprocessing import FunctionTransformer
log = FunctionTransformer(np.log1p).fit([[0.0], [9.0]])
print(log.transform([[99.0]]).round(3))
```

`[[4.605]]`

Why: `FunctionTransformer` with a fixed function is stateless: `fit` learns nothing from 0 and 9, and `transform` applies log(1 + 99) = log(100) = 4.605 to whatever arrives.

Q6

The IT team reloads `churn_pipeline.joblib` and sends next month's records, but their export leaves out the `email_opens_90d` column. What happens?

A) The imputer fills the whole column with the training median

B) The pipeline uses the other 15 columns and ignores the missing one

C) It predicts the baseline churn rate for every customer

D) `ValueError: columns are missing: {'email_opens_90d'}` : send the column, empty (NaN) where unknown

Why: The ColumnTransformer selects columns by name and refuses a record that lacks one it was fitted with (`reloaded.feature_names_in_` lists them). A column that is present but empty is fine: the imputer handles NaN.

A is wrong: The imputer fills gaps INSIDE a column; it cannot invent a missing column.

B is wrong: The model's weights assume every column; nothing silently drops one.

C is wrong: Nothing falls back to a baseline; it raises an error.

2.2 · PIPELINES: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
pipe = make_pipeline(SimpleImputer(), StandardScaler(), LogisticRegression())
print(list(pipe.named_steps))
print(len(pipe[:-1]))
```

```
['simpleimputer', 'standardscaler', 'logisticregression'] 2
```

> **Why:** `make_pipeline` names each step after its class in lowercase. `pipe[:-1]` is a pipeline of every step before the model: two steps.

P2

Predict the output, then run it.

```
from sklearn.impute import SimpleImputer
imp = SimpleImputer(strategy="median").fit([[1.0], [2.0], [9.0]])
print(imp.transform([[None], [100.0]]).ravel())
```

```
[ 2. 100.]
```

> **Why:** The median of 1, 2 and 9 is 2. The gap gets the training median; 100 is not a gap, so it passes through unchanged. Nothing is learned from the new rows.

2.3 · CREATING FEATURES: SELF-CHECK

Q7

What does this print?

```
import pandas as pd
df = pd.DataFrame({"store": ["A", "A", "B", "B", "A"], "units": [5, 7, 9, 4, 6]})
print(df.groupby("store")["units"].shift(1).tolist())
```

`[nan, 5.0, nan, 9.0, 7.0]`

> **Why:** `shift(1)` within each store: each row gets the previous value of ITS OWN store. Rows 0 and 2 are the first of stores A and B (no history: NaN); row 3 gets B's 9; row 4 gets A's previous value, 7 (row 1), not B's 4. That is why lags are always built per series.

Q8

Churn is 32% for customers under 6 months, about 8 to 11% in the middle years and 21.5% above 72 months, yet a logistic regression on the raw columns gives tenure only a small weight. Why?

A) Tenure does not really matter for churn

B) Tenure was not scaled

C) A straight line through a U-shape is nearly flat; flags such as new_customer and veteran let the model express it

D) Tenure is redundant with `signup_date`

> **Why:** A linear model gives each column one slope. High at both ends and low in the middle averages out to almost no slope. With two flags, each arm of the U gets its own weight (0.92 and 0.49 in the lesson).
> **A** is wrong: The tenure table shows a big effect, just not a straight-line one.
> **B** is wrong: The lesson's pipelines all scale; scaling changes units, not shape.
> **D** is wrong: It is (the workbook example shows a correlation of 1.0), but that does not explain the weight, and `signup_date` is not in the model.

Q9

What does `tickets_no_app = support_tickets_12m * (1 - app_user)` let a linear model do?

A) Give support tickets their own weight for customers without the app

B) Remove app users from the training data

C) Count every app user's tickets twice

D) Replace `app_user`, which can then be dropped

> **Why:** The product equals the ticket count for non-app customers and 0 for app users, so its weight describes tickets where they matter (churn 14% to 46% without the app, flat with it). That is an interaction term.
> **B** is wrong: Every row stays; app users just get 0 in the new column.
> **C** is wrong: App users get 0, not double.
> **D** is wrong: The interaction is added next to `app_user`, which still carries its own effect.

Q10

What does this print?

```
import numpy as np
months = np.array([3, 6, 12])
print(np.cos(2 * np.pi * months / 12).round(2))
```

`[ 0. -1. 1.]`

> **Why:** cos(2π·3/12) = cos(π/2) = 0, cos(π) = -1, cos(2π) = 1. December (12) lands on the same point of the circle as "month 0", one step from January: the encoding wraps around. Cosine alone is not enough, though: April and August both have a cosine of -0.5. The sine column tells them apart.

Q11

An average of units sold per product, store and month computed from BOTH years forecasts 2025 with MAE 9.87; computed from 2024 only, 11.36. Why is 9.87 not a real improvement?

A) Averages over more rows always overfit

B) The 2025 weeks being predicted helped compute their own feature

C) MAE rewards forecasts that are too low

D) 2025 has fewer weeks than 2024

> **Why:** Half of every average now comes from the test answers. On the day a 2025 forecast is made, those weeks have not happened. Compute target aggregates from the training rows only (or per fold, with TargetEncoder).
> **A** is wrong: More honest rows would help; the problem is WHICH rows.
> **C** is wrong: MAE treats over- and under-forecasts alike.
> **D** is wrong: The row counts (5,088 and 4,896) do not explain an error that drops by 13%.

Q12

What does this print?

```
import pandas as pd
df = pd.DataFrame({"returns": [1, 0, 2], "tickets": [2, 0, 0]})
print((df["returns"] / df["tickets"]).tolist())
```

A) `[0.5, 0.0, 0.0]`

B) A `ZeroDivisionError`

C) `[0.5, nan, inf]`

D) `[0.5, nan, nan]`

> **Why:** pandas does not raise on division by zero: 0 / 0 is NaN and 2 / 0 is infinity. The error appears only later, inside the model (`Input X contains infinity`). Divide by something that cannot be 0, such as `tickets + 1` .
> **A** is wrong: pandas does not replace the result with 0.
> **B** is wrong: Plain Python integers raise; pandas Series do not.
> **D** is wrong: Only 0 / 0 is undefined; a positive number over 0 is infinity.

2.3 · CREATING FEATURES: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
import pandas as pd
df = pd.DataFrame({"tickets": [0, 2, 3, 1], "app_user": [1, 1, 0, 0]})
df["tickets_no_app"] = df["tickets"] * (1 - df["app_user"])
print(df["tickets_no_app"].tolist())
```

```
[0, 0, 3, 1]
```

> **Why:** Rows 0 and 1 are app users (1 - 1 = 0), so 0 whatever their tickets; rows 2 and 3 keep their ticket counts 3 and 1.

P4

Predict the output, then run it.

```
import pandas as pd
days = pd.to_datetime(pd.Series(["2025-06-28", "2025-06-30", "2025-07-04"]))
print(days.dt.dayofweek.tolist(), days.dt.month.tolist())
```

```
[5, 0, 4] [6, 6, 7]
```

> **Why:** Monday is 0, so Saturday June 28 is 5, Monday June 30 is 0 and Friday July 4 is 4; the months are 6, 6 and 7.

WORKED EXAMPLES: WHAT TO POINT OUT

2.2 · A PIPELINE THAT FILLS A GAP

Outputs as printed in the workbook:

```
CV ROC AUC 0.813 +/- 0.024
medians learned: [52.     0.141  6.   ]
[[-0.73 -0.61 -0.22]
 [ 1.57  1.63 -0.22]]
churn probability: [0.043 0.669]
```

Both unknown email counts become the training median 6 and scale to the same -0.22: nothing is refitted on new rows. The three-column model's CV ROC AUC (0.813) sits below the 16-column pipeline's 0.833: fine for an example, and a hint of what Lesson 2.4 asks (how few columns do you need?).

2.3 · A DATE FEATURE THAT WAS ALREADY THERE

Outputs:

```
     signup_date  days_since_signup  tenure_months
1556  2025-05-11                 50              1
965   2021-10-28               1341             44
5145  2025-02-25                125              4
correlation with tenure: 1.0
signup_month    1     2     3     4     5     6     7     8     9     10    11    12
churn         0.19  0.20  0.20  0.19  0.24  0.11  0.09  0.12  0.14  0.14  0.11  0.15
new_share     0.40  0.44  0.42  0.40  0.38  0.00  0.00  0.00  0.00  0.00  0.00  0.01
customers with 6+ months: churn by signup month from 0.091 to 0.153
```

Two lessons in one example: a computed feature can duplicate an existing column exactly (correlation 1.0), and a feature can be a proxy (signup month stands in for "joined in 2025", that is, for the new-customer effect). Expect some learners to propose signup month as a churn feature; this is the answer.

LAB REFERENCE SOLUTIONS

- LAB 2.2-A: LABS/MODULE-04/L2_2-PIPELINES/FRAUD_PIPELINE_STARTER.PY → FRAUD_PIPELINE_SOLUTION.PY

The solution prints:

```
(35000, 10) (15000, 10) frauds in train: 307
['prep', 'model']
baseline             CV ROC AUC 0.500   CV average precision 0.009
logistic regression  CV ROC AUC 0.867   CV average precision 0.535
categories learned: 10 ['Accessories', 'Apparel', 'Camp Kitchen'] ...
largest weights: {'scale__device_new': 0.56, 'scale__velocity_24h': 0.77, 'onehot__cate
model__C=0.01  CV average precision 0.518
model__C=1.0   CV average precision 0.535
test ROC AUC 0.843   test average precision 0.463   frauds in the top 100: 56
reloaded steps: ['prep', 'model'] | columns expected: 10
fraud probability: [0.999, 0.0, 0.002]
```

What each TODO teaches: (1) the same `prep` / `model` structure as the lesson, on a new table; (2) the baseline's average precision equals the fraud rate (0.009), the number to beat on rare classes, while its ROC AUC is 0.5; (3) `named_transformers_` and feature names from `pipe[:-1]` (the Gift Cards category carries the largest weight); (4) `model__C` from the top (more regularization hurts a little here); (5) the test set once: 0.843 against a CV estimate of 0.867, within the folds' wide spread on 131 test frauds; (6-7) a saved, reloaded pipeline scores raw rows, and the unknown `Bikes` category becomes zeros thanks to `handle_unknown="ignore"`. The saved file goes to `OUTPUT` (`output/` next to the lab; a temporary folder during the kit check).

- LAB 2.2-B: LABS/MODULE-04/L2_2-PIPELINES/LEAK_SIZE_STARTER.PY → LEAK_SIZE_SOLUTION.PY

The solution prints:

```
1016 distinct dollar amounts in train; 434 of them occur 5 times or fewer
target rate  before CV 0.913   inside pipeline 0.583   test 0.568
scaler       before CV 0.8613  inside pipeline 0.8613  difference -0.00001
log1p        before CV 0.8617  inside pipeline 0.8617  difference +0.00000

learns from y, few rows per value : huge lie  (put it in the pipeline, or drop the idea
learns from X only                : tiny lie  (still: put it in the pipeline)
learns nothing (stateless)        : no lie    (safe anywhere; the pipeline costs nothin
```

Reading it: the per-amount fraud rate is the lesson's signup-date leak on new data. Most amounts occur only a handful of times, so "the fraud rate of this amount" is mostly the row's own label: 0.913. Inside the pipeline the encoder never sees a validation row's label: 0.583, and the test agrees (0.568). The amount does carry a little real signal (gift cards come in $25 steps). The scaler leak exists but is invisible at this size (-0.00001); the log step cannot leak at all. The habit, not the judgment: put every learning step in the pipeline.

HANDOUT: LABS/MODULE-04/L2_2-PIPELINES/DEBUG_PIPELINE.PY (DELIBERATELY BROKEN)

1. `ValueError: Invalid parameter 'alpha' for estimator Pipeline(...)` → `pipe.set_params(model__alpha=10.0)`.
2. `ValueError: Estimator names must not contain __: got ['one__hot']` → name the branch `one_hot` (or `onehot`).
3. `ValueError: columns are missing: {'product_id', 'store'}` → predict on `test[numeric + cats]`.
4. No error: `test = demand[demand["year"] != 2025]` selects 2024, the training year (the first line prints `5088 training weeks, 5088 test weeks`, the clue). Fixed to `== 2025`: 4,896 test weeks and MAE 11.75 instead of the flattering 11.02.

LAB 2.3-A: LABS/MODULE-04/L2_3-CREATING-FEATURES/NIGHT_FEATURES_STARTER.PY → NIGHT_FEATURES_SOLUTION.PY

The solution prints:

```
         mean   size
hour
0-5    0.0671   1296
6-11   0.0067   8266
12-17  0.0064  13179
18-23  0.0065  12259
device_new      0      1
night
False       0.005  0.022
True        0.005  0.425
{'night': 0.037, 'night_new_device': 0.006, 'log_amount': 5.009}
raw columns        CV ROC AUC 0.867   CV average precision 0.535
+ night features   CV ROC AUC 0.864   CV average precision 0.564
raw columns        {'scale__hour': -0.28, 'scale__device_new': 0.56}
+ night features   {'scale__hour': 0.01, 'scale__night': 0.02, 'scale__night_new_device
raw columns        test ROC AUC 0.843   average precision 0.463   frauds in the top 100
+ night features   test ROC AUC 0.844   average precision 0.504   frauds in the top 100
```

Reading it: fraud is ten times more common from midnight to 5:59, and the night-plus-new-device combination is where it concentrates (42.5%). The raw model could only give `hour` one slope, and it came out negative (fraud at hours 0 to 5 drags the line down for "late"); with the flags, `hour` drops to about 0 and the flags carry the effect. ROC AUC stays flat (0.867 vs 0.864, well inside the spread) while average precision rises 0.535 → 0.564 in CV and 0.463 → 0.504 on test, and the top 100 holds 61 frauds instead of 56: the features help at the top of the ranking, where a review team works. `pipe[:-1].get_feature_names_out()` fails because the first step is a `FunctionTransformer` without `feature_names_out`; asking the `prep` step works. The scaled weight of `night_new_device` looks small (0.19) because the flag is rare (0.6% of rows): scaled weights are per standard deviation.

- LAB 2.3-B: LABS/MODULE-04/L2_3-CREATING-FEATURES/DAILY_SALES_FEATURES_STARTER.PY → DAILY_SALES_FEATURES_SOLUTION.PY

The solution prints:

```
mean sales by day of week, 2023-2024 (Mon..Sun): [4639, 4270, 4512, 4750, 6024, 8209, 6
         date   sales   lag_1   lag_7  rolling_7
30 2023-01-08  7949.0  8165.0  6027.0     5506.0
34 2023-01-09  3922.0  7949.0  4433.0     5781.0
38 2023-01-10  4401.0  3922.0  4471.0     5708.0
lag_1     as the forecast: MAE 1,350 dollars
lag_7     as the forecast: MAE 1,072 dollars
rolling_7 as the forecast: MAE 1,451 dollars
calendar only                        MAE 996
calendar + lag_7 + rolling_7         MAE 708
+ store x day + lag_1                MAE 586
leaky window (includes today)        MAE 551
```

Reading it: Saturday sells almost twice what Tuesday does, so the history feature that matches the weekly cycle wins: the same day last week (1,072) beats yesterday (1,350) and the last seven days (1,451). In a linear pipeline, calendar parts plus the known events (closed days, sales) reach 996; adding history 708; letting each store have its own weekly pattern (store × day) plus yesterday 586. The window without `shift(1)` contains the day being forecast: 551, better and unusable, because the evening before, today's sales do not exist yet. (These numbers are not comparable with the deep-learning forecast in Lesson 10.4, which predicts log sales with a different setup.)

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 2.2 · PIPELINES

1. **C** It calls `fit_transform` on X_train and passes the result to the model: every step but the last fits and transforms during fit.
2. **A** `quiz_pipe.set_params(logisticregression__C=0.1)` : `make_pipeline` names the step after its class, and the separator is two underscores.
3. **B** The rates were computed before the folds were split, so each validation row's own label went into its feature.
4. **A** It fills the gap with the median it learned from the training rows: prediction only transforms.
5. **B** No: a pipeline keeps rows apart, but it cannot know that a column comes from the future.

LESSON 2.3 · CREATING FEATURES

1. **B** `[nan, nan, 6.0, 8.0]` : shift moves 5, 7, 9 down one place, then each window of two averages (5, 7) and (7, 9).
2. **C** Categories have opposite seasons, so one shared effect per month averages them away.
3. **A** The training rows only: choosing a threshold is learning from data.
4. **C** The window includes the week being predicted, so the feature contains the answer.
5. **A** Sine alone gives two different months the same value (for example April and August); cosine tells them apart.

EXIT TICKET

1. **Pipeline:** it fits each step on the training rows and only transforms at prediction time (no remembering the order); cross-validation and grid search refit every step inside each fold; one object saves to one file. It cannot stop a leaky COLUMN (such as `cancellation_reason`) that you put in the feature list.
2. **0.930 vs 0.571:** the rates were computed with every training label, so each validation row's feature contained its own answer (most signup dates have one or two customers). `TargetEncoder` inside a pipeline, refitted

per fold: honest CV 0.586, which matches the test.

3. **A feature and its proof:** e.g. `lapsed_150` (a jump), `new_customer` / `veteran` (a U-shape), `tickets_no_app` or `night_new_device` (an interaction), category × month (a season per category). Proof = cross-validation against the plain model (0.829 → 0.883 for churn; 11.23 → 9.39 MAE for demand), then one test check.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 A (`fit_transform` on new rows).** The learner still pictures preprocessing as something that happens to whatever data arrives. Use the batch argument: the same customer would get a different score in a different batch.
- **Q2 A or B.** Skips a level of nesting. Have them print `list(pipe.get_params())` and find the key; the path always starts at the top-level step.
- **Q3 B ("it sets C").** Assumes the step is called `model` everywhere. Only `Pipeline([...])` with your own names gives `model`.
- **Q4 A (the imputer).** Thinks "it does not look at y" means "it cannot leak". It still learns a median from the validation rows: a tiny leak, but not zero. Only a step that learns nothing is exactly safe.
- **Q8 A ("tenure does not matter").** Reads a small weight as no effect. Show the tenure table next to the weight: a shape the model cannot draw.
- **Q11 A ("averages overfit").** Misses the timing. Ask: "Were the 2025 weeks known when the forecast was made?"
- **Exit ticket 1: "a pipeline prevents all leaks".** The most important misconception of the day; point back to the selector that grabbed `cancellation_reason` (0.937).

INSTRUCTOR KIT · ANSWER KEY

Session 5: Feature Selection and Imbalanced Classes

Answers and reasoning for every Session 5 item: the warm-up, both self-checks, the predict-then-run items, the lab reference outputs, the debug handout, the lesson pages' own quizzes and the exit ticket.

`Session 5` `12 self-check items` `Confidential`

WARM-UP

Retrieval on Lessons 2.2 and 2.3. The note under each answer says where it comes back today.

W1

A pipeline is `make_pipeline(SimpleImputer(), StandardScaler(), LogisticRegression())`. What does `pipe.predict(X_new)` do to each of the three steps?

The imputer and the scaler only transform the new rows, with the medians, means and standard deviations they learned during `fit`; the model then predicts. Nothing is refitted.

> **Why:** If someone says "it refits the scaler", ask what would happen with one new customer: a scaler fitted on one row.

W2

In Lesson 2.2 the signup-date churn rate scored CV ROC AUC 0.930 when computed with pandas, and 0.586 as a `TargetEncoder` inside the pipeline. Which is honest, and why was the other so high?

0.586. The pandas rates used every training row's label, including the rows that later sat in each validation fold; with one or two customers per date, "the rate on your date" was mostly your own label.

> **Why:** This is today's selection leak in another costume: any step that looks at `y` belongs inside the folds.

W3

What does this print?

```
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
pipe = make_pipeline(StandardScaler(), LogisticRegression())
print([name for name in pipe.get_params() if name.endswith("__C")])
```

`['logisticregression__C']`

> **Why:** `make_pipeline` names each step after its class in lowercase; `step__parameter` reaches any setting. Today you swap a whole step the same way: `set_params(logisticregression=...)`.

W4

`add_churn_features` builds `tickets_no_app = support_tickets_12m * (1 - app_user)`. What is it for an app user with 3 tickets, and why build it?

0. Tickets raise churn for customers without the app but not for app users; a linear model cannot express that from the two raw columns, so the product gives non-app tickets their own weight.

> **Why:** Keep this pair in mind: in today's demo forward selection keeps both raw columns because they matter together.

W5

Why can `FunctionTransformer(add_churn_features)` be fitted outside the folds without leaking, while `TargetEncoder` cannot?

The feature function is stateless: its thresholds are fixed numbers and it learns nothing from the rows it sees. `TargetEncoder` learns rates from `y`, so fitting it on validation rows hands them their own answers.

> **Why:** Feature selection and resampling, today's two subjects, are both "learn from y" steps.

2.4 · FEATURE SELECTION: SELF-CHECK

Q1

Forward selection kept both `support_tickets_12m` and `app_user`; `SelectKBest` with the F-test or with mutual information kept neither. What explains the difference?

A) Filters cannot score 0/1 columns such as `app_user`

B) The F-test needs scaled data and was given raw counts

C) A filter scores each column on its own; the wrapper judges each column next to the ones already chosen, so it sees that the two matter together

D) Forward selection always keeps more columns than `SelectKBest`

> **Why:** Tickets raise churn mainly for customers without the app. Alone, each column shows only a diluted average effect, so a one-column score ranks them low; a wrapper that tries tickets next to the app flag sees the pair help.
> **A** is wrong: `app_user` got F rank 5 of 11: filters score 0/1 columns without trouble.
> **B** is wrong: The F-test compares class means relative to their spread, so scaling a column does not change its score.
> **D** is wrong: Forward selection stopped at six columns, the same number `SelectKBest` was asked to keep.

Q2

What does this print?

```
import pandas as pd
from sklearn.feature_selection import VarianceThreshold
X = pd.DataFrame({"flag": [0, 0, 0, 0, 1], "score": [1, 5, 2, 8, 4], "const": [3, 3
print(VarianceThreshold(threshold=0.2).fit(X).get_feature_names_out())
```

`['score']`

> **Why:** `VarianceThreshold` drops columns whose variance is not above the threshold. `flag` has variance 0.2 × 0.8 = 0.16, `const` has 0 and `score` has 6.0, so only `score` survives. It never looks at a target.

Q3

An analyst has 300 customers and 2,000 survey columns. She runs `SelectKBest(f_classif, k=15)` on all 300 rows, then cross-validates a logistic regression on the 15 chosen columns and gets ROC AUC 0.84. What should she do?

A) Report 0.84: five folds are enough to trust it

B) Put the selector inside a pipeline and cross-validate the whole pipeline, expecting a lower number

C) Switch to mutual information, which detects more kinds of relationship

D) Use 10 folds instead of 5 to make the estimate more stable

> **Why:** The selector looked at every row's label, including the rows that later became validation folds, so chance alignments were baked into every fold. Inside a pipeline the choice is redone on each training part. In the lesson the same mistake turned pure noise into about 80% accuracy.
> **A** is wrong: The number of folds does not matter when the validation rows already helped choose the columns.
> **C** is wrong: Any score that looks at the labels leaks in the same way when it runs before the split.
> **D** is wrong: More folds give a more stable estimate of the wrong thing.

Q4

Without a scaler, an L1 logistic regression on the churn columns keeps `avg_order_value` and drops `discount_share`, the second-best predictor. Why?

A) `discount_share` is redundant with spend

B) `avg_order_value` is the stronger predictor

C) L1 always keeps the column with more distinct values

D) `discount_share` runs from 0 to 1, so it needs a large weight, and the penalty charges for the size of each weight

> **Why:** The penalty adds up the absolute weights. A column in small units needs a big weight to matter and is priced out; a column in dollars needs a tiny weight and survives. Without scaling, L1 selects by units, not by usefulness. With a scaler the choice flips.
> **A** is wrong: It is not: the redundant ones are `avg_order_value`, `returns_12m` and `return_rate`.
> **B** is wrong: It is redundant (spend divided by orders) and has the second-lowest F score.
> **C** is wrong: L1 knows nothing about distinct values; it only sees the size of the weights.

Q5

Forward selection prints mean CV ROC AUC by number of columns: 1: 0.7215, 2: 0.8063, 3: 0.8165, 4: 0.8235, 5: 0.8289, 6: 0.8328, 7: 0.8336, 8: 0.8341, 9: 0.8358, 10: 0.8357. With a tolerance of 0.005, how many columns do you keep?

A) 6

B) 5

C) 9

D) 10

Why: The best score is 0.8358, so the line is 0.8308. The smallest set at or above it has 6 columns (0.8328). This is the lesson project's path.

B is wrong: 0.8289 is below the line of 0.8308.

C is wrong: Nine columns has the best score, but the rule asks for the smallest set within the tolerance.

D is wrong: Ten columns scores slightly lower than nine and is larger: never the choice.

Q6

L1 set the weight of `avg_order_value` to zero. Which sentence belongs in your report?

A) Average order value has nothing to do with churn

B) Given spend and orders, average order value adds nothing new to this model

C) Average order value should be deleted from Pinecrest's database

D) Customers with a higher average order value churn less

Why: Selection describes this model with these other columns. Average order value is spend divided by orders, so with both of them present it is redundant. Drop spend instead and it may well be kept.

A is wrong: Dropped means "adds nothing given the others", not "unrelated to the target".

C is wrong: Other models, or a model without spend, may need it. Selection is about one model.

D is wrong: A zero weight says nothing about the direction of any effect, and a model never proves a cause.

2.4 · FEATURE SELECTION: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
import pandas as pd
from sklearn.feature_selection import SelectKBest, f_classif
X = pd.DataFrame({"a": [1, 2, 3, 4, 5, 6], "b": [5, 1, 4, 2, 6, 3], "c": [0, 0, 1,
y = [0, 0, 0, 1, 1, 1]
print(SelectKBest(f_classif, k=2).fit(X, y).get_feature_names_out())
```

```
['a' 'c']
```

> **Why:** `a` rises with the class (mean 2 against 5) and `c` is mostly 0 for class 0 and 1 for class 1; `b` is scrambled. The F-test ranks `a` and `c` highest; `get_feature_names_out()` keeps the original order.

P2

Predict the output, then run it.

```
import numpy as np
from sklearn.linear_model import Lasso
X = np.array([[1, 0], [2, 1], [3, 0], [4, 1]])
y = np.array([2, 4, 6, 8])
print(Lasso(alpha=0.1).fit(X, y).coef_.round(2))
```

```
[1.92 0. ]
```

> **Why:** The first column alone explains y perfectly (y = 2 × a), so L1 sets the useless second column to exactly zero. The first weight is a little under 2 because the penalty shrinks it.

2.5 · IMBALANCED CLASSES: SELF-CHECK

Q7

What does this print?

```
import numpy as np
from sklearn.utils.class_weight import compute_class_weight
y = np.array([0] * 95 + [1] * 5)
print(compute_class_weight("balanced", classes=np.array([0, 1]), y=y).round(3))
```

`[ 0.526 10. ]`

> **Why:** Each class gets n_samples / (n_classes × count): 100 / (2 × 95) = 0.526 for the common class and 100 / (2 × 5) = 10 for the rare one. Both classes then carry the same total weight (50 each).

Q8

A model's probabilities are calibrated. A manual review costs $8 and a missed fraud costs $200. Above which probability should a payment be flagged?

A) 0.5

B) 0.0088, the fraud rate

C) 0.4

D) 0.04

> **Why:** Flag when the expected loss of letting it through, p × $200, is bigger than the $8 review: p > 8 / 200 = 0.04.
> **A** is wrong: 0.5 is right only when both mistakes cost the same.
> **B** is wrong: The base rate is not a cost trade-off.
> **C** is wrong: That is 80 / 200, as if a review cost $80.

Q9

Resampling with SMOTE before cross-validation reported AP 0.912; with SMOTE inside an imblearn pipeline, 0.495. Which is honest, and why?

A) 0.495: resampling first put synthetic frauds, built from frauds in the training folds, into the validation folds, at a 50/50 balance production never has

B) 0.912: SMOTE gives the model more frauds to learn from, so it really is better

C) Neither: average precision cannot be used after resampling

D) 0.912, because the encoder was fitted on more rows

Why: Inside the pipeline each fold resamples only its own training part, so validation folds keep the real 0.9% fraud rate and contain no near-copies of training rows.
B is wrong: Both runs use SMOTE. The only difference is when it ran, so the gap is measurement, not model quality.
C is wrong: AP is fine; it must be computed on folds with the real class balance.
D is wrong: Fitting the encoder on all rows is a small extra leak, not a reason to trust the bigger number.

Q10

What does this print? (Review $10, missed fraud $375.)

```
from sklearn.metrics import confusion_matrix
y_true = [0, 0, 0, 0, 0, 0, 1, 1]
y_pred = [0, 1, 1, 0, 0, 0, 1, 0]
tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
print(10 * (tp + fp) + 375 * fn)
```

`405`

Why: `ravel()` gives tn, fp, fn, tp in that order: 3, 2, 1, 1. Three payments flagged cost $30; one fraud missed costs $375: 405.

Q11

After `tuned = TunedThresholdClassifierCV(...).fit(X_train, y_train)`, what do `tuned.predict` and `tuned.predict_proba` return?

A) Both use the new threshold: `predict_proba` is rescaled so that 0.5 sits at the tuned threshold

B) `predict` still uses 0.5; only `predict_proba` changes

C) `predict` applies the tuned threshold; `predict_proba` returns the underlying model's probabilities unchanged

D) `predict_proba` is no longer available; only `predict` works

> **Why:** The wrapper only moves the cut. The probabilities stay the plain model's calibrated ones, which is why the project can sort the review queue by them. `tuned.estimator_.predict` gives the old 0.5 decisions for comparison.
> **A** is wrong: Nothing is rescaled; the probabilities are still honest estimates.
> **B** is wrong: That is the wrapped estimator's behavior, not the wrapper's.
> **D** is wrong: Both methods work; the project uses both.

Q12

Cross-validating "all legitimate" and the plain fraud model without `scoring=` gives 0.9912 and 0.9941. What is wrong?

A) Nothing: the plain model is better, and the scores agree

B) The default score of a classifier is accuracy, which hides whether any fraud is found; pass `scoring="average_precision"` or a cost scorer

C) The folds were not stratified, so some had no fraud

D) The dummy should use `strategy="prior"`

> **Why:** A model that catches 30% of fraud and one that catches none differ only in the third decimal of accuracy. Tuning or comparing on this number picks models almost at random.
> **A** is wrong: The difference of 0.003 hides a model that finds frauds against one that finds none.
> **C** is wrong: `cross_val_score` stratifies classifiers by default, and the lesson's cv is stratified.
> **D** is wrong: A prior dummy has the same accuracy; the problem is the yardstick.

2.5 · IMBALANCED CLASSES: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
import numpy as np
from sklearn.utils.class_weight import compute_class_weight
y = np.array([0] * 8 + [1] * 2)
print(compute_class_weight("balanced", classes=np.array([0, 1]), y=y))
```

```
[0.625 2.5 ]
```

> **Why:** n_samples / (n_classes × count): 10 / (2 × 8) = 0.625 and 10 / (2 × 2) = 2.5. Each class then carries a total weight of 5.

P4

Predict the output, then run it.

```
from sklearn.metrics import confusion_matrix
y_true = [0, 0, 0, 1, 1]
y_pred = [0, 1, 0, 1, 0]
print(confusion_matrix(y_true, y_pred).ravel())
```

```
[2 1 1 1]
```

> **Why:** `ravel()` lists tn, fp, fn, tp: two true negatives, one false positive (row 2), one false negative (row 5), one true positive. Memorize the order: every cost function today depends on it.

LAB REFERENCE SOLUTIONS

- LAB 2.4-A: LABS/MODULE-05/L2_4-FEATURE-SELECTION/LEAN_FRAUD_STARTER.PY → LEAN_FRAUD_SOLUTION.PY

The solution prints:

```
14 candidates; 35000 training payments, 307 frauds

pairs with |correlation| above 0.5:
amount  log_amount    0.88

F-test, top 6: {'night_new_device': 4344.0, 'velocity_24h': 1516.0, 'card_attempts': 9

all 14 columns: CV AP 0.5512 +/- 0.0618
raw 9 columns : CV AP 0.5202
 1  + velocity_24h               CV AP 0.1852
 2  + night_new_device           CV AP 0.3420
 3  + log_amount                 CV AP 0.4252
 4  + card_attempts              CV AP 0.4786
 5  + account_age_days           CV AP 0.5011
 6  + billing_shipping_mismatch  CV AP 0.5204
 7  + country_mismatch           CV AP 0.5336
 8  + gift_card                  CV AP 0.5457
 9  + device_new                 CV AP 0.5505
10  + items                      CV AP 0.5516

best 0.5516, tolerance line 0.5416: 8 columns
['velocity_24h', 'night_new_device', 'log_amount', 'card_attempts', 'account_age_days',

raw 9   test AP 0.444
all 14  test AP 0.505
lean 8  test AP 0.505
random ranking AP 0.0087
```

Reading it: the only near-duplicate is `amount` with its own log (0.88). The F-test's favorite, `night_new_device`, is not forward selection's first pick: alone, `velocity_24h` ranks payments better, and the interaction adds the most next to it (teamwork again). The path never really flattens, because fraud has many weak real signals; with folds that disagree by 0.06, a tolerance of 0.01 keeps 8 of 14 columns. The test set agrees: lean 8 = all 14 = 0.505, both well above the raw 9 (0.444). Engineering bought the accuracy; selection bought the simplicity.

- LAB 2.4-B: LABS/MODULE-05/L2_4-FEATURE-SELECTION/VENDOR_COLUMNS_STARTER.PY → VENDOR_COLUMNS_SOLUTION.PY

The solution prints:

```
(300, 1010) pilot churn rate 0.180
10 real columns     CV AUC 0.745
all 1,010 columns   CV AUC 0.603

select first, then CV  CV AUC 0.825
real columns among the 10 picked: 3
select inside CV       CV AUC 0.660

10 real columns        test AUC 0.794
the vendor's 10 picks  test AUC 0.609
```

Reading it: adding 1,000 useless columns hurts an honest model (0.745 → 0.603): with 300 rows, logistic regression spreads its weight over noise. Selecting on all 300 rows first reports 0.825, higher than the real model, with only 3 real columns among the 10 picks; inside the pipeline the honest estimate is 0.660. The test set settles it: 0.794 for the real columns, 0.609 for the vendor's picks. The leaky number was the only one that pointed the wrong way.

- LAB 2.5-A: LABS/MODULE-05/L2_5-IMBALANCED-CLASSES/RETENTION_THRESHOLD_STARTER.PY → RETENTION_THRESHOLD_SOLUTION.PY

The solution prints:

```
train 4500, test 1500, test churners 248
offer nobody : $14,880
offer everyone: $22,500

ROC AUC 0.827   AP 0.496   (random ranking AP 0.165)
at 0.5: 112 offers, cost $12,540
threshold 0.5   offers  112  cost $12,540
threshold 0.4   offers  170  cost $11,970
threshold 0.3   offers  254  cost $11,250
threshold 0.25  offers  325  cost $10,935
threshold 0.2   offers  406  cost $10,830
threshold 0.15  offers  523  cost $11,265
threshold 0.1   offers  681  cost $12,075
formula threshold: 15 / 60 = 0.25

balanced @ 0.5      : 471 offers, cost $10,965
mean probability    : real 0.165, plain 0.161, balanced 0.377
balanced @ formula  : 862 offers, cost $14,070

tuned threshold 0.234: 343 offers, 153 of 248 churners reached, cost $10,845
```

Reading it: with offers at $15 and churners at $60, the formula says 0.25, and the test sweep agrees (cheapest at 0.2 to 0.25). The default 0.5 makes only 112 offers and costs $1,600 more. Balanced weights at 0.5 land near the cheap zone by accident ($10,965), but their probabilities average 0.377 for a real rate of 0.165, so the formula applied to them overshoots (862 offers, $14,070). The tuned threshold, chosen on the training folds, is 0.234 and costs $10,845. Lesson 2.5's conclusion holds at 16.5% as at 0.9%.

LAB 2.5-B: LABS/MODULE-05/L2_5-IMBALANCED-CLASSES/REBALANCE_TREE_STARTER.PY → REBALANCE_TREE_SOLUTION.PY

The solution prints:

```
train 35,000 rows, 307 frauds; random-ranking AP 0.0088
plain              CV AP 0.335 +/- 0.040
balanced weights   CV AP 0.269 +/- 0.040
undersample        CV AP 0.044 +/- 0.007
oversample         CV AP 0.271 +/- 0.050
SMOTE              CV AP 0.182 +/- 0.033

plain              test AP 0.315   flagged    44   recall 0.27   precision 0.80
balanced weights   test AP 0.335   flagged  1600   recall 0.66   precision 0.05
undersample        test AP 0.037   flagged  2242   recall 0.66   precision 0.04
oversample         test AP 0.338   flagged  1517   recall 0.64   precision 0.06
SMOTE              test AP 0.133   flagged  2061   recall 0.62   precision 0.04

SMOTE first, then CV: AP 0.873   (validation folds are 50% fraud)
```

Reading it: on this data no rebalancing helps the tree's ranking; the plain tree has the best CV AP (0.335). Undersampling leaves 614 training rows and a depth-6 tree overfits them badly (0.044). At 0.5 every rebalanced tree flags 1,500 to 2,200 payments: the cut moved, as with logistic regression. The test APs of plain, balanced and oversampled trees (0.315 to 0.338) differ by less than the fold spread (0.04), so the choice rests on CV, not on one test number. The leak reports 0.873. The lesson's caution stands: SMOTE can help some models on some data; try it, do not assume it.

HANDOUT: LABS/MODULE-05/L2_5-IMBALANCED-CLASSES/DEBUG_IMBALANCE.PY (DELIBERATELY BROKEN)

1. `TypeError: All intermediate steps should be transformers and implement fit and transform or be the string 'passthrough' 'SMOTE(random_state=42)' ... doesn't` → import `make_pipeline` from `imblearn.pipeline`, not `sklearn.pipeline`.
2. `ValueError: could not convert string to float: 'Gift Cards'` → SMOTE runs before the preprocessing step and meets the text column `category`; put it after `preprocess`.
3. No error: `cross_val_score(pipe, X_train, y_train, cv=cv)` reports accuracy (0.994 and 0.876) under the label "CV AP" → add `scoring="average_precision"`.
4. No error: `tp, fp, fn, tn = confusion_matrix(...).ravel()` has the order wrong, and the cost prints $168,565 → `tn, fp, fn, tp`. Fixed: `plain CV AP 0.535`, `SMOTE CV AP 0.495`, `611 flagged, cost $25,985`.

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 2.4 · FEATURE SELECTION

1. **C** `['b' 'c']`: with the default threshold of 0, `VarianceThreshold` drops the constant column `a`.
2. **A** The selection step used the labels of the validation rows, so chance alignments were baked into every fold.
3. **B** Tenure has a U-shaped effect, which a comparison of means misses; mutual information sees it.
4. **B** The 6-column set: the 0.0005 difference is far smaller than the 0.02 fold spread.
5. **A** Smaller `C` means a heavier L1 penalty: more weights become exactly zero.

LESSON 2.5 · IMBALANCED CLASSES

1. **B** Ask for recall and average precision against the all-legitimate baseline (which already scores about 99.1%).
2. **C** `[ 0.505 50. ]`: 1000 / (2 × 990) and 1000 / (2 × 10).
3. **A** So that only the training part of each fold is resampled and validation folds stay real.
4. **B** 0.05: flag when p × 400 > 20.
5. **C** The ranking is about the same; the weights mainly moved where the 0.5 cut falls.

EXIT TICKET

1. **Selection in the pipeline:** the selector looks at `y`. Outside the pipeline it chooses columns with the validation rows' labels too, so cross-validation is optimistic: about 80% on pure noise, 53% when selection is a pipeline step.
2. **Weights:** the ranking (AP) barely changed; the scores shifted so that far more payments cross 0.5: 1,863 flags instead of 44, precision 0.05. The probabilities are inflated (mean 0.27 against 0.009), so they no longer mean what they say.
3. **Threshold:** 10 / 375 ≈ 0.027, valid only for calibrated probabilities (the plain model, no weights, no resampling). For any other model, tune the threshold with `TunedThresholdClassifierCV` on the training rows.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 B ("the F-test needs scaled data").** Mixes up filters with penalties. The F-test compares means relative to spread; only penalties and distances care about units.
- **Q3 A or D ("report 0.84", "use 10 folds").** Trusts the procedure because it has the right name. Ask: "Which rows chose the columns?"
- **Q5 C ("9").** Picks the highest score, not the smallest set within the tolerance. Draw the tolerance line on the path.

- **Q6 A ("has nothing to do with churn").** The most common reporting error in selection. Rewrite it with "given the other columns".
- **Q9 B ("0.912, SMOTE gives more frauds").** Believes the procedure improved the model; both numbers use SMOTE. Point at where it ran.
- **Q10 written as 1,140 or 30.** Wrong `ravel()` order. Write tn, fp, fn, tp on the board and leave it there.
- **Exit ticket 2: "the weighted model is better, it catches more fraud".** Has not separated RANK from DECIDE. Ask for the flags and the AP.

INSTRUCTOR KIT · ANSWER KEY

Session 6: Linear Models and Regularization

Answers and reasoning for every Session 6 item: the warm-up, both self-checks, the predict-then-run items, the lab reference outputs, the Checkpoint 1 rubric, model answer and weak answers, the lesson pages' own quizzes and the exit ticket.

Session 6 | 12 self-check items | Checkpoint 1 rubric | Confidential

WARM-UP

W1

RFECV's curve is 0.8328 at 6 columns and 0.8333 at 11; the folds disagree by about 0.02. How many columns do you keep, and why?

6: the difference is far smaller than the noise between folds, and the smaller set is cheaper and sturdier (the tolerance rule).

> **Why:** Checkpoint 1 asks for honest comparisons like this one.

W2

Review $10, missed fraud $375. What does this print?

```
import numpy as np
p = np.array([0.02, 0.04, 0.3, 0.6])
print((p >= 10 / 375).astype(int))
```

`[0 1 1 1]`

> **Why:** 10 / 375 ≈ 0.027, so every probability from 0.04 up is flagged. Valid only for calibrated probabilities.

W3

Where must SMOTE sit when you cross-validate, and what happens otherwise?

Inside an imblearn pipeline, after the preprocessing; resampling first put synthetic near-copies into the validation folds and reported AP 0.91 instead of about 0.49.

> **Why:** Same rule as feature selection: a step that learns from y lives inside the folds.

W4

Why must an L1 model be scaled before it selects columns?

The penalty charges for the size of each weight, so without scaling it keeps columns in large units and drops columns in small units, whatever their usefulness.

> **Why:** Lesson 3.2 today: every penalty needs scaled features.

W5

A class-weighted model's probabilities average 0.27 when the real fraud rate is 0.009. What does that change?

Its ranking is about the same, but the probabilities are no longer calibrated: a cost formula applied to them flags almost everything. Tune the threshold with cross-validation instead.

> **Why:** Today's 3.1 shows where those probabilities come from: the sigmoid of the log-odds.

3.1 · LINEAR AND LOGISTIC REGRESSION: SELF-CHECK

Q1

What does this print?

```
from sklearn.linear_model import LinearRegression
line = LinearRegression().fit([[0], [1], [2]], [5, 8, 11])
print(line.predict([[10]]))
```

`[35.]`

> **Why:** The three points lie exactly on y = 5 + 3x (slope 3, intercept 5), so at x = 10 the line gives 35. A line extrapolates without limits, whether or not x = 10 makes sense.

Q2

A demand model trained on `log1p(units_sold)` gives `promo` a coefficient of 0.420. What does that say?

A) A promo week sells 0.42 more units

B) A promo week sells about 52% more, for every product, whatever its size

C) A promo week sells 42% more

D) Promotions explain 42% of the variation in sales

> **Why:** On a log target, effects multiply: exp(0.420) - 1 ≈ 0.52, so +52%. That is why the log model fits a fan-shaped residual plot better than "14 extra units of anything".
> **A** is wrong: That would be a coefficient on units, not on log units.
> **C** is wrong: Close for small coefficients only; at 0.42 the exact change is exp(0.42) - 1 = 52%.
> **D** is wrong: That describes R^2, not a coefficient.

Q3

An unscaled logistic regression gives the 0/1 flag `device_new` a coefficient of 0.69. Which sentence is right?

A) A new device adds 0.69 to the probability of fraud

B) A new device makes fraud 69% more likely

C) A new device causes fraud

D) A new device multiplies the odds of fraud by about 2, holding the other features constant

> **Why:** Logistic coefficients are changes in log-odds; exp(0.69) ≈ 2.0 is the odds ratio for a one-unit change, here from 0 to 1.
> **A** is wrong: Coefficients act on the log-odds, not on the probability.
> **B** is wrong: Odds double; how much the probability rises depends on where it started.
> **C** is wrong: A model describes associations in the data, never causes.

Q4

The lesson project's naive price elasticity was -0.53; adding `product_id` made it -1.50. Why did it change so much?

A) The naive slope compared different products (cheap socks sell more units than tents), so price stood in for product type

B) Adding a column always makes coefficients larger

C) The log of the price was taken twice

D) The second model was fitted on fewer rows

> **Why:** A coefficient means "holding the other features constant". With the product in the model, the price coefficient uses only each product's own price changes, its discounts. Season (product × month) then moves it to -1.61.
> **B** is wrong: Coefficients can shrink, grow or flip sign when columns are added; here it is about what is held constant.
> **C** is wrong: Both models use the same log price.
> **D** is wrong: Both models use all rows.

Q5

A logistic regression's decision_function returns -1.0986 for one customer. What does this print?

```
import numpy as np
z = -1.0986
print(round(1 / (1 + np.exp(-z)), 2))
```

`0.25`

> **Why:** The sigmoid of the log-odds: 1 / (1 + e^1.0986) = 1 / (1 + 3) = 0.25. Log-odds of -1.0986 = ln(1/3) means odds of 1 to 3. Negative z, so predict says 0.

Q6

Someone fits `LogisticRegression()` with the whole-number target `units_sold`. What happens?

A) `ValueError: Unknown label type: continuous`

B) It fits a regression line through the counts

C) No error: every distinct count becomes its own class, which is wrong and slow

D) It predicts probabilities of selling more than the mean

> **Why:** Logistic regression is a classifier. Integers look like labels, so scikit-learn silently treats each count as a class; only a continuous target such as `units_sold / 10` raises the error. Check `model.classes_` if in doubt; use a regressor for numbers.
> **A** is wrong: That error appears for continuous values, such as units divided by 10.
> **B** is wrong: Logistic regression never fits a line to the target; it models class probabilities.
> **D** is wrong: It knows nothing about the mean; each count is just a label.

3.1 · LINEAR AND LOGISTIC REGRESSION: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
from sklearn.linear_model import LinearRegression
line = LinearRegression().fit([[0], [1], [2]], [1, 1, 4])
print(line.coef_.round(2), round(line.intercept_, 2), line.predict([[4]]).round(2))
```

`[1.5] 0.5 [6.5]`

> **Why:** Least squares through (0, 1), (1, 1), (2, 4): slope 1.5, intercept 0.5 (the line passes through the averages, $\bar{x} = 1$, $\bar{y} = 2$). At $x = 4$: $0.5 + 1.5 \times 4 = 6.5$.

P2

Predict the output, then run it.

```
import numpy as np
z = np.log(3)            # log-odds: odds of 3 to 1
print(round(1 / (1 + np.exp(-z)), 2))
```

`0.75`

Why: Odds of 3 to 1 means p / (1 − p) = 3, so p = 0.75. The sigmoid of the log-odds turns odds back into a probability.

3.2 · REGULARIZATION AND THE BIAS-VARIANCE TRADE-OFF: SELF-CHECK

Q7

An interaction model trained on a 400-row pilot has training MAE 3.4 and test MAE 42.2. Which change is most likely to help?

A) Remove the penalty so the model can fit better

B) Add degree-3 terms to capture more patterns

C) Increase the Ridge penalty (alpha), chosen by cross-validation

D) Judge it on the training rows, where it does well

Why: A tiny training error with a huge test error is high variance. A penalty (or more data, or fewer columns) shrinks the weights that chase noise; in the lesson alpha = 10 cut the test MAE to 10.0.
A is wrong: Less penalty means more variance: the opposite of what is needed.
B is wrong: More columns add flexibility, and this model already has too much.
D is wrong: Training error alone says nothing about new weeks.

Q8

How many columns does this report?

```
import numpy as np
from sklearn.preprocessing import PolynomialFeatures
print(PolynomialFeatures(degree=2, include_bias=False).fit(np.zeros((1, 4))).n_outp
```

`14`

Why: 4 originals, 4 squares and 6 products of pairs (4 × 3 / 2): 14. `include_bias=False` leaves out the column of ones; with `interaction_only=True` the squares go too (10).

Q9

Snow sales get the temperature twice, in °F and in °C. Ridge gives weights [-0.607, -0.607]; Lasso gives [-1.164, 0]. What follows?

A) Ridge shares the effect between the twins; Lasso keeps one twin and zeros the other, somewhat arbitrarily

B) Lasso is wrong and Ridge is right

C) The Celsius column does not matter, so temperature in °C has no effect on sales

D) The penalties need more scaling

Why: With perfectly correlated columns, Lasso's choice of which twin to keep depends on details such as column order. A zero means "redundant here", not "no effect". Elastic Net shares the weight again.

B is wrong: Both fit equally well; they divide the same effect differently.

C is wrong: The two columns hold the same information; Lasso simply needed only one of them.

D is wrong: The pipeline scaled both; the twins are the reason.

Q10

Without a scaler, Ridge(alpha=100) fits Snow sales with the price in dollars (R^2 0.770) and with the price in thousands of dollars (R^2 0.752). Why do the fits differ?

A) Prices in thousands are rounded

B) Ridge cannot use prices below 1

C) R^2 changes whenever the units change

D) In thousands the price needs a weight 1,000 times larger, and the penalty shrinks it hard

> **Why:** The penalty adds up squared weights, so a column's units decide how much its effect costs. Results that change with units are a red flag: scale inside the pipeline.
> **A** is wrong: Dividing by 1,000 loses no information.
> **B** is wrong: Ridge accepts any numbers.
> **C** is wrong: Plain least squares gives the same R^2 in any units; the penalty is what is unit-sensitive.

Q11

What does this print?

```
from sklearn.linear_model import Ridge
m = Ridge(alpha=1e6).fit([[0], [1], [2], [3]], [10, 12, 14, 16])
print(m.coef_.round(3), round(m.intercept_, 1))
```

`[0.] 13.0`

> **Why:** A huge alpha pulls the slope to (almost) zero. The intercept is not penalized, so the model predicts the mean of y, 13.0, for every x: maximum bias.

Q12

RidgeCV chose alpha = 3.16 on the 400-row pilot but alpha = 1.0 on the full 5,088 rows of 2024. Why less penalty with more data?

A) Cross-validation is unreliable on small data, so the first number is wrong

B) More rows reduce variance on their own, so less penalty is needed to control it

C) Ridge always prefers alpha = 1

D) The full year has fewer columns

Why: More data is also a regularizer: with 5,088 rows the 489 columns are far less dangerous. That is why you choose alpha by cross-validation each time instead of reusing a number.
A is wrong: Both choices are sensible for their data; they differ because the data differ.
C is wrong: The default happens to be 1.0; RidgeCV searches the list it is given.
D is wrong: The design has 489 columns in both cases.

3.2 · REGULARIZATION AND THE BIAS-VARIANCE TRADE-OFF: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
import numpy as np
from sklearn.preprocessing import PolynomialFeatures
pf = PolynomialFeatures(degree=2, interaction_only=True, include_bias=False).fit(np
print(pf.n_output_features_)
```

`10`

Why: 4 originals plus 6 products of pairs (4 × 3 / 2); `interaction_only=True` leaves out the squares and `include_bias=False` the column of ones.

P4

Predict the output, then run it.

```
from sklearn.linear_model import LinearRegression, Ridge
X, y = [[0], [1], [2], [3]], [10, 12, 14, 16]
print(LinearRegression().fit(X, y).coef_, Ridge(alpha=10).fit(X, y).coef_.round(2))
```

```
[2.] [0.67]
```

> **Why:** The points lie exactly on y = 10 + 2x, so least squares finds slope 2. Ridge with alpha = 10 pays for the weight and settles for 0.67: bias bought on purpose (here there is no variance to save).

LAB REFERENCE SOLUTIONS

LAB 3.1-A: DAILY_SALES_LINE

Starter and solution in `labs/module-06/l3_1-linear-logistic/`: `daily_sales_line_starter.py` → `daily_sales_line_solution.py`. The solution prints:

```
2924 training days x stores, 1460 test
store's mean          MAE $1,670
store's weekday mean  MAE $1,292

Boise by hand: $1,333 per year, start $6,037
Boise sklearn: $1,333 per year, start $6,037

linear regression     MAE $949
cat__dow_Sat               3509.0
cat__dow_Tue               -168.0
num__trend                  487.0
num__promo                 2625.0
cat__event_Black Friday    2083.0

promo                 0   1
event
Black Friday          0  32
Closed               16   0
Holiday rush          0  48
Labor Day sale        0  32
Memorial Day sale     0  32
none               2764   0
same predictions without promo: True

log-target model       MAE $853
cat__dow_Sat               74.4
cat__dow_Tue               -5.3
num__trend                  8.1
cat__event_Black Friday    82.1
```

Reading it: the model beats the better baseline by about $340 a day. Coefficients are measured against an Asheville Monday in January with no event: a Saturday adds about $3,500. `promo` is 1 on exactly the sale and holiday-rush days, so promo and those event columns are twins: dropping `promo` leaves every prediction unchanged and the $2,625 vs $2,083 split is arbitrary. The log target reads as percentages (Saturday +74%, +8.1% a year) and lowers the error to $853.

LAB 3.1-B: FRAUD_ODDS

Starter and solution in `labs/module-06/l3_1-linear-logistic/`: `fraud_odds_starter.py` → `fraud_odds_solution.py`. The solution prints:

```
35000 training payments, 307 frauds

z = -5.984 + 1.515 * velocity_24h
 velocity_24h  p_fraud  by_hand
            0   0.0025   0.0025
            1   0.0113   0.0113
            2   0.0496   0.0496
            3   0.1917   0.1917
            5   0.8308   0.8308
odds ratio per extra payment in 24 hours: 4.55

                           coef  odds_ratio
device_new                 1.96        7.11
night                      1.94        6.93
billing_shipping_mismatch  1.36        3.91
country_mismatch           2.19        8.94
amount_100                 0.27        1.31
card_attempts              1.84        6.31
hour as a number  coef -0.111   odds ratio 0.90
night flag        coef +2.354   odds ratio 10.53

predict == (z > 0): True
flagged at 0.5: 21, of which fraud: 18
test AP 0.332
```

Reading it: without a scaler each odds ratio is per unit: a new device multiplies the odds by about 7, each extra $100 by 1.31, each extra 24-hour payment by 4.55 in the one-feature model. Hour as a straight line gets a small negative weight (0.90 per hour) because fraud clusters at 0 to 5 o'clock and a line cannot say "night"; the flag gets 10.5. At 0.5 the model flags only 21 payments: Lesson 2.5's timid default cut.

LAB 3.2-A: CHURN_CURVES

Starter and solution in `labs/module-06/l3_2-regularization/`: `churn_curves_starter.py` → `churn_curves_solution.py`. The solution prints:

```
1000 pilot customers
degree 1 ( 10 columns)  train AUC 0.823   CV AUC 0.807
degree 2 ( 65 columns)  train AUC 0.899   CV AUC 0.807
degree 3 (285 columns)  train AUC 1.000   CV AUC 0.706
C=0.001  train AUC 0.830   CV AUC 0.808
C=0.01   train AUC 0.866   CV AUC 0.810
C=0.1    train AUC 0.914   CV AUC 0.809
C=1      train AUC 0.946   CV AUC 0.776
C=10     train AUC 0.971   CV AUC 0.730
chosen C for the pilot: 0.01

C         0.1    inf
degree
1       0.830  0.829
2       0.851  0.851
3       0.851  0.827

chosen: degree 2, C=inf
test AUC chosen 0.853   plain degree 1 0.823
```

Reading it: on the pilot, degree 3 (285 columns) reaches training AUC 1.000 and CV 0.706: variance. A strong penalty (C = 0.01) rescues it to 0.810. On 4,500 customers degree 2 wins (0.851) with or without a penalty, because churn really has interactions; degree 3 still needs one. Degree 2 with C = inf and with C = 0.1 tie at 0.851; the loop keeps the first. The test AUC (0.853) agrees with CV and beats the plain model (0.823).

LAB 3.2-B: VARIANCE_WITHOUT_TRUTH

Starter and solution in `labs/module-06/l3_2-regularization/`: `variance_without_truth_starter.py` → `variance_without_truth_solution.py`. The solution prints:

```
2908 training rows, 1452 rows to predict; 406 columns

                 spread ($)  mean MAE ($)  worst MAE ($)
least squares        1217.0        1351.0         1739.0
ridge alpha=1         526.0        1089.0         1175.0
ridge alpha=10        275.0        1122.0         1232.0
ridge alpha=100       167.0        1527.0         1633.0
500-row pilots, least squares  spread $486
500-row pilots, ridge alpha=1  spread $384
```

Reading it: the spread is variance measured directly: how much a forecast for the same day moves when the pilot changes. It falls steadily as alpha grows. The mean MAE falls, then rises (alpha = 100 is stable and wrong: bias). Bias needs the true curve, which real data never gives; the test error is the honest stand-in. With 500-row pilots the spread shrinks on its own: more data is also a regularizer.

CHECKPOINT 1: RUBRIC, MODEL ANSWER AND WEAK ANSWERS

THE QUESTION

Pinecrest's marketing team will email a win-back voucher to the customers a churn model flags. An email with a voucher costs **$5**; a churner nobody reaches costs about **$50** of margin. A colleague's notebook (Part A of `labs/module-06/checkpoint-1/checkpoint1_starter.py`) reports about 95% accuracy. **Plan** what you will check; **audit** the colleague's code (one line per problem); **rebuild** an honest model in a pipeline, cross-validated against a baseline with a metric suited to a rare class, and choose who gets the email from the prices, on the training rows only; look at the test set **once**; **report** three sentences for the marketing lead.

Rubric. Five rows, four levels, 20 points. "Proficient" in every row (15 points) is a successful checkpoint. Circle one cell per row and write one sentence of advice.

CRITERION	1 · BEGINNING	2 · DEVELOPING	3 · PROFICIENT	4 · EXEMPLARY
Leakage audit	Accepts the 95% or finds nothing wrong.	Finds one problem (usually "no baseline") but misses the leak.	Names `cancellation_reason` as a leak (filled only after a customer leaves) and at least two more problems: preprocessing fitted before the split, accuracy with no baseline, one unstratified split.	Also proves the leak with a number (625 filled, 100% churners) and says why the 0.5 cut ignores the prices.
Pipeline and preprocessing	Fits the scaler or fills gaps on all rows, or keeps the leak column.	Leak removed, but some preprocessing still happens outside the pipeline.	Every step that learns (imputers, scaler, encoder) is inside one pipeline fitted on the training rows; the canonical split is used.	Clean, reusable `make_model()`; categorical gaps (`age_group`) handled on purpose, not by accident.
Honest evaluation	Reports test accuracy only.	Cross-validates, but with accuracy or without a baseline.	5-fold stratified CV on the training rows with ROC AUC and AP, each next to a dummy baseline (0.5 and 0.165); the test set touched once.	Also reports the fold spread and checks that the test numbers agree with CV (0.826 against 0.832).

CRITERION	1 · BEGINNING	2 · DEVELOPING	3 · PROFICIENT	4 · EXEMPLARY
Decision from costs	Uses `predict()` at 0.5 without comment.	Mentions costs but picks a threshold by eye or on the test set.	A cost function with the right `ravel()` order and a threshold from the formula ($5 / $50 = 0.1) or `TunedThresholdClassifierCV` on the training rows; test cost compared with emailing nobody and everyone.	Also compares with the default cut ($9,630) and notes that the formula needs calibrated probabilities.
Communication	Retells the steps; no answer.	An answer, but with no numbers or only generic caveats.	Three plain sentences: why 95% was wrong, what the model does (emails, churners reached, cost against the alternatives), two specific caveats.	Also never claims more than the data allow (the voucher will not save every churner it reaches; costs are estimates).

Quick read when time is short: read the three sentences first, then look in the code for a pipeline, a dummy baseline next to AUC or AP, and a threshold chosen on the training rows.

MODEL ANSWER (FROM A REAL RUN)

The file is `labs/module-06/checkpoint-1/checkpoint1_solution.py`: the plan as comments, then six short cells; about 25 minutes at a learner's pace. Its complete output:

```
cancellation_reason filled for 625 customers; churn among them 100%
baseline             CV ROC AUC 0.500   CV AP 0.165
logistic regression  CV ROC AUC 0.832   CV AP 0.528

threshold 0.107 (formula 0.10)
test ROC AUC 0.826   AP 0.493
emails 654 of 1500, churners reached 214 of 248
cost $4,970   email nobody $12,400   email everyone $7,500   at 0.5 $9,630

The 95% figure came from a column that is filled in only after a customer has left; wit
ranks customers well (ROC AUC about 0.83, against 0.50 for guessing) but is not "95% ac
Emailing the 654 test customers above the cost-based cut-off reaches 214 of the 248 chu
$4,970, against $7,500 for emailing everyone, $9,630 at the default cut-off and $12,40(
Caveats: the $5 and $50 are estimates (the cut-off moves if they change), and the vouch
every churner it reaches, so the real saving is smaller.
```

The colleague's notebook prints `colleague's accuracy: 0.947`. Problems a full audit lists: (1) `cancellation_reason` is filled only for churners (625 of 625): a leak; (2) gaps filled with medians of all rows, `get_dummies` and the scaler fitted on all 6,000 rows before the split; (3) accuracy with no baseline (0.835 for doing nothing); (4) one unstratified split, no cross-validation; (5) `predict` at 0.5 ignores the $5 and $50 prices ($9,630 against $4,970).

COMMON WEAK ANSWERS AND WHAT THEY REVEAL

- **"Accuracy 0.85 after removing the leak, so the model works."** Doing nothing scores 0.835. Ask for the baseline and a ranking metric.
- **The leak removed, but preprocessing still runs on all 6,000 rows** (`pd.get_dummies`, `StandardScaler`). The test rows shaped the preprocessing. The number moves only a little, but it is the habit the checkpoint tests.
- **A threshold read off the test-set cost curve.** That threshold is fitted to the test set. Ask which rows chose it.
- **The confusion-matrix order wrong** (`tn, fp, fn, tp`). Costs come out absurd (tens of thousands). Ask them to price "email nobody" first: it must be 248 × $50 = $12,400.
- **SMOTE or class weights added "because churn is imbalanced", then the formula on those probabilities.** The cut-off moves to almost everyone. Lesson 2.5's Mistake 3.
- **A report that lists steps ("I imputed, I scaled, I cross-validated").** Ask for the first sentence the marketing lead will read.

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 3.1 · LINEAR AND LOGISTIC REGRESSION

1. **B** `2.0 1.0`: the points lie exactly on y = 1 + 2x.
2. **A** One SD more days multiplies the odds of churning by about exp(0.88) ≈ 2.4, others held constant.
3. **C** Model `log1p(units_sold)` with `TransformedTargetRegressor`: the exponential is never negative.
4. **C** A probability of 0.5: exactly on the boundary.
5. **A** Predictions stay the same; the store and floor-area coefficients become arbitrary.

LESSON 3.2 · REGULARIZATION AND THE BIAS-VARIANCE TRADE-OFF

1. **B** High variance: it fits the training rows but does not generalize.
2. **C** 9: three originals, three squares, three products.
3. **A** All weights shrink toward zero; predictions approach a constant.
4. **B** Nothing about floor area: Lasso kept its twin `sqft`.
5. **C** `RidgeCV` with cross-validation on the training set, then score the test set once.

EXIT TICKET

1. **0.67 for discount_share:** one standard deviation more discount reliance multiplies the odds of churning by about 1.96, other features held constant. It does not say discounts cause churn, nor that the probability rises by 0.67.
2. **Tiny training error, large test error:** high variance (overfitting). Fixes: a stronger penalty chosen by CV, fewer or simpler features, more data.
3. **Scaling:** the penalty charges for the size of each weight; without scaling, a feature's units decide how much it is penalized.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q2 A ("0.42 more units").** Forgets the log target. Ask what scale the model trained on.
- **Q3 A or B.** Probability and odds mixed up. Compute one example: from p = 0.10 (odds 0.11) doubling the odds gives p ≈ 0.18, not 0.20 or 0.79.
- **Q6 A.** Has seen the error in the lesson and assumes it always appears. Integers are silently treated as classes.
- **Q7 A or B.** Thinks more flexibility fixes a bad test score. Draw the U.
- **Q9 A.** "Lasso dropped it, so it does not matter": the same error as "dropped means irrelevant" in Lesson 2.4.
- **Exit ticket 1: "discounts cause churn".** Ask whether anyone changed a customer's discounts and watched.

INSTRUCTOR KIT · ANSWER KEY

Session 7: Nearest Neighbors and Naive Bayes

Answers and reasoning for every Session 7 item: the warm-up, both self-checks, the predict-then-run items, the lab reference outputs, the debug handout, the lesson pages' own quizzes and the exit ticket.

Session 7 | 12 self-check items | Confidential

WARM-UP

W1

A log-target demand model gives `holiday_week` a coefficient of 0.405. In words?

A holiday week sells about exp(0.405) − 1 ≈ 50% more, for every product.

Why: Percent effects come back today as likelihood ratios that multiply.

W2

A logistic regression flag has coefficient 0.69. What does this print, and what does it mean?

```
import numpy as np
print(round(np.exp(0.69), 2))
```

`1.99`

Why: The odds ratio: the flag about doubles the odds. Naive Bayes today also multiplies odds, one clue at a time.

W3

Degree 15 has training MSE 0.016 and test MSE 0.187 (noise floor 0.09). Which problem, and which dial helps?

High variance (overfitting); a penalty such as Ridge's alpha, fewer features or more data.

Why: Today's dial is k: k = 1 is the degree-15 model of nearest neighbors.

W4

Temperature in °F and °C: Ridge gives [-0.607, -0.607], Lasso [-1.164, 0]. What does Lasso's zero mean?

Only that the column was redundant given its twin, not that it has no effect.

> **Why:** Naive Bayes has the opposite problem with twins: it counts both.

W5

Why must features be scaled before Ridge or Lasso?

The penalty charges for weight size, and a feature's units decide how big its weight must be.

> **Why:** For kNN scaling is even more basic: it defines what "similar" means.

4.1 · K-NEAREST NEIGHBORS: SELF-CHECK

Q1

What does this print?

```
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=3).fit([[0], [1], [2], [6], [7]], [1, 1, 0,
print(knn.predict_proba([[1.2]]).round(3))
```

`[[0.333 0.667]]`

> **Why:** The three training points nearest 1.2 are 1 (0.2 away), 2 (0.8) and 0 (1.2), with labels 1, 0 and 1. The probabilities are the vote shares in the order of `classes_`: class 0 1/3, class 1 2/3.

Q2

As `n_neighbors` grows from 1 to 501 on the churn data, what happens?

A) Variance falls and bias rises: predictions drift toward the overall churn rate

B) Variance rises: more neighbors means more noise in each vote

C) Nothing changes once the features are scaled

D) The model trains more slowly, because it learns more weights

> **Why:** k = 1 copies one neighbor (training AUC 1.000, test 0.645: variance); a huge k averages so many customers that every prediction approaches 16.5% (bias). The test AUC peaks on a plateau around 50 to 150.
> **B** is wrong: Averaging more neighbors smooths the vote; noise shrinks.
> **C** is wrong: Scaling decides who the neighbors are; k decides how many vote. Both matter.
> **D** is wrong: kNN learns no weights; fit only stores the rows, whatever k is.

Q3

Unscaled kNN on fraud scored AP 0.056 (scaled: 0.397). The first two training payments' squared distance was 99.7% account age. What went wrong?

A) kNN cannot handle rare classes

B) 51 neighbors is too many for fraud

C) Account age runs up to 7,030 days, so it alone decided who the neighbors were; the flags that signal fraud had no say

D) Average precision does not work for kNN

> **Why:** A distance adds up raw differences. Without scaling, the column with the biggest numbers defines similarity. Scaling gives every column an equal say to begin with.
> **A** is wrong: Scaled kNN on the same rare class reached AP 0.397.
> **B** is wrong: k was the same in both runs; only scaling changed.
> **D** is wrong: AP needs only a ranking; kNN's vote shares provide one.

Q4

Fifty columns of random noise lower kNN's churn AUC from 0.842 to 0.755 but logistic regression's only from 0.823 to 0.816. Why?

A) kNN needs more training iterations when there are more columns

B) kNN gives every scaled column an equal say in the distance; logistic regression learns near-zero weights for useless columns

C) Logistic regression scales its own inputs

D) The noise columns were not scaled for kNN

> **Why:** kNN has no weights to switch a column off, so every irrelevant column blurs who the true neighbors are (the curse of dimensionality). For kNN, feature selection is part of the model.
> **A** is wrong: kNN does not iterate: fit only stores the data.
> **C** is wrong: Both models had the same StandardScaler in front.
> **D** is wrong: Both pipelines scaled every column.

Q5

What does this print?

```
from sklearn.metrics import pairwise_distances
print(pairwise_distances([[1, 2, 3]], [[4, 0, 3]], metric="manhattan"))
```

`[[5.]]`

> **Why:** Manhattan distance adds the absolute differences: $|1 - 4| + |2 - 0| + |3 - 3| = 3 + 2 + 0 = 5$. (Euclidean would be $\sqrt{13} \approx 3.61$.)

Q6

Pinecrest wants to put the lookalike churn model into a nightly scoring job. What must ship with the fitted kNN model?

A) Only the scaler's means and standard deviations

B) One weight per feature, like logistic regression

C) Nothing: kNN predicts from the new customer alone

D) The scaler and every training customer, because each prediction searches them

> **Why:** kNN is a lazy learner: fit stores the training table, and predict searches it. Prediction time grows with the training set, and the explanation by lookalikes is other customers' data, which is also a privacy question.
> **A** is wrong: The scaler is needed, but so are the rows it scales.
> **B** is wrong: kNN has no weights.
> **C** is wrong: A prediction is a vote of stored neighbors.

4.1 · K-NEAREST NEIGHBORS: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
from sklearn.neighbors import KNeighborsRegressor
reg = KNeighborsRegressor(n_neighbors=2).fit([[1], [2], [4], [8]], [10, 20, 40, 80]
print(reg.predict([[3]]))
```

`[30.]`

> **Why:** 2 and 4 are both 1 away from 3; their targets 20 and 40 average to 30. kNN regression averages the neighbors' targets; it cannot predict outside their range.

P2

Predict the output, then run it.

```
from sklearn.metrics import pairwise_distances
pts = [[0, 0], [3, 4]]
print(pairwise_distances(pts)[0, 1], pairwise_distances(pts, metric="manhattan")[0,
```

`5.0 7.0`

Why: Euclidean: $\sqrt{(3^2 + 4^2)} = 5$. Manhattan: 3 + 4 = 7, walking the grid. Manhattan is never shorter.

4.2 · NAIVE BAYES: SELF-CHECK

Q7

20% of reviews are negative. The word "flimsy" appears in 10% of negative reviews and 1% of positive ones. What is P(negative | "flimsy")?

A) 0.10

B) 0.20, the prior does not change

C) 0.50

D) About 0.71

Why: Prior odds 0.2 / 0.8 = 0.25; the likelihood ratio 10% / 1% = 10 multiplies them to 2.5; 2.5 / 3.5 ≈ 0.71.
A is wrong: That is P(word | negative), the likelihood, not the posterior.
B is wrong: The word appears ten times as often in negative reviews; the belief must move.
C is wrong: That would need a likelihood ratio of 4.

Q8

What does this print?

```
from sklearn.feature_extraction.text import CountVectorizer
v = CountVectorizer()
print(v.fit_transform(["not bad not", "bad"]).toarray())
```

`[[1 2] [1 0]]`

Why: The vocabulary is sorted alphabetically: bad, not. The first text has bad once and not twice; the second only bad. Counts, not presence (that would need `binary=True`).

Q9

With `MultinomialNB(alpha=1e-10)` an angry review that says "leaked", "fell apart" and "returned" but also "love" comes out 100% positive. Why?

A) "love" never appeared in a negative training review, so with almost no smoothing its probability in that class is about 0, and one zero factor vetoes everything else

B) "love" is the strongest word in the vocabulary

C) The model ignores negative words

D) Small alpha makes the model prefer the majority class

> **Why:** Naive Bayes multiplies one factor per word. Smoothing (alpha = 1) adds a pretend count to every word in every class, so no word can veto the rest: the same review comes out 97% negative.
> **B** is wrong: Its log ratio is large, but with smoothing the other words outweigh it.
> **C** is wrong: The negative words are counted; they are cancelled by a zero factor.
> **D** is wrong: Alpha smooths word counts, not the class prior.

Q10

`make_pipeline(StandardScaler(), MultinomialNB())` on the churn columns raises `ValueError: Negative values in data passed to MultinomialNB (input X).` What should you use?

A) A MinMaxScaler, so the values are between 0 and 1

B) ComplementNB, which accepts negative counts

C) GaussianNB, unscaled: it models continuous measurements with a bell curve per class

D) Take absolute values before MultinomialNB

> **Why:** Multinomial, Complement and Bernoulli NB expect counts or 0/1 flags. Continuous measurements belong in GaussianNB, which needs no scaling because each feature is compared with its own per-class mean and spread.
> **A** is wrong: It would run, but churn columns are not word counts; the model's assumptions would still be wrong.
> **B** is wrong: ComplementNB also expects non-negative counts.
> **D** is wrong: Absolute values would merge customers above and below average: nonsense.

Q11

GaussianNB on 10 churn columns scores AUC 0.778; on 6 (without spend, average order value and the two returns columns) 0.813. Why does dropping columns help?

A) Six columns train faster

B) The dropped columns repeat information already present, and Naive Bayes counts each one as a new, independent clue

C) The dropped columns had missing values

D) GaussianNB can only use six columns

> **Why:** Spend is orders × average order value; returns_12m overlaps return_rate. The naive assumption counts the same evidence two or three times, which distorts the scores. Removing duplicates stops the double counting.
> **A** is wrong: Speed does not change AUC.
> **C** is wrong: None of those four columns has gaps.
> **D** is wrong: It takes any number of columns.

Q12

What does this print?

```
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS
print("not" in ENGLISH_STOP_WORDS, "boots" in ENGLISH_STOP_WORDS)
```

`True False`

> **Why:** scikit-learn's built-in English stop-word list contains "not" (and "no", "never"), the words that carry negative sentiment. Removing stop words cost accuracy in the lesson (0.849 against 0.857).

4.2 · NAIVE BAYES: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
from sklearn.feature_extraction.text import CountVectorizer
v = CountVectorizer(binary=True)
print(v.fit_transform(["warm warm boots", "boots"]).toarray())
```

```
[[1 1] [1 0]]
```

> **Why:** Vocabulary boots, warm. With `binary=True` "warm warm" counts once: presence, the input BernoulliNB expects.

P4

Predict the output, then run it.

```
prior, ratio = 0.25, 3            # P(negative) and P(word | neg) / P(word | pos)
odds = prior / (1 - prior) * ratio
print(round(odds / (1 + odds), 2))
```

```
0.5
```

> **Why:** Prior odds 0.25 / 0.75 = 1/3; times 3 = 1; odds of 1 mean a probability of 0.5.

LAB REFERENCE SOLUTIONS

LAB 4.1-A: FRAUD_NEIGHBORS

Starter and solution in `labs/module-07/l4_1-knn/`: `fraud_neighbors_starter.py` → `fraud_neighbors_solution.py`. The solution prints:

```
(35000, 9) random-ranking AP 0.0087

share of the squared distance, first two payments:
account_age_days    0.997
amount              0.003
hour                0.000

kNN unscaled         ROC AUC 0.659   AP 0.056
kNN scaled           ROC AUC 0.812   AP 0.397
logistic regression  ROC AUC 0.848   AP 0.444
k=  5   CV AP 0.362 +/- 0.065
k= 51   CV AP 0.448 +/- 0.069
k=201   CV AP 0.468 +/- 0.066
k=  5:   6 distinct probabilities; 14,588 test payments at exactly 0; test AP 0.329
k=201:  43 distinct probabilities; 8,240 test payments at exactly 0; test AP 0.419
```

Reading it: unscaled, account age (up to 7,030 days) is the distance, and AP collapses to 0.056, barely above random (0.0087). Scaled, AP 0.397. A larger k ranks better (0.468 at 201 by CV): with k = 5 almost every payment has 0, 1 or 2 fraud neighbors, so most of the ranking is ties. Tuned kNN (test AP 0.419) still trails logistic regression (0.444): each of nine columns has an equal vote, including weak ones.

LAB 4.1-B: LOOKALIKE_DAYS

Starter and solution in `labs/module-07/l4_1-knn/`: `lookalike_days_starter.py` → `lookalike_days_solution.py`. The solution prints:

```
731 training days, 365 days to forecast
weekday mean             MAE $1,669
raw, unscaled            MAE $1,889
raw, scaled              MAE $1,327
cycles + events, k= 1   MAE $1,389   mean error (actual - forecast) $+517
cycles + events, k= 5   MAE $1,244   mean error (actual - forecast) $+921
cycles + events, k=15   MAE $1,256   mean error (actual - forecast) $+1,016

Boise's mean daily sales: 2023 $6,831, 2024 $7,821, 2025 $8,342
2024 only, k= 5           MAE $1,202   mean error $+458
```

Reading it: unscaled, day-of-year (1 to 366) swamps the weekday (0 to 6), and kNN is worse than the weekday baseline. Scaling, then cyclical features that put Sunday next to Monday and December next to January, bring the error to $1,244. Every forecast is too low on average: Boise grew about $500 to $1,000 a day each year, and an average of past days cannot reach a level it has never seen. Training on 2024 only halves the bias and lowers the error, at the cost of fewer lookalikes.

HANDOUT: LABS/MODULE-07/L4_1-KNN/DEBUG_KNN.PY (DELIBERATELY BROKEN)

1. `ValueError: Expected n_neighbors <= n_samples_fit, but n_neighbors = 51, n_samples_fit = 40, n_samples = 3` → the smoke test fits 51 neighbors on 40 rows; delete it or use k

below 40.

2. `ValueError: Input X contains NaN.` → `email_opens_90d` has gaps; add `SimpleImputer(strategy="median")` to the pipeline.
3. No error: no `StandardScaler` in the pipeline. CV AUC 0.787 instead of 0.854.
4. No error: `knn[-1].kneighbors(customer)` searches with an unscaled row: 39 of 51 lookalikes churned, median 364 days, for a customer at 53 days. Fix: `knn[-1].kneighbors(knn[:-1].transform(customer))`. Fixed output: `CV ROC AUC 0.854` and `3 of 51 lookalikes churned; ... median ... 46 (the customer's: 53)`.

LAB 4.2-A: FRAUD_FLAGS_BAYES

Starter and solution in `labs/module-07/l4_2-naive-bayes/`: `fraud_flags_bayes_starter.py` → `fraud_flags_bayes_solution.py`. The solution prints:

```
prior P(fraud) = 0.0088

P(country_mismatch | fraud) 0.137   P(country_mismatch | legit) 0.0121
posterior by Bayes' rule 0.091   by counting 0.091

likelihood ratio P(flag | fraud) / P(flag | legit):
country_mismatch             11.3
busy_24h                     10.1
night                         8.1
gift_card                     6.5
multi_attempt                 6.2
device_new                    5.4
big_amount                    3.9
billing_shipping_mismatch     3.7
new_account                   1.9

night + device_new: naive 0.280   counted 0.425   (193 payments)
big_amount + gift_card: naive 0.185   counted 0.508   (59 payments)

BernoulliNB          ROC AUC 0.810   AP 0.439   above 0.9: 21 payments, 100% fraud
logistic regression  ROC AUC 0.810   AP 0.435   above 0.9: 8 payments, 100% fraud
```

Reading it: with one flag, Bayes' rule and counting agree exactly. With two flags the naive product is too low (0.280 against 0.425; 0.185 against 0.508): among frauds, night-time payments come from new devices more often than independence predicts, so the flags reinforce each other. In reviews the naive assumption over-counts; here it under-counts. BernoulliNB on nine flags ranks like logistic regression (AP 0.439 against 0.435) and is more extreme: 21 payments above 0.9 against 8.

LAB 4.2-B: REVIEW_CATEGORY

Starter and solution in `labs/module-07/l4_2-naive-bayes/`: `review_category_starter.py` → `review_category_solution.py`. The solution prints:

```
3000 train, 1000 test; 10 categories
{'Apparel': 948, 'Accessories': 527, 'Paddling': 518}
baseline             accuracy 0.237
MultinomialNB        accuracy 0.586
ComplementNB         accuracy 0.592
logistic regression  accuracy 0.605
alpha=0.1  CV accuracy 0.583
alpha=0.3  CV accuracy 0.582
alpha=1.0  CV accuracy 0.579
alpha=3.0  CV accuracy 0.580
Tents     ['canopy', 'camp', 'tent', 'person', 'ultralight']
Paddling  ['deck', 'paddle', 'leash', 'bag', 'life']
Footwear  ['hiking', 'socks', 'shoes', 'boots', 'approach']

most frequent mix-ups (true -> predicted):
Camp Kitchen  Accessories    20
Apparel       Paddling       20
Paddling      Apparel        19
recall by category: {'Packs': 0.39, 'Camp Kitchen': 0.43, 'Footwear': 0.49}
```

Reading it: with ten classes, 58.6% is far above the 23.7% baseline. Category words (canopy, paddle, boots) carry it; generic review phrases (shipping, price, fit) are shared by all categories, which is why accuracy stops near 60%. The mix-ups make sense: Camp Kitchen items look like Accessories, and Apparel and Paddling reviews share the same aspect phrases about fit and waterproofing. ComplementNB helps slightly with the unbalanced classes; alpha hardly matters.

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 4.1 · K-NEAREST NEIGHBORS

1. **B** `[[0.667 0.333]]`: the three nearest points to 1.4 are 1, 2 and 0 with labels 0, 1, 0.
2. **A** Overfitting: each training customer is its own nearest neighbor.
3. **C** Spend, simply because its numbers are bigger (90,000 against 14,400 in the squared distance).
4. **B** kNN gives every column an equal say; logistic regression learns to ignore useless columns.
5. **C** kNN trains almost instantly but must keep and search all the training rows at prediction time.

LESSON 4.2 · NAIVE BAYES

1. **B** It rises from 0.25 to about 0.8 (likelihood ratio 12).
2. **C** The model treats the new column as independent evidence and counts the same information twice.

3. **A** `[[0 2 1] [1 0 1]]`: vocabulary boots, dry, warm.
4. **C** It pushes toward positive, but other words can still make the review negative.
5. **B** Rank customers with it, but recalibrate before quoting its probabilities.

EXIT TICKET

1. **Scaling:** kNN's distance adds raw differences, so without scaling the units decide similarity; MultinomialNB models counts, and scaling creates negative values it refuses.
2. **k = 1:** high variance: every training customer is its own nearest neighbor, so training AUC is meaningless. Choose a larger k (around 50 here) by cross-validation.
3. **Posterior:** prior odds 0.25 / 0.75 = 1/3; likelihood ratio 8 / 2 = 4; posterior odds 4/3; probability 4/7 ≈ 0.57.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 written with the two numbers swapped** (`[[0.667 0.333]]`). Reads the columns in the wrong order: they follow `classes_`, 0 then 1.
- **Q2 B.** Thinks more neighbors means more noise; draw 1 neighbor against 50.
- **Q4 C or D.** Assumes logistic regression is protected by scaling; both were scaled. The difference is weights.
- **Q7 A.** Gives the likelihood, not the posterior. Write prior × likelihood ratio in odds.
- **Q10 A.** "Make it positive and it runs": running is not the same as modeling the right thing.
- **Exit ticket 3 written as 0.08 or 0.8.** Same confusion as Q7.

INSTRUCTOR KIT · ANSWER KEY

Session 8: Support Vector Machines and Decision Trees

Answers and reasoning for every Session 8 item: the warm-up, both self-checks, the predict-then-run items, the lab reference outputs, the debug handouts, the lesson pages' own quizzes and the exit ticket.

Session 8 | 12 self-check items | Confidential

WARM-UP

Session 7 retrieval. The answers below are also in the guide, with what to say next.

W1

Lesson 4.1: a kNN churn model scores a perfect 1.000 on its training data with k=1. Why, and what do you do about it?

Each training customer is its own nearest neighbor, so k=1 is right by construction (high variance). Choose a larger k by cross-validation; the 4.1 project chose k=51.

> **Why:** Ask for the word: overfitting. Today's two models have their own versions: gamma for SVMs, depth for trees.

W2

Two unscaled features: spend differs by $300 and days since last order by 120. What share of the squared distance comes from spend? Predict, then check.

```
import numpy as np
spend, days = 300, 120
print(round(spend ** 2 / (spend ** 2 + days ** 2), 2))
```

`0.86` (86%)

> **Why:** 300^2 = 90,000 against 120^2 = 14,400: spend takes 86% of the distance purely because of its units. The same reason SVMs need a scaler today.

W3

Lesson 4.2: a word appears in 6% of negative reviews and 0.5% of positive ones; 25% of reviews are negative. After seeing the word, about how likely is the review to be negative?

About 0.8: the odds go from 1:3 to (1 × 6) : (3 × 0.5) = 6 : 1.5 = 4 : 1.

> **Why:** Prior odds times the likelihood ratio (12). Anyone stuck: draw the 1,000-review table.

W4

Why does MultinomialNB need alpha above 0?

Without smoothing a word never seen in one class gives that class a probability of exactly 0, which vetoes all the other evidence.

> **Why:** Then bridge: today's SVM on reviews (Lab A) needs no smoothing, but it needs scaling, in its text form (TF-IDF).

W5

Your Naive Bayes model ranks customers well (AUC 0.81) but its probabilities pile up near 0 and 1. Can you quote them to the care team?

Rank with it, but recalibrate (CalibratedClassifierCV) before quoting probabilities.

> **Why:** Today's SVM gives no probabilities at all, and the same tool fixes it.

4.3 · SUPPORT VECTOR MACHINES: SELF-CHECK

Q1

On overlapping classes you lower a linear SVM's C from 1 to 0.01. What do you expect?

A) A narrower street and fewer support vectors

B) A wider street held up by many more support vectors

C) No change: C only matters for the RBF kernel

D) A perfect training score, because violations are now free

Why: C is the price of a margin violation. Cheap violations let the SVM widen the street and accept many points inside it, and every point on or inside the margin is a support vector. The lesson's blobs: 11 support vectors and width 1.34 at C=1, 42 and 4.06 at C=0.01.

A is wrong: That is what RAISING C does (C=100: 7 support vectors, width 0.75).

C is wrong: C prices violations for every kernel, linear included.

D is wrong: Free violations make the model simpler, not a better fit to the training points.

Q2

What does this print?

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
pipe = make_pipeline(StandardScaler(), SVC())
print([key for key in pipe.get_params() if key.endswith("gamma")])
```

`['svc__gamma']`

Why: `make_pipeline` names each step after its class in lowercase, and a nested parameter is the step name, two underscores, then the parameter. This is the key a grid search needs; `gamma` alone raises `Invalid parameter`.

Q3

Why does an RBF `SVC` on the churn table need a `StandardScaler` in front of it?

A) The SVM cannot read integer columns

B) Scaling makes the SVM train much faster, nothing else changes

C) Without scaling the model cannot output probabilities

D) The kernel and the margin are built on distances, and unscaled `spend_12m` and `days_since_last_order` would drown out the 0-to-1 columns

Why: Distances add up squared differences, so columns measured in hundreds dominate columns measured in fractions. In the lesson, scaling lifted the AUC from 0.767 to 0.814. Put the scaler inside the pipeline so cross-validation stays leak-free.
A is wrong: Integers are numbers; the SVM reads them fine.
B is wrong: Speed may change, but the reason is the distance: the AUC changes.
C is wrong: An `SVC` never outputs probabilities, scaled or not; that is what calibration is for.

Q4

What happens when this runs?

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
pipe = make_pipeline(StandardScaler(), SVC()).fit([[0], [1], [5], [6]], [0, 0, 1, 1
print(pipe.predict_proba([[3]]))
```

A) It prints `[[0.5 0.5]]`: the point is halfway

B) `AttributeError: This 'Pipeline' has no attribute 'predict_proba'`

C) It prints `[1]`

D) `NotFittedError`: the pipeline was not fitted

Why: An SVM scores signed distances, not probabilities, so `predict_proba` does not exist; the pipeline passes the question to its last step, hence "Pipeline" in the message. Rank with `decision_function`, or wrap the SVM in `CalibratedClassifierCV(SVC(), ensemble=False)` for probabilities.
A is wrong: No probabilities are computed at all.
C is wrong: That would be `predict`.
D is wrong: `.fit(...)` is chained on the same line, so the pipeline is fitted.

Q5

On churn (16.5% churners) you add `class_weight="balanced"` to an RBF SVM. In the lesson, what happened?

A) Recall of churners jumped (0.258 to 0.774) while accuracy fell (0.859 to 0.792)

B) Nothing: class weights only affect logistic regression

C) Accuracy rose and recall fell

D) The model started predicting churn for every customer

> **Why:** A mistake on a churner now costs about five times as much, so the boundary moves toward the stayers: many more churners are caught, more stayers are flagged, and the ranking (AUC 0.814 to 0.850) improves too.
> **B** is wrong: `SVC`, logistic regression and trees all accept `class_weight`.
> **C** is wrong: The opposite: the boundary moves to catch more churners.
> **D** is wrong: It flagged many more customers, not all of them; accuracy 0.792 is far above what flagging everyone (0.165) would score.

Q6

What does this print? (RBF similarity of two points at squared distance 4, for two values of gamma.)

```
import numpy as np
print(round(float(np.exp(-0.1 * 4)), 3), round(float(np.exp(-10 * 4)), 3))
```

`0.67 0.0`

> **Why:** Similarity is exp(-gamma × squared distance). With a small gamma the two points are still fairly similar (0.67): each support vector reaches far. With gamma=10 the similarity is essentially 0: each point's influence is a tiny bubble, the recipe for memorizing noise.

4.3 · SUPPORT VECTOR MACHINES: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
from sklearn.svm import SVC
m = SVC(kernel="linear", C=1).fit([[0], [1], [3], [4]], [0, 0, 1, 1])
print(m.support_vectors_.ravel(), m.predict([[1.8], [2.2]]))
```

```
[1. 3.] [0 1]
```

> **Why:** The street's center is at 2 and its curbs at 1 and 3, so the support vectors are exactly the points 1 and 3. 1.8 is left of the center (class 0), 2.2 right of it (class 1).

P2

Predict the output, then run it.

```
import numpy as np
distance_squared = 4
for gamma in [0.1, 1, 10]:
    print(gamma, round(float(np.exp(-gamma * distance_squared)), 4))
```

```
0.1 0.6703 1 0.0183 10 0.0
```

> **Why:** exp(-0.4) = 0.6703, exp(-4) = 0.0183, exp(-40) is essentially 0: the larger gamma, the faster similarity fades with distance.

5.1 · DECISION TREES: SELF-CHECK

Q7

What does this print? (A node with 30 stayers and 10 churners.)

```
import numpy as np
counts = np.array([30, 10])
p = counts / counts.sum()
print(round(float(1 - np.sum(p ** 2)), 3))
```

```
0.375
```

> **Why:** Shares 0.75 and 0.25: Gini = 1 - (0.5625 + 0.0625) = 0.375. Between a pure node (0) and a 50/50 node (0.5).

Q8

Where does a decision tree's predicted churn probability for a new customer come from?

A) A weighted sum of the features, squashed into 0 to 1

B) The distance from the new customer to the boundary

C) The share of churners among the training customers in the leaf the new customer lands in

D) The average of many trees' votes

> **Why:** Predicting means walking down the signposts to a campsite and reading its sign. That is why a depth-2 tree has only four possible probabilities, and an unlimited tree (pure leaves) gives only 0s and 1s.
> **A** is wrong: That is logistic regression.
> **B** is wrong: That is an SVM's decision_function.
> **D** is wrong: That is a random forest (Lesson 5.2).

Q9

What does this print?

```
from sklearn.tree import DecisionTreeClassifier
t = DecisionTreeClassifier(max_depth=1, random_state=42).fit([[1], [2], [8], [15]],
print(t.tree_.threshold[0])
```

A) `2.0`

B) `5.0`

C) `8.0`

D) `6.5`

> **Why:** A tree tests thresholds halfway between neighboring sorted values. The only split that separates the classes falls between 2 and 8, at their midpoint 5.0.
> **A** is wrong: The threshold is not one of the training values.
> **C** is wrong: Same: the split sits between values, not on one.
> **D** is wrong: 6.5 is the mean of the four values; thresholds are midpoints between neighboring values, not averages of the column.

Q10

An unlimited churn tree scores 1.000 on the training set and 0.822 on the test set, below the 0.835 baseline. What is the best next move?

A) Report 1.000: the tree learned the data perfectly

B) Scale the features, then refit

C) Switch the criterion from gini to entropy

D) Choose a size control such as `min_samples_leaf` by cross-validation on the training set

> **Why:** A perfect training score is a warning: the tree memorized individual customers. In the lesson, min_samples_leaf=50 chosen by CV lifted the CV AUC from 0.663 to 0.848, with 58 leaves instead of 523.
> **A** is wrong: A training score says nothing about new customers.
> **B** is wrong: Scaling cannot change a tree's predictions: thresholds just change units.
> **C** is wrong: The impurity measure matters far less than the size controls.

Q11

In the pruned churn tree, `spend_12m` gets an importance of 0.012, yet churn falls from 24% to 8% across spend quartiles. What is the right sentence?

A) The tree did not need spend: other features it split on first carried the same information

B) Spend has no relationship with churn

C) Raising a customer's spend would not change their churn risk

D) The importance is wrong because the features were not scaled

> **Why:** Impurity importance describes how THIS tree used each column. Spend overlaps with tenure, recency and orders, so whichever the tree picked first took the credit. Say "the model did not need it", never "it does not matter".
> **B** is wrong: The quartile table shows a clear relationship.
> **C** is wrong: Importance is not a causal statement either way.
> **D** is wrong: Trees do not need scaling, and importances do not depend on it.

Q12

What happens when this runs?

```
from sklearn.tree import DecisionTreeClassifier
DecisionTreeClassifier(max_depth=0).fit([[1], [2]], [0, 1])
```

A) A tree with no questions that predicts the majority class

B) A stump: one question

C) `InvalidParameterError`: max_depth must be at least 1 (or None)

D) An unlimited tree, because 0 means no limit

> **Why:** scikit-learn checks hyperparameters when you call fit. The smallest legal tree is a stump (max_depth=1); `None` means no limit; a no-questions model is `DummyClassifier`.
> **A** is wrong: That is `DummyClassifier`; the tree refuses depth 0.
> **B** is wrong: That would be max_depth=1.
> **D** is wrong: No limit is `None`, not 0.

5.1 · DECISION TREES: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
import numpy as np
def gini(counts):
    p = np.asarray(counts) / np.sum(counts)
    return round(float(1 - np.sum(p ** 2)), 3)
print(gini([50, 50]), gini([90, 10]), gini([100, 0]))
```

`0.5 0.18 0.0`

> **Why:** A 50/50 node is the most mixed (0.5); 90/10 gives 1 - (0.81 + 0.01) = 0.18; a pure node 0.

P4

Predict the output, then run it.

```
from sklearn.tree import DecisionTreeClassifier
X, y = [[1], [2], [3], [10], [11], [12]], [0, 0, 1, 1, 1, 1]
t = DecisionTreeClassifier(max_depth=1, random_state=42).fit(X, y)
print(t.tree_.threshold[0], t.predict_proba([[2.4], [11]]).round(2).tolist())
```

```
2.5 [[1.0, 0.0], [0.0, 1.0]]
```

> **Why:** The best split is between 2 and 3 (midpoint 2.5), which separates the classes perfectly, so both leaves are pure: 2.4 goes left (class 0 with probability 1.0), 11 right.

LAB REFERENCE SOLUTIONS

LAB 4.3-A: LABS/MODULE-08/L4_3-SVM/REVIEW_SVM_STARTER.PY → REVIEW_SVM_SOLUTION.PY

The solution prints:

```
2321 training reviews, 774 test reviews; first test id R02194
Naive Bayes (Lesson 4.2)     accuracy 0.868  ROC AUC 0.934
LinearSVC on raw counts      accuracy 0.859  ROC AUC 0.921
LinearSVC on TF-IDF          accuracy 0.884  ROC AUC 0.936
features (words and word pairs): 1856

parameter key: ['linearsvc__C']
C=0.01  CV ROC AUC 0.9077
C=0.1   CV ROC AUC 0.9253
C=1     CV ROC AUC 0.9269
C=10    CV ROC AUC 0.9066

most negative: {'not': -2.51, 'not happy': -1.99, 'never again': -1.93, 'would not': -1
most positive: {'expected it': 1.61, 'love it': 1.66, 'love': 1.66, 'late it': 1.79, 'w

test reviews inside the street (|score| < 1): 445 of 774
test reviews on the wrong side of the center line: 90

calibrated: average P(positive) 0.734 vs real share 0.745; ROC AUC 0.936
SVC (RBF) on TF-IDF          accuracy 0.876  ROC AUC 0.928  support vectors 1338 of 232
```

Reading it: on raw counts the linear SVM loses to Naive Bayes (0.921 vs 0.934 AUC): long reviews have big counts and dominate the margin, exactly the unscaled-feature problem. TF-IDF normalizes each review and the SVM pulls ahead (accuracy 0.884, AUC 0.936). C=1 is best by CV; C=10 overfits the 1,856 word features. The weights read like the reviews (`not`, `never again` against `would buy`, `love it`); `late it` is an artifact: "It arrived late. It ..." becomes the pair "late it" because the vectorizer ignores punctuation, and in these template reviews that

phrase happens to sit in more positive reviews. 445 of 774 test reviews are inside the street: text classes overlap a lot. The RBF kernel adds nothing on text: with this many features a straight boundary is already flexible.

LAB 4.3-B: LABS/MODULE-08/L4_3-SVM/KERNEL_BY_HAND_STARTER.PY → KERNEL_BY_HAND_SOLUTION.PY

The solution prints (fit times differ per machine):

```
LinearSVC, 10 features         test AUC 0.826   top-50 churners 33   fit 0.01 s

columns after the degree-2 lift: 66
a few of them: ['tenure_months', 'tenure_months^2', 'tenure_months days_since_last_orde
lift (deg 2) + LinearSVC       test AUC 0.852   top-50 churners 33   fit 0.05 s
SVC(kernel='poly', degree=2)   test AUC 0.852   top-50 churners 30   fit 0.30 s
support vectors: 2293 of 4500
degree 2:    66 columns
degree 3:   286 columns
degree 4:  1001 columns
degree 5:  3003 columns
SVC(kernel='rbf')              test AUC 0.850   top-50 churners 32   fit 0.30 s

largest lifted weights (absolute):
days_since_last_order    0.43
tenure_months^2          0.31
discount_share           0.25
app_user^2              -0.19
support_tickets_12m      0.17
```

Reading it: building every square and pairwise product (66 columns including the constant) lifts a straight boundary from 0.826 to 0.852; the polynomial kernel reaches exactly the same AUC without building a column, which is the kernel trick on real data. The column count explodes with the degree (3,003 at degree 5), and the RBF kernel corresponds to an infinite lift, still computed from pairwise similarities only. The stretch shows why the lift helps: `tenure_months^2` gets a large positive weight, the U-shape in tenure (risky for very new and very old customers) that Lesson 5.1's trees also find. `app_user^2` equals `app_user` (a 0/1 column squared). The top-50 counts (30 to 33) differ less than the AUCs: a top 50 is a thin slice of the ranking.

HANDOUT: LABS/MODULE-08/L4_3-SVM/DEBUG_SVM.PY (DELIBERATELY BROKEN)

1. `ValueError: Invalid parameter 'C' for estimator Pipeline(...)` → the grid key is `svc__C`.
2. `AttributeError: This 'GridSearchCV' has no attribute 'predict_proba'` → `grid.decision_function(X_test)`.
3. No error: the pipeline has no `StandardScaler` → `make_pipeline(StandardScaler(), SVC(class_weight="balanced"))`. Unscaled: best C 10, CV 0.779, test 0.781; scaled: best C 1, CV 0.848, test 0.850.

4. No error: `np.argsort(scores)[:50]` takes the 50 LOWEST scores (2 churners) → `np.argsort(-scores)[:50]` (32 churners against 8.3 at random).

- LAB 5.1-A: LABS/MODULE-08/L5_1-DECISION-TREES/FRAUD_RULES_STARTER.PY → FRAUD_RULES_SOLUTION.PY

The solution prints:

```
(35000, 9) (15000, 9) frauds: train 307, test 131
|--- velocity_24h <= 3.50
|   |--- card_attempts <= 2.50
|   |   |--- weights: [34618.00, 236.00] class: 0
|   |--- card_attempts >  2.50
|   |   |--- weights: [63.00, 29.00] class: 0
|--- velocity_24h >  3.50
|   |--- account_age_days <= 366.00
|   |   |--- weights: [2.00, 40.00] class: 1
|   |--- account_age_days >  366.00
|   |   |--- weights: [10.00, 2.00] class: 0

root [34693   307]  left [34681   265]  right [12 42]
Gini root 0.0174  left 0.0151  right 0.3457  decrease 0.0018
scikit-learn's values: [0.0174 0.0151 0.3457]

unlimited tree: 433 leaves, training AUC 1.000, test AUC 0.684, test AP 0.129
max_leaf_nodes= 4  CV average precision 0.178   CV ROC AUC 0.657
max_leaf_nodes= 8  CV average precision 0.259   CV ROC AUC 0.783
max_leaf_nodes=12  CV average precision 0.287   CV ROC AUC 0.799
max_leaf_nodes=16  CV average precision 0.295   CV ROC AUC 0.819

rule book: test AUC 0.756, test AP 0.253
|--- velocity_24h <= 3.50
|   |--- card_attempts <= 2.50
|   |   |--- velocity_24h <= 2.50
|   |   |   |--- device_new <= 0.50
|   |   |   |   |--- weights: [31374.00, 114.00] class: 0
|   |   |   |--- device_new >  0.50
|   |   |   |   |--- hour <= 5.50
|   |   |   |   |   |--- account_age_days <= 221.50
|   |   |   |   |   |   |--- weights: [30.00, 38.00] class: 1
|   |   |   |   |   |--- account_age_days >  221.50
|   |   |   |   |   |   |--- weights: [79.00, 8.00] class: 0
|   |   |   |   |--- hour >  5.50
|   |   |   |   |   |--- weights: [2972.00, 42.00] class: 0
|   |   |--- velocity_24h >  2.50
|   |   |   |--- amount <= 280.26
|   |   |   |   |--- weights: [131.00, 10.00] class: 0
|   |   |   |--- amount >  280.26
|   |   |   |   |--- weights: [32.00, 24.00] class: 0
|   |--- card_attempts >  2.50
|   |   |--- weights: [63.00, 29.00] class: 0
|--- velocity_24h >  3.50
|   |--- weights: [12.00, 42.00] class: 1

      payments  predicted  actual  frauds
leaf
```

```
2           19      0.778   0.789      15
13          32      0.559   0.531      17
8           20      0.429   0.300       6
4           29      0.315   0.172       5
14          34      0.092   0.147       5
7           61      0.071   0.115       7
12        1353      0.014   0.015      20
9        13452      0.004   0.004      56

velocity_24h                 0.512
hour                         0.161
card_attempts                0.119
account_age_days             0.114
amount                       0.070
device_new                   0.023
items                        0.000
country_mismatch             0.000
billing_shipping_mismatch    0.000

mean items: fraud 2.39  legit 1.89
```

Reading it: the root Gini is tiny (0.0174) because 99% of payments are legitimate, so impurity decreases are tiny too; the split is still the best available. The unlimited tree has the classic pattern (training 1.000, test 0.684). For a rare class, cross-validate by average precision: accuracy is 0.99 for every size. The eight-rule book is a compromise the team asked for (CV AP 0.259 against 0.295 for 16 leaves); say so in the hand-over. Its best rule, night + new device + account younger than 222 days, is the planted interaction. `items` gets zero importance although frauds buy more items on average: the tree found other columns that told it the same thing first, so "the rule book did not need items", not "items do not matter".

- LAB 5.1-B: LABS/MODULE-08/L5_1-DECISION-TREES/SALES_TREE_STARTER.PY → SALES_TREE_SOLUTION.PY

The solution prints:

```
(2924, 9) (1460, 9)
['store_Asheville', 'store_Bend', 'store_Boise', 'store_Flagstaff', 'day_of_week', 'mon
overall mean     test MAE $1,967
per-store mean   test MAE $1,670
|--- day_of_week <= 4
|   |--- store_Boise <= 0
|   |   |--- month <= 4
|   |   |   |--- value: [3312]
|   |   |--- month >  4
|   |   |   |--- value: [4343]
|   |--- store_Boise >  0
|   |   |--- month <= 4
|   |   |   |--- value: [4452]
|   |   |--- month >  4
|   |   |   |--- value: [6391]
|--- day_of_week >  4
|   |--- store_Boise <= 0
|   |   |--- event_day <= 0
|   |   |   |--- value: [6124]
|   |   |--- event_day >  0
|   |   |   |--- value: [9308]
|   |--- store_Boise >  0
|   |   |--- month <= 4
|   |   |   |--- value: [6648]
|   |   |--- month >  4
|   |   |   |--- value: [10059]

max_depth=2     CV MAE $1,374
max_depth=3     CV MAE $1,181
max_depth=4     CV MAE $1,110
max_depth=6     CV MAE $971
max_depth=8     CV MAE $904
max_depth=10    CV MAE $901
max_depth=None  CV MAE $913

depth-10 tree   test MAE $976   leaves 298
           actual 2025  predicted  2024 actual
store
Asheville       4380.0     4624.0       4649.0
Bend            5527.0     5214.0       5235.0
Boise           8342.0     7339.0       7821.0
Flagstaff       6561.0     5341.0       5744.0
highest prediction $18,918 vs highest training day $19,755

day_of_week    0.418
month          0.218
store_Boise    0.217
```

```
event_day       0.048
promo           0.046
```

Reading it: the weekday is the first question (weekends sell more), then Boise (the biggest store), then the season. Depth helps up to about 10 in time-ordered CV; even the unlimited tree barely suffers ($913) because many days share exactly the same feature values, so leaves are averages over several days rather than single days. Test MAE $976 against $1,670 for the per-store mean. The trend table is the point: every prediction is an average of 2023 and 2024 days, so Boise and Flagstaff, which grew, are under-predicted, and Asheville, which shrank, is over-predicted. A tree cannot extrapolate (Lesson 5.1 Exercise 3; Lesson 5.2 Exercise 4 shows the same for forests). Fixes: model the trend separately, or predict changes.

HANDOUT: LABS/MODULE-08/L5_1-DECISION-TREES/DEBUG_TREE.PY (DELIBERATELY BROKEN)

1. `InvalidParameterError: The 'max_depth' parameter ... Got 0 instead.` → `max_depth=2` (scikit-learn validates parameters before it looks at the data, so this error comes first).
2. `ValueError: could not convert string to float: 'Bend'` → remove `home_store` from the features (or one-hot encode it).
3. No error: `class_names=["churned", "stayed"]` swaps every label → `["stayed" if c == 0 else "churned" for c in tree.classes_]`.
4. No error: `tree.score(X_train, y_train)` printed as accuracy on new customers (0.856) → `tree.score(X_test, y_test)` = 0.851.

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 4.3 · SUPPORT VECTOR MACHINES

1. **B** Nothing: only support vectors shape the boundary.
2. **A** A narrower margin and fewer support vectors: violations got expensive.
3. **C** Each point's influence is tiny, so the model memorized the training set.
4. **B** `"svc__C"`: step name, two underscores, parameter.
5. **A** A scaled LinearSVC first, then a kernel approximation if curvature seems worth it: kernel SVC time grows with the square of the rows.

LESSON 5.1 · DECISION TREES

1. **B** 0.32: 1 - (0.64 + 0.04).
2. **C** 110.0, the midpoint between 20 and 200.
3. **A** Training AUC rises toward 1.0 while cross-validated AUC falls.

4. **B** The thresholds change units, but the predictions stay the same.

5. **C** This tree did not split on order count; other features may carry the same information.

EXIT TICKET

1. Your SVM scores 0.996 on the training customers and AUC 0.699 on the test set, with 4,491 of 4,500 customers as support vectors. Which dial is set wrong, and how do you choose it properly? **Answer:** gamma is far too large (each customer's influence is a tiny bubble, so the model memorized the training set; nearly every point is a support vector). Choose C and gamma together with cross-validation on the training set (keys svc__C, svc__gamma), never with the training score.

2. A node holds 40 stayers and 10 churners. What is its Gini impurity, and what churn probability does a tree give a customer who lands there? **Answer:** Gini = $1 - (0.8^2 + 0.2^2) = 0.32$. The probability is the leaf's churn share: 10 / 50 = 0.2.

3. The care team wants rules they can pin to the wall AND the best possible top-50 list. What do you give them from today's two models, and why not one model for both? **Answer:** A small tree (about 12 leaves, size chosen by CV) as the rule book: readable, but its probabilities come in a few big ties. For the list, a model that scores every customer individually: today the tuned SVM (34 of 50; AUC 0.848), and next session a forest (41 of 50). Readability and ranking pull in different directions.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 A (narrower street).** Has C backwards. Use the price picture: cheap violations, wide street.
- **Q3 B ("scaling is for speed").** Has not connected kernels with distance. Replay demo Step 7: 0.767 to 0.814.
- **Q5 C (accuracy up, recall down).** Thinks class weights "improve" the model in every metric. Redraw the boundary moving toward the stayers.
- **Q8 A or B.** Mixing up model families; ask "where does a tree keep its answers?" (in the leaves).
- **Q9 D (5.5) or "the mean".** Thinks thresholds are averages of all values; it is the midpoint of two neighbors.
- **Q10 B (scale the features).** Carries the SVM habit over to trees. Show demo Step 6: identical predictions.
- **Exit ticket 3: "one model for both".** The most instructive wrong answer: point at the 4 tied probabilities of the depth-2 tree against the SVM's individual scores.

INSTRUCTOR KIT · ANSWER KEY

Session 9: Random Forests and Gradient Boosting

Answers and reasoning for every Session 9 item: the warm-up, both self-checks, the predict-then-run items, the lab reference outputs, the debug handouts, the lesson pages' own quizzes and the exit ticket.

Session 9 | 12 self-check items | Confidential

WARM-UP

Session 8 retrieval; the guide has what to say after each.

W1

Lesson 4.3: you raise a linear SVM's C from 0.01 to 100 on overlapping data. Street width and number of support vectors?

Narrower street, fewer support vectors (the lesson: 4.06 wide with 42 support vectors, then 0.75 with 7).

> **Why:** C is the price of a violation.

W2

An RBF SVM with gamma=1000 scores 1.000 on training data and 0.650 on test data. What happened, and what would have warned you without the test set?

Each point's influence shrank to a tiny bubble: it memorized the training set. Cross-validation would have shown it, and so would the support-vector count (nearly every point).

> **Why:** Today's boosting has the same trap with a new dial: the number of trees.

W3

A tree node holds 40 stayers and 10 churners. Gini impurity? Predict, then check.

```
import numpy as np
p = np.array([40, 10]) / 50
print(round(float(1 - np.sum(p ** 2)), 2))
```

`0.32`

> **Why:** 1 - (0.64 + 0.04) = 0.32. A customer landing there gets churn probability 0.2.

W4

Why does an SVM need StandardScaler while a decision tree does not?

The SVM's kernel and margin are distances, so units matter. A tree only asks "above this threshold?", and rescaling moves the threshold, not the customers.

> **Why:** Same for today's forests and boosted trees: no scaling.

W5

Homework check: a bootstrap sample of 4,500 rows drawn with replacement from 4,500. What share of the distinct customers does it contain?

About 63%. Lesson 5.2's first block shows 62.9% (2,830 customers); theory 63.2%.

> **Why:** Collect the written predictions: most people guess 100% or 50%. Today opens with exactly this.

5.2 · BAGGING AND RANDOM FORESTS: SELF-CHECK

Q1

What does this print? (The share of distinct rows in a bootstrap sample of n rows from n.)

```
n = 1000
print(round(1 - (1 - 1 / n) ** n, 3))
```

`0.632`

> **Why:** Each row is missed on one draw with probability 1 - 1/n, on all n draws with (1 - 1/n)^n, which approaches 1/e = 0.368. So about 63.2% of the distinct rows are in the sample and 36.8% are out-of-bag.

Q2

Which change makes the trees of a random forest LESS correlated with each other?

A) Raising `n_estimators`

B) Setting `max_features=None`

C) Lowering `max_features`

D) Raising `min_samples_leaf` to 500

> **Why:** With fewer candidate features per split, trees are often denied the best feature and must find other splits, so their errors overlap less (the $\rho\sigma^2$ term shrinks). Lab B measures it: correlation 0.357 at max_features=1 against 0.535 at 10.
> **A** is wrong: More trees make the average more precise; each tree is just as alike as before.
> **B** is wrong: That turns the forest back into bagging: the MOST correlated trees.
> **D** is wrong: Huge leaves make every tree simple and similar.

Q3

For one training customer, which trees produce the out-of-bag prediction?

A) Only the trees whose bootstrap sample did not include that customer

B) All trees in the forest

C) Trees refitted on a held-out fold

D) The trees that predicted that customer correctly

> **Why:** About 37% of the trees never saw a given customer; their average is an unseen-data prediction, made without holding anything out. That is why the OOB AUC (0.866) matches 5-fold CV (0.868) at a fifth of the cost.
> **B** is wrong: Trees that trained on the customer memorized them: that is the 1.000 training AUC.
> **C** is wrong: No refitting happens: one fit gives the OOB score.
> **D** is wrong: That would be cheating with the answer.

Q4

What happens when this runs?

```
from sklearn.ensemble import RandomForestClassifier
RandomForestClassifier(bootstrap=False, oob_score=True).fit([[0], [1], [2], [3]], [
```

A) It fits, and oob_score_ is 1.0

B) It fits, and oob_score_ is missing

C) A warning that too few trees were used

D) `ValueError: Out of bag estimation only available if bootstrap=True`

> **Why:** Without bootstrap sampling every tree sees every row, so no row is ever out of the bag. Keep bootstrap=True (the default) or use cross-validation.
> **A** is wrong: It does not fit at all.
> **B** is wrong: scikit-learn refuses the combination at fit time.
> **C** is wrong: That warning is for too FEW trees with bootstrap on (Lesson 5.2, Common Mistake 3).

Q5

What does this print? (The variance of an average of n models with correlation 0.3.)

```
rho, var = 0.3, 1.0
print([round(rho * var + (1 - rho) * var / n, 3) for n in (1, 10, 100)])
```

`[1.0, 0.37, 0.307]`

> **Why:** $\rho\sigma^2 + (1 - \rho)\sigma^2/n$: the second term shrinks with n, the first (0.3) stays. More trees cannot remove the error the trees share; less alike trees (smaller ρ) can.

Q6

A column of pure random noise gets an impurity importance (MDI) of 0.065 in a forest, but a permutation importance of 0.005 on the test set. Best conclusion?

A) The column carries real signal that permutation importance missed

B) MDI was inflated: a many-valued column wins chance splits on the training data

C) The column has a small causal effect on churn

D) The forest needs more trees

> **Why:** MDI is computed from training-data splits and rewards columns with many distinct values. Shuffling the column on unseen data hardly hurts, so the splits did not generalize. Lab A shows the mirror image on fraud: the 0/1 column device_new is underrated by MDI (0.081) and ranks second by permutation (0.122).
> **A** is wrong: Pure noise has no signal by construction.
> **C** is wrong: Neither importance measures causes.
> **D** is wrong: More trees average the noise splits; they do not stop them.

5.2 · BAGGING AND RANDOM FORESTS: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
import numpy as np
votes = np.array([[1, 0, 1],      # tree 1's churn votes for three customers
                  [1, 1, 0],      # tree 2
                  [0, 0, 1],      # tree 3
                  [1, 0, 1]])     # tree 4
print(votes.mean(axis=0))
```

`[0.75 0.25 0.75]`

> **Why:** Each column is one customer; the mean of the four 0/1 votes is the share of trees voting churn. A forest averages probabilities the same way.

P2

Predict the output, then run it.

```
rho, var = 0.5, 1.0
for n in [1, 10, 1000]:
    print(n, round(rho * var + (1 - rho) * var / n, 3))
```

```
1 1.0 10 0.55 1000 0.5
```

> **Why:** With n = 1 the variance is $\sigma^2 = 1$. More models shrink only the second term; with correlation 0.5 the average never gets below 0.5.

5.3 · GRADIENT BOOSTING: SELF-CHECK

Q7

What does this print? (Current predictions 20 and 20, truths 26 and 14, learning rate 0.1, a tree that fits the residuals perfectly.)

```
import numpy as np
pred = np.array([20.0, 20.0])
truth = np.array([26.0, 14.0])
print(pred + 0.1 * (truth - pred))
```

```
[20.6 19.4]
```

> **Why:** Residuals +6 and -6, scaled by 0.1: each prediction moves a tenth of the way, to 20.6 and 19.4. The next tree aims at the remaining +5.4 and -5.4: a gentle putt.

Q8

Bagging and boosting both combine trees. Which statement is right?

A) Bagging averages independent deep trees and mainly reduces variance; boosting adds small trees in sequence, each fixing what is still wrong, and mainly reduces bias

B) Both grow deep trees on bootstrap samples; boosting just uses more of them

C) Boosting averages the trees; bagging adds them

D) Bagging needs early stopping; boosting never overfits

> **Why:** A forest's trees never talk to each other; averaging cancels their quirks. Boosting's trees each aim at the remaining errors, so the model grows more detailed with every tree, which is why it needs early stopping and a forest does not.
> **B** is wrong: Boosting's trees are small and trained on the residuals, not on bootstrap samples.
> **C** is wrong: Backwards.
> **D** is wrong: Backwards: more trees never make a forest overfit; boosting overfits if it runs unchecked.

Q9

You fit `HistGradientBoostingClassifier()` with all defaults on the 4,500 training customers. How many trees does it build?

A) It stops at the best number of trees on an internal validation set

B) 2,000

C) 100: early_stopping="auto" is off below 10,000 rows, so it builds max_iter=100

D) As many as there are features

> **Why:** "auto" turns early stopping on only above 10,000 rows. On churn the validation loss is best at 27 trees, so the default 100 are already past it. On the 35,000 fraud payments (Lab B), "auto" is on and the default stops at 46.
> **A** is wrong: Only above 10,000 rows, or with early_stopping=True.
> **B** is wrong: max_iter defaults to 100.
> **D** is wrong: The number of trees has nothing to do with the number of features.

Q10

What happens when this runs?

```
import pandas as pd
from sklearn.ensemble import HistGradientBoostingClassifier
X = pd.DataFrame({"tenure": [3, 40, 5, 60], "store": ["Bend", "Boise", "Bend", "Onl
HistGradientBoostingClassifier().fit(X, [1, 0, 1, 0])
```

A) It fits, treating store as categorical

B) `ValueError: could not convert string to float: 'Bend'`

C) It fits, one-hot encoding store automatically

D) A warning, then store is dropped

Why: HistGradientBoosting treats a column as categorical only if its dtype is `category`. Text is treated as a number, and 'Bend' is not one. Fix: `X[cols] = X[cols].astype("category")` before splitting.

A is wrong: Only the category dtype is detected (categorical_features="from_dtype").

C is wrong: No library one-hot encodes for you.

D is wrong: Nothing is dropped silently.

Q11

You early-stop an XGBClassifier. What goes into eval_set?

A) The test set, so it stops where it will be graded

B) The training rows it is fitted on

C) Nothing: XGBoost holds out its own validation set

D) A validation set split off from the training rows

Why: Stopping on the test set makes the test score partly a training score (the lesson's peeking model reports 0.872; Lab B's 0.867 / AP 0.510). XGBoost and LightGBM do not hold out data for you; HistGradientBoosting does (validation_fraction).

A is wrong: That is peeking: the number of trees is chosen to suit the test set.

B is wrong: The training loss falls forever: it would never stop.

C is wrong: That is HistGradientBoosting's early_stopping=True, not XGBoost's.

Q12

You forecast weekly units sold (counts, never negative) with HistGradientBoostingRegressor and get 45 negative forecasts. What is the best first fix?

A) Clip the forecasts at zero and move on

B) Use loss="poisson", which predicts on a log scale

C) Scale the target with StandardScaler

D) Add more trees

Why: Poisson loss suits counts: every forecast is positive and effects become multiplicative (a promotion adds 30%, for socks and tents alike). In the lesson project it also lowered MAE from 8.16 to 8.05. Choosing the loss to match the target is often worth more than tuning.

A is wrong: Clipping hides the symptom; the model is still additive in the wrong way.

C is wrong: Scaling the target does not stop negative predictions.

D is wrong: More trees fit the same wrong loss harder.

5.3 · GRADIENT BOOSTING: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
import numpy as np
pred, truth = np.array([10.0]), np.array([14.0])
for step in range(3):
    pred = pred + 0.5 * (truth - pred)
    print(pred)
```

`[12.] [13.] [13.5]`

Why: Each step closes half of the remaining gap of 4: 10 → 12 → 13 → 13.5. A gentle learning rate never overshoots.

P4

Predict the output, then run it.

```
import numpy as np
rate = 0.2
start = np.log(rate / (1 - rate))
print(round(float(start), 3), round(float(1 / (1 + np.exp(-start))), 3))
```

```
-1.386 0.2
```

> **Why:** log(0.2 / 0.8) = log(0.25) = -1.386; the sigmoid turns it back into 0.2. A boosted classifier starts there and adds trees on this scale.

LAB REFERENCE SOLUTIONS

- LAB 5.2-A: LABS/MODULE-09/L5_2-RANDOM-FORESTS/FRAUD_FOREST_STARTER.PY → FRAUD_FOREST_SOLUTION.PY

The solution prints:

```
(35000, 9) frauds in training: 307; a random 100 test payments hold 0.9
min_samples_leaf= 1  OOB average precision 0.489   OOB ROC AUC 0.829
min_samples_leaf= 3  OOB average precision 0.511   OOB ROC AUC 0.848
min_samples_leaf= 5  OOB average precision 0.513   OOB ROC AUC 0.863
min_samples_leaf=10  OOB average precision 0.503   OOB ROC AUC 0.861
logistic regression  test AUC 0.848   AP 0.444   frauds in top 100: 53
one unlimited tree   test AUC 0.684   AP 0.129   frauds in top 100: 41
random forest        test AUC 0.842   AP 0.479   frauds in top 100: 57

distinct scores: one tree 2 | forest 10197
                          impurity (MDI)  permutation (AP drop)
velocity_24h                       0.288                 0.202
amount                             0.176                 0.058
account_age_days                   0.161                 0.057
hour                               0.123                 0.103
card_attempts                      0.091                 0.050
device_new                         0.081                 0.122
billing_shipping_mismatch          0.037                 0.029
items                              0.024                 0.010
country_mismatch                   0.018                 0.018

distinct values: amount 27674 | account_age_days 3351 | device_new 2
```

Reading it: tuned out-of-bag by average precision, leaf 5 is best (0.513), with no CV loop. On the test set the forest's AUC (0.842) is slightly BELOW logistic regression's (0.848), yet its AP (0.479 vs 0.444) and its top 100 (57 vs 53) are

better: for a review queue only the top of the ranking matters, and AUC grades all 15,000 payments. The single unlimited tree gives only two distinct scores, so its "top 100" is an arbitrary slice of ties. Importances: MDI underrates the 0/1 column `device_new` (it can only split one way) and overrates the many-valued `amount` and `account_age_days`; permutation importance on unseen data ranks `device_new` second, consistent with the night-and-new-device interaction from Session 8.

- LAB 5.2-B: LABS/MODULE-09/L5_2-RANDOM-FORESTS/DECORRELATE_STARTER.PY → DECORRELATE_SOLUTION.PY

The solution prints:

```
per-tree predictions: (200, 1500)
forest test AUC 0.865; same as averaging the trees yourself: 0.865
 max_features  one_tree_auc  tree_correlation  oob_auc  test_auc
            1         0.709             0.357    0.862     0.854
            2         0.732             0.423    0.866     0.864
            3         0.740             0.469    0.865     0.865
            5         0.747             0.512    0.866     0.861
           10         0.747             0.535    0.862     0.856

test AUC with the first k trees: {1: 0.755, 5: 0.843, 10: 0.85, 25: 0.861, 50: 0.864, 1

top 50: 41 real churners; average spread of the trees' votes 0.284
all test customers: average spread 0.178
the five most divided of the top 50:
      p_churn  spread  churned
654     0.713   0.349        1
1016    0.689   0.342        0
168     0.696   0.328        1
734     0.679   0.326        1
1377    0.682   0.325        1
```

Reading it: as max_features rises, each tree gets better (0.709 → 0.747) AND more alike (correlation 0.357 → 0.535): the two terms of the variance formula pulling in opposite directions, and the forest is best in between (2 or 3, the default "sqrt" of 10 is 3). The differences are a few thousandths, which is why forests are forgiving. The first-k curve rises fast and flattens by about 50 trees. The top 50 holds 41 churners, and the trees disagree more about those customers than about the average one: the panel's spread is a rough confidence signal.

- HANDOUT: LABS/MODULE-09/L5_2-RANDOM-FORESTS/DEBUG_FOREST.PY (DELIBERATELY BROKEN)

1. `TypeError: BaggingClassifier.__init__() got an unexpected keyword argument 'base_estimator'` → `estimator=`.
2. `ValueError: Out of bag estimation only available if bootstrap=True` → remove `bootstrap=False`.
3. No error: `honest` is the TRAINING AUC (0.979), printed as "OOB" → `roc_auc_score(y_train,`

`forest.oob_decision_function_[:, 1])` = 0.866.

4. No error: no `random_state` on the forest → every run prints slightly different numbers; add `random_state=42`.

- LAB 5.3-A: LABS/MODULE-09/L5_3-GRADIENT-BOOSTING/BOOSTING_BY_HAND_STARTER.PY → BOOSTING_BY_HAND_SOLUTION.PY

The solution prints:

```
3600 rows to fit, 900 to validate, 1500 locked away for the test
churn rate 0.165 -> starting log-odds -1.619 -> starting probability 0.165
log loss of predicting 0.165 for everyone: 0.4488
  1 trees  fit loss 0.4375  validation loss 0.4375  validation AUC 0.850
 10 trees  fit loss 0.3737  validation loss 0.3699  validation AUC 0.866
 30 trees  fit loss 0.3247  validation loss 0.3217  validation AUC 0.882
100 trees  fit loss 0.2791  validation loss 0.2954  validation AUC 0.886
300 trees  fit loss 0.2373  validation loss 0.2898  validation AUC 0.883
best number of trees on validation: 275 (loss 0.2892)

hand-made boosting, 275 trees: test AUC 0.870, test log loss 0.3082
HistGradientBoosting, 165 trees: test AUC 0.866, test log loss 0.3091

learning rate 1.0: best 72 trees, validation loss 0.2901 (0.3: 275 trees, 0.2892); afte
```

Reading it: the hand loop is exactly the lesson's sentence, "each tree is fitted to y − p on the log-odds scale". Validation loss falls quickly and flattens; the best point (275 trees at rate 0.3) is chosen without the test set, and the test score (0.870) is in line with the library's (0.866). The library needs fewer trees because each of its leaves takes a Newton step (it divides by p(1 − p)), a bigger and smarter step than the raw residual. The stretch is the lesson's learning-rate lesson again: full-strength steps find their best sooner and then overfit.

- LAB 5.3-B: LABS/MODULE-09/L5_3-GRADIENT-BOOSTING/FRAUD_BOOSTERS_STARTER.PY → FRAUD_BOOSTERS_SOLUTION.PY

The solution prints (XGBoost and LightGBM may vary in the last digit):

```
fit 28000 rows (246 frauds), validation 7000 (61), test 15000 (131)
HGB, defaults          test AUC 0.841   AP 0.415   frauds in top 100:  55   trees 46
treated as categorical: ['category']
HGB, gentle            test AUC 0.857   AP 0.475   frauds in top 100:  57   trees 121
XGBoost                test AUC 0.862   AP 0.503   frauds in top 100:  61   best iterat
LightGBM               test AUC 0.861   AP 0.492   frauds in top 100:  59   best iterat
XGBoost, peeking       test AUC 0.867   AP 0.510   frauds in top 100:  60   best iterat
```

Reading it: with 35,000 rows the default HistGradientBoosting early-stops by itself after 46 trees and still overfits the 307 training frauds (AP 0.415); gentler, smaller trees fix that (0.475). XGBoost and LightGBM, early-stopped on average precision (the right yardstick for a rare class: log loss is dominated by the 99% legitimate payments), are best (AP about 0.50, 59 to 61 frauds in the top 100). The peeking model looks marginally better, but its stopping

point was chosen on the test set, so its number cannot be trusted. Fraud thread: 53 (logistic regression) → 57 (forest) → 61 (XGBoost).

HANDOUT: LABS/MODULE-09/L5_3-GRADIENT-BOOSTING/DEBUG_BOOSTING.PY (DELIBERATELY BROKEN)

1. `ValueError: could not convert string to float: 'Bend'` → `X[categorical] = X[categorical].astype("category")` before the split.
2. `TypeError: XGBClassifier.fit() got an unexpected keyword argument 'early_stopping_rounds'` → `early_stopping_rounds=50` in the constructor.
3. No error: `eval_set=[(X_test, y_test)]` → `eval_set=[(X_val, y_val)]`. Fixed: HGB 0.862; XGBoost 0.864 at 218 trees (peeking: 0.866 at 167).

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 5.2 · BAGGING AND RANDOM FORESTS

1. **B** About 6,300: 1 − (1 − 1/n)^n ≈ 63.2%.
2. **A** Lowering max_features: trees are denied the best feature more often and disagree more.
3. **C** Only the trees whose bootstrap sample did not include that customer.
4. **C** About the same AUC, with ten times the fitting time: the curve has plateaued.
5. **A** The column is useless; MDI inflated it through chance splits on training data.

LESSON 5.3 · GRADIENT BOOSTING

1. **B** What the current model still gets wrong: the residuals, or more generally the negative gradient of the loss.
2. **C** `[12. 8.]`: residuals ±4, scaled by 0.5.
3. **A** Many more trees before it stops, and a similar or slightly better validation score.
4. **C** A validation set split off from the training data.
5. **A** It accepts missing values and treats category-dtype columns as categorical, with no one-hot encoding.

EXIT TICKET

1. A forest and a boosted model are both made of trees. In two sentences: how are their trees different, and which kind of error does each mainly fix? **Answer:** A forest grows many deep, independent trees on bootstrap samples and averages them: it mainly reduces variance. Boosting grows small trees one after another, each fitted to what the model still gets wrong: it mainly reduces bias (and needs early stopping).

2. Why is a forest's out-of-bag score free, and when should you not trust it? **Answer:** Each tree never saw about 37% of the training rows, so averaging only those trees gives an unseen-data prediction for every row from one fit. Do not trust it when preprocessing learned from the target, when rows are not independent (time or groups), or with too few trees.

3. A colleague early-stops XGBoost with eval_set=[(X_test, y_test)] and reports AUC 0.867. What is wrong, and what do you tell them? **Answer:** The number of trees was chosen by watching the test set, so the test score is partly a training score. Early-stop on a validation set split off the training rows (or HistGradientBoosting's built-in early stopping) and touch the test set once.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 written as 0.5 or 1.0.** Thinks a sample of n contains all or half the rows. Run the lesson's first block once more.
- **Q2 A (more trees).** Confuses the two terms of the formula: n shrinks the second term only.
- **Q6 A.** Trusts MDI. Show the noise column's two bars side by side.
- **Q8 B or D.** The panel and the golfer have merged. Redraw both.
- **Q9 A.** Assumes early stopping is always on; it is "auto" (above 10,000 rows).
- **Q11 A.** The most dangerous one: they will report inflated numbers in their final project. Spend a minute on it at the start of Session 10.
- **Exit ticket 2: "OOB is always trustworthy".** Revisit the target-encoding caveat before Lesson 6.1.

INSTRUCTOR KIT · ANSWER KEY

Session 10: Ensembles and Model Selection

Answers and reasoning for every Session 10 item: warm-up, both self-checks, the predict-then-run items, the lab reference outputs, the lessons' own quizzes, final-project plan guidance and the exit ticket.

Session 10 | 12 self-check items | Confidential

WARM-UP

1. You draw a bootstrap sample of 10,000 rows from 10,000. About how many distinct rows does it hold, and what are the others called? **Answer:** About 6,300 (63.2%); the rest are out-of-bag, and a forest's OOB score is computed from them.
2. A forest with 300 trees has an OOB ROC AUC of 0.866. What happens if you raise n_estimators to 3,000? **Answer:** About the same AUC, ten times the fitting time. More trees never overfit; the score plateaus.
3. In gradient boosting, what is each new tree trained to predict? **Answer:** What the current model still gets wrong: the residuals (more generally the negative gradient of the loss).
4. You early-stop an XGBoost or HistGradientBoosting model. Which rows may decide when it stops? **Answer:** A validation set split off from the training rows, never the test set.
5. HistGradientBoostingClassifier on churn needs which preparation for the five text columns? **Answer:** Only `astype("category")`: no one-hot encoding, no imputer, no scaler (missing values stay as they are).

5.4 · VOTING AND STACKING: SELF-CHECK

Q1

Three churn models have out-of-fold probabilities that correlate 0.98 with each other, and each scores about 0.86 ROC AUC. What should you expect from a soft vote of the three?

A) A large gain, because three models always beat one

B) About 0.86: members that make the same mistakes add almost nothing

C) A score below 0.5, because correlated members cancel out

D) An error: a soft vote needs uncorrelated members

> **Why:** A team gains only where its members disagree. Near-copies make the same mistakes on the same customers, like the panel in which everyone read the same review.
>
> **A** is wrong: The lesson's simulation: with 90% copying, 25 voters score no better than one.
>
> **C** is wrong: Averaging similar probabilities gives similar probabilities, not worse ones.
>
> **D** is wrong: Voting runs on any members; it just does not help when they agree.

Q2

What does this print?

```
import numpy as np
probs = np.array([[0.30, 0.80, 0.60],
                  [0.10, 0.40, 0.45]])       # 2 customers x 3 models
print(probs.mean(axis=1).round(2), (probs > 0.5).sum(axis=1))
```

`[0.57 0.32] [2 0]`

> **Why:** Soft vote: (0.30 + 0.80 + 0.60) / 3 = 0.57 and (0.10 + 0.40 + 0.45) / 3 = 0.32. Hard vote: the first customer gets two votes above 0.5, the second none. Note how a soft vote keeps a ranking (0.57 vs 0.32) that the hard vote throws away.

Q3

A stack's meta-model was fitted on each member's predictions for its own training rows. Its weights come out as logreg -3.25, forest 2.69, boost 12.66. What went wrong?

A) Nothing: the booster is simply the best member

B) The meta-model needs scaled inputs, so the weights are meaningless

C) The members were not tuned before stacking

D) In-sample predictions are overconfident, so the meta-model over-trusts the most flexible member

Why: On its own training rows the booster scores 0.999 ROC AUC, against 0.859 out-of-fold. A meta-model trained on those numbers learns to trust it blindly and even gives logistic regression a negative vote; on the test set this leaky stack scores 0.854, below its best member. Build meta-features with cross_val_predict, or let StackingClassifier's cv do it.

A is wrong: The out-of-fold weights for the same members are 1.03, 5.23 and 1.55: the booster is not dominant at all.

B is wrong: The members' probabilities are already on the same 0-to-1 scale.

C is wrong: Tuning helps, but it does not cause a negative weight on a useful member.

Q4

`hard_vote.predict_proba(X_test)` raises `AttributeError`. You need a ranked call list. What is the fix?

A) Build the vote with `voting="soft"`, from members that all have `predict_proba`

B) Call `hard_vote.predict(X_test)` and sort by it

C) Add `probability=True` to the VotingClassifier

D) Use a StackingClassifier with a hard vote inside it

Why: A hard vote counts 0/1 labels, so it has no probability to return. A soft vote averages probabilities, keeps predict_proba, and can rank customers, draw a ROC curve and move the threshold.

B is wrong: predict gives 0 or 1 for everyone: sorting by it gives ties, not a ranking.

C is wrong: VotingClassifier has no such argument (that is an SVC setting).

D is wrong: Stacking learns from probabilities; it does not take a hard vote as a member to fix this.

Q5

What happens when this runs?

```
from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
vote = VotingClassifier([LogisticRegression(), DecisionTreeClassifier()], voting="s
vote.fit([[0], [1], [2], [3]], [0, 0, 1, 1])
```

A) It fits a soft vote of the two models

B) It fits, but only the first model is used

C) `ValueError`: every member needs a name, as a (string, estimator) pair

D) `TypeError`: a VotingClassifier needs at least three members

Why: VotingClassifier and StackingClassifier need named members so that their settings can be addressed later (for example in a grid search) and found in named_estimators_. The message: `ValueError: Invalid 'estimators' attribute, 'estimators' should be a non-empty...`

A is wrong: It never gets as far as fitting.

B is wrong: Nothing is fitted; the list is rejected.

D is wrong: Two members are allowed; names are the problem.

Q6

A paired bootstrap of the stack minus the single booster gives a mean gain of +0.0023 ROC AUC, 95% interval -0.0070 to +0.0124. Both put 98 real churners in their top 150. What is the professional recommendation?

A) Ship the stack: its mean gain is positive

B) Ship the stack only if it is retrained every week

C) Rerun the bootstrap on the training rows until the interval excludes zero

D) Keep the single booster: the gain is within the noise and changes no call, and the stack has 19 fits and four pieces to run

> **Why:** An interval that comfortably contains zero means this test set cannot tell the two apart, and the business number (churners in the calls) is identical. The extra training time, latency and explanation cost buy nothing measurable.
> **A** is wrong: A positive mean inside a wide interval that includes zero is not evidence.
> **B** is wrong: Retraining frequency does not create a gain that is not there.
> **C** is wrong: Searching until a test passes is p-hacking; and training rows would be the wrong place to measure anyway.

5.4 · VOTING AND STACKING: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
import numpy as np
votes = np.array([[1, 0, 1],
                  [0, 0, 1],
                  [1, 1, 1]])          # three models, three customers
print((votes.sum(axis=1) >= 2).astype(int))
```

`[1 0 1]`

> **Why:** Each row is one customer; a customer gets 1 when at least two of the three models vote 1. Hard voting is exactly this count.

P2

Predict the output, then run it.

```
import numpy as np
a = np.array([0.10, 0.40, 0.80, 0.30])
b = 2 * a + 0.05                        # a "second model" that only rescales the fir
print(round(np.corrcoef(a, b)[0, 1], 3), np.argsort(a).tolist() == np.argsort(b).to
```

```
1.0 True
```

> **Why:** A rescaled copy correlates perfectly (1.0) and ranks customers identically: it is a second voter repeating the first. Correlation of out-of-fold probabilities measures exactly this.

6.1 · MODEL SELECTION AND LEARNING CURVES: SELF-CHECK

Q7

Why does the lesson pass ONE splitter object, created with a fixed random_state, to every candidate's cross_validate call?

A) So the test set takes part in the comparison

B) So every candidate is scored on exactly the same folds and the scores can be compared fold by fold

C) So each model is fitted only once

D) Because cross_validate refuses an integer cv

> **Why:** Some folds are simply harder than others. When every model meets the same folds, paired differences cancel that shared difficulty. With a different shuffle per model, the forest's edge over boosting fell from 9 wins of 10 to 6 of 10.
> **A** is wrong: The test set never takes part in model selection.
> **C** is wrong: Every candidate is still fitted once per fold.
> **D** is wrong: cv=5 is accepted (and also gives identical folds, because it does not shuffle).

Q8

What does this print?

```
import numpy as np
a = np.array([0.84, 0.87, 0.88, 0.85, 0.86])   # model A, 5 shared folds
b = np.array([0.83, 0.85, 0.89, 0.84, 0.84])   # model B, same folds
diff = a - b
print(round(diff.mean(), 3), (diff > 0).sum())
```

`0.01 4`

Why: The per-fold differences are 0.01, 0.02, -0.01, 0.01, 0.02: mean 0.01, and A is ahead in four of the five folds. That is the paired view: fold difficulty is shared, so the difference is what tells you about the models.

Q9

A validation curve for a tree shows training AUC 0.857 and validation AUC 0.835 at depth 4, and training AUC 1.000 with validation AUC 0.673 at depth 15. What is happening at depth 15?

A) Underfitting: the tree is too simple

B) Nothing: a perfect training score means the model is right

C) Overfitting: the tree memorizes the training customers

D) The validation folds contain an error

Why: A near-perfect training score with a falling validation score is the overfitting signature: the gap between the curves is the overfitting meter. Depth 4 to 5 is the sweet spot.

A is wrong: Underfitting looks like depth 1 to 3: both scores low and close together.

B is wrong: A perfect training score is usually the warning, not the reward.

D is wrong: The same pattern appears on every fold: it is the model, not the data.

Q10

Logistic regression's learning curve flattens at about 0.83 by 1,200 rows, with training and validation scores close together. Which next step is most likely to help?

A) Better features or a more flexible model

B) Collecting twice as many rows of the same data

C) Stronger regularization

D) A larger test set

Why: Curves that meet early and flatten mean high bias: the model has learned all it can from these columns. One lapsed-customer flag added 0.005 (Exercise 3), and a feature-built logistic regression reaches 0.877 (this session's worked example).

B is wrong: More rows of the same data do not move a flat, converged curve.

C is wrong: Regularization fights variance; this model is not overfitting.

D is wrong: A test set measures; it does not improve the model.

Q11

You call `validation_curve(tree_pipe, ..., param_name="max_depth", ...)` where tree_pipe was built with make_pipeline. What happens?

A) It works: the pipeline passes max_depth to the tree

B) `ValueError`: the pipeline has no max_depth; use `decisiontreeclassifier__max_depth`

C) It silently varies the scaler instead

D) `KeyError`: max_depth

Why: A pipeline's own parameters are memory, steps, transform_input and verbose. A step's parameter is addressed as step name, two underscores, parameter; make_pipeline names steps after their class in lowercase. sorted(tree_pipe.get_params()) lists every valid name.

A is wrong: Pipelines do not guess which step you mean.

C is wrong: Nothing runs silently: the name is rejected.

D is wrong: scikit-learn raises ValueError with the list of valid parameters.

Q12

One-standard-error rule. What does this print?

```
import numpy as np
depths = [3, 4, 5, 6]
means = np.array([0.829, 0.835, 0.834, 0.829])
se = np.array([0.007, 0.007, 0.007, 0.008])
best = int(np.argmax(means))
print(next(d for d, m in zip(depths, means) if m >= means[best] - se[best]))
```

```
3
```

Why: The best depth is 4 (0.835, standard error 0.007), so the threshold is 0.828. Scanning from the simplest depth, depth 3 already reaches 0.829: the rule picks the simpler tree, trading a sliver of measured score for a model less likely to have won by luck.

6.1 · MODEL SELECTION AND LEARNING CURVES: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
from sklearn.model_selection import RepeatedStratifiedKFold
print(RepeatedStratifiedKFold(n_splits=3, n_repeats=4, random_state=42).get_n_split
```

```
12
```

Why: 3 folds × 4 repeats = 12 train/validation splits, so 12 fits per candidate.

P4

Predict the output, then run it.

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
pipe = make_pipeline(StandardScaler(), DecisionTreeClassifier())
print([name for name in pipe.get_params() if name.endswith("max_depth")])
```

```
['decisiontreeclassifier__max_depth']
```

Why: make_pipeline names each step after its class in lowercase; a step's parameter is step name, two underscores, parameter.

LAB REFERENCE SOLUTIONS

- LAB 5.4-A: LABS/MODULE-10/L5_4-VOTING-STACKING/FRAUD_ENSEMBLE_STARTER.PY → FRAUD_ENSEMBLE_SOLUTION.PY

The solution prints:

```
Jan-May: 41,738 payments, 374 frauds | June: 8,262 payments, 64 frauds

out-of-fold average precision (Jan-May):
  logreg  0.506
  forest  0.513
  boost   0.524
correlation of out-of-fold probabilities:
        logreg  forest  boost
logreg    1.00    0.89   0.89
forest    0.89    1.00   0.92
boost     0.89    0.92   1.00

soft votes, out-of-fold average precision:
  all three       0.541
  forest + boost  0.531
  logreg + boost  0.539
chosen on Jan-May: all three

meta-model weights (logreg, forest, boost): [5.04 4.45 5.5 ]

June (opened once), 64 frauds; a random 100 would hold 0.8:
  logreg                AP 0.539   frauds in top 100: 36
  forest                AP 0.509   frauds in top 100: 35
  boost                 AP 0.556   frauds in top 100: 36
  vote: all three       AP 0.570   frauds in top 100: 36
  stack                 AP 0.573   frauds in top 100: 36

stack minus boost: +0.016 AP, 95% interval [-0.016, +0.057]
Verdict: keep the single model (boost): June cannot tell them apart, and the stack cost
```

Reading it: the fraud members disagree more than the churn members (they still correlate 0.89 to 0.92), and the soft vote of all three gains about 0.017 AP out-of-fold over the best member. On June the stack and the vote score highest, but the bootstrap interval for the gain runs from -0.016 to +0.057: 64 frauds cannot separate them, and every contender puts 35 or 36 frauds in a 100-payment queue. The verdict printed from the variables is the lesson's: keep one model. Meta-model weights near 5 for all three are not "importance"; they say the meta-model leans on all three opinions about equally.

- LAB 5.4-B: LABS/MODULE-10/L5_4-VOTING-STACKING/DEMAND_BLEND_STARTER.PY → DEMAND_BLEND_SOLUTION.PY

The solution prints:

```
fit Jan-Sep 2024: 3,840 rows | blend on Oct-Dec 2024: 1,248 | test 2025: 4,896
boost  MAE on Oct-Dec 2024: 9.78
forest MAE on Oct-Dec 2024: 10.92
correlation of the two holdout forecasts: 0.95
  weight on boost 0.0: holdout MAE 10.92
  weight on boost 0.1: holdout MAE 10.75
  weight on boost 0.2: holdout MAE 10.58
  weight on boost 0.3: holdout MAE 10.44
  weight on boost 0.4: holdout MAE 10.31
  weight on boost 0.5: holdout MAE 10.19
  weight on boost 0.6: holdout MAE 10.08
  weight on boost 0.7: holdout MAE 9.99
  weight on boost 0.8: holdout MAE 9.91
  weight on boost 0.9: holdout MAE 9.84
  weight on boost 1.0: holdout MAE 9.78
chosen weight on boost: 1.0

2025 (opened once):
  boost          MAE 8.05
  forest         MAE 8.47
  blend          MAE 8.05
  50/50 average  MAE 8.11

blend saves 0.00 units per row against the booster alone (0.0%)
Verdict: keep the single booster: the blend's gain is too small to pay for a second mo
```

Reading it: the weight is chosen on October to December 2024, a period after the members' training months, which is how time-ordered data replaces out-of-fold predictions (Lesson 5.4, Mistake 3). The holdout says "all booster", and 2025 agrees: the 50/50 average (8.11) is worse than the booster alone (8.05). Same outcome as the lesson's Exercise 4 stack (weights 0.93 / 0.04 / 0): an honest way of finding out you do not need an ensemble.

HANDOUT: LABS/MODULE-10/L5_4-VOTING-STACKING/DEBUG_STACK.PY (DELIBERATELY BROKEN)

1. `ValueError: Invalid 'estimators' attribute, 'estimators' should be a non-empty list of (string, estimator) tuples.` → `VotingClassifier([("logreg", logreg), ("boost", boost)], ...)`.
2. `AttributeError: This 'VotingClassifier' has no attribute 'predict_proba'` → `voting="soft"` (the comment even says soft vote).
3. `ValueError: cross_val_predict only works for partitions` → `cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=42)`: churn rows are not ordered in time, and the meta-model needs one prediction for every training row.
4. No error: the hand-made meta-model is fitted on in-sample predictions. With the bug it prints weights [-2.44 13.59] and AUC 0.853; with `cross_val_predict(..., cv=cv, method="predict_proba")` it prints [2.58 3.44] and 0.860, the same as the StackingClassifier. The soft vote prints 0.858.

- LAB 6.1-A: LABS/MODULE-10/L6_1-MODEL-SELECTION/FRAUD_BAKEOFF_STARTER.PY → FRAUD_BAKEOFF_SOLUTION.PY

The solution prints:

```
               ap_mean  ap_std  auc_mean
model
boost_gentle     0.521   0.057     0.847
logistic         0.504   0.051     0.855
boost_default    0.446   0.050     0.827
baseline         0.009   0.000     0.500
boost_gentle - boost_default: mean +0.075, std 0.034, boost_gentle wins 10 of 10 folds
boost_gentle - logistic: mean +0.017, std 0.023, boost_gentle wins 7 of 10 folds

chosen on Jan-May: boost_gentle (CV AP 0.521 +/- 0.057)
June boost_gentle  AP 0.556   frauds in top 100: 36 of 64
June logistic      AP 0.539   frauds in top 100: 36 of 64
June random order  AP 0.008
```

Reading it: (1) the metric changes the winner: AUC ranks logistic regression first, AP ranks gentle boosting first; the review queue is the top of a ranked list, so AP was the right choice, and it was made before looking. (2) Default boosting overfits the 374 frauds: gentle settings win all 10 folds by 0.075. (3) Gentle boosting against logistic regression is a close call: it passes the lesson's rule ($0.017 > 0.023 / \sqrt{10} \approx 0.007$) but wins only 7 of 10 folds, and the June queues hold the same 36 frauds. A report that picks either model and says "close; the simpler model is a defensible choice" is correct. Baseline AP 0.009 = the fraud rate of the training months.

- LAB 6.1-B: LABS/MODULE-10/L6_1-MODEL-SELECTION/DEMAND_CURVES_STARTER.PY → DEMAND_CURVES_SOLUTION.PY

The solution prints:

```
5088 rows of 2024, sorted by week
shuffled KFold   CV MAE 7.60 +/- 0.19
blocks of weeks  CV MAE 8.49 +/- 1.34
2025 test MAE of this model (Lesson 5.3): 8.05

training rows  train MAE  validation MAE
         1343       3.42           10.66
         2726       4.82            9.96
         4070       5.32            8.41
saved demand_learning_curve.png

at full size: validation MAE 8.41, gap to training 3.09 units; the last third of the r
Diagnosis: high variance and still improving: more history (a third year) should help
```

Reading it: shuffled folds put weeks from both sides of each validation week into training, so the model interpolates within a season it has already seen: 7.60, far better than the real 2025 test (8.05). Blocks of weeks (unshuffled KFold on rows sorted by week) are pessimistic instead (8.49), because each block's season is missing

from its training data; for a whole-year-ahead forecast that is the safer side to err on. Learning curve: training MAE far below validation and validation still falling fast: variance, so more history (a third year) should help. One honest caveat for strong learners: on time data, more rows also means more of the calendar, so part of the gain is seasonal coverage. learning_curve trains every fold on the same number of rows (4,070), so its full-size point (8.41) is close to, not identical with, cross_val_score's 8.49.

HANDOUT: LABS/MODULE-10/L6_1-MODEL-SELECTION/DEBUG_BAKEOFF.PY (DELIBERATELY BROKEN)

1. `InvalidParameterError: The 'scoring' parameter of cross_validate must be a str among {...} ... Got 'auc' instead.` → `scoring="roc_auc"`.
2. `ValueError: Invalid parameter 'max_depth' for estimator Pipeline(...)` → `param_name="decisiontreeclassifier__max_depth"`.
3. No error: each candidate gets its own splitter (`random_state=seed`), so the paired differences mix two folds' luck (with the bug: logistic 0.830, tree 0.838, +0.008 winning 7 of 10). Fix: one `RepeatedStratifiedKFold(..., random_state=42)` created before the loop.
4. No error: the winner is picked by test AUC, so the test set chose the model. Fix: choose by the CV mean, then fit and score only the chosen model once. Fixed output: logistic 0.832 +/- 0.017, tree_d5 0.842 +/- 0.018, +0.010 winning 8 of 10, best depth 4, chosen tree_d5, test AUC 0.846 (here the choice happens to be the same; the process was still wrong).

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 5.4 · VOTING AND STACKING

1. **C** 70%: identical voters are one voter repeated; they are wrong together on the same 30% of cases.
2. **A** In-sample predictions are overconfident, so the meta-model would learn to over-trust the most flexible member.
3. **B** `[0.6] [2]`: the average of 0.2, 0.9 and 0.7, and two of the three are above 0.5.
4. **A** Given the other two members' predictions, the forest's probability moves the final prediction the most.
5. **C** The stack's gain is within the noise of a bootstrap, and the model must score customers quickly and be explained.

LESSON 6.1 · MODEL SELECTION AND LEARNING CURVES

1. **B** Overfitting: training 1.000, validation 0.67.
2. **C** A more flexible model or better features: the curves met and flattened (high bias).
3. **A** So every model is scored on the same folds and the scores can be compared fold by fold.
4. **C** Within noise (6 of 10 wins, mean 0.004 against a std of 0.012): choose on speed, simplicity or explainability.

5. **A** 15: 5 folds × 3 repeats.

FINAL-PROJECT PLANS: WHAT TO LOOK FOR

- **A good plan** names a decision and who makes it, the moment of prediction, the course's split (copied, not invented), one deciding metric chosen now, the do-nothing number, two candidates, one specific leak, and what would change the recommendation.
- **Churn plans** that list every column as a feature: ask about `cancellation_reason` (filled only after a customer leaves; it lifts AUC to about 0.94). **Demand plans** with "units sold last week" as a feature: ask when the forecast is made (the start of 2025: last week's sales are not known for most weeks). **Fraud plans** with a stratified random split: ask what June is for (the model is used on the future; the fraud rate drifts from 1.00% to 0.77%).
- **Metric:** accuracy as the deciding metric is a red flag for churn and fraud (doing nothing scores 0.835 and 0.991). For demand, MAE in units is the course's choice; RMSE is acceptable if argued.
- The full rubric and instructor answer notes with reference numbers are in `final-assessment.html` (kit root).

EXIT TICKET

1. **Out-of-fold:** in-sample predictions are overconfident (booster 0.999 in-sample, 0.859 out-of-fold), so a meta-model trained on them over-trusts the most flexible member; the leaky stack scored 0.854, below its best member.
2. **0.867 vs 0.860, +0.007, std 0.008, 9 of 10:** small but consistent (the mean is about three times 0.008 / √10). Then weigh cost, speed and explainability, and confirm on the test set once.
3. **Final project:** any coherent triple: churn / AP (or profit) / 0.165 and -$7,620; demand / MAE / 13.42 (per-product mean); fraud / AP then cost / $24,000 for reviewing nothing.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 A ("a large gain").** Believes more models always help. Show the simulation's 90%-copying line.
- **Q3 A ("the booster is simply the best").** Reads weights as accuracy. Put the out-of-fold weights (1.03 / 5.23 / 1.55) beside the leaky ones.
- **Q6 A ("ship it: the mean gain is positive").** Ignores the interval. Ask what the care team would notice (nothing: 98 vs 98).
- **Q10 B ("more data").** Reads "lower score" as "needs data". Point at the two curves touching.
- **Exit ticket 2 "forest wins" with no reasoning.** The habit the lesson targets: insist on the paired numbers.

INSTRUCTOR KIT · ANSWER KEY

Session 11: Hyperparameter Tuning and Final Project Studio

Answers and reasoning for every Session 11 item: warm-up, the project-skills self-check, the predict-then-run items, the lab reference outputs, the lesson page's quiz, studio and showcase guidance and the exit ticket.

Session 11 | 12 self-check items | Confidential

WARM-UP

1. Five 70%-accurate classifiers always make exactly the same predictions. How accurate is their majority vote? **Answer:** 70%: identical voters are one voter repeated.
2. Where must a stacking meta-model's training features come from? **Answer:** Out-of-fold predictions on the training rows (`cross_val_predict`, or `StackingClassifier`'s `cv`).
3. Why pass one splitter object with a fixed random_state to every candidate? **Answer:** So all are scored on the same folds and can be compared fold by fold (paired differences).
4. A learning curve's training and validation scores meet at 0.83 and stay flat. Next step? **Answer:** Better features or a more flexible model (high bias); more rows of the same data will not help.
5. Boosting's learning curve: training 1.0, validation still rising to 0.861. What would close the gap? **Answer:** Regularization (smaller trees, larger leaves, a smaller learning rate) or more data. Today's lesson tests it: max_leaf_nodes 7 lifts CV AUC to 0.8795.

SELF-CHECK: PROJECT SKILLS

Q1

Which of these is a well-framed final-project question?

A) Build the most accurate churn model possible

B) Use gradient boosting on the transactions table

C) Which customers should get the $15 retention offer next quarter, judged by profit on held-out customers, against offering to everyone (-$7,620)?

D) Find out what causes churn

Why: A framed question names the decision, who acts on it, the deciding measure and the comparison. The other three name a technique, a target without a decision, or a causal question a predictive model cannot answer.
A is wrong: Accuracy is not the business measure, and "most accurate possible" has no comparison or decision.
B is wrong: A technique is not a question; the model should be chosen by evidence.
D is wrong: A predictive model describes associations, not causes (Module 7 keeps these apart).

Q2

Your weekly-demand model (train 2024, test 2025) has an MAE of 9.1 units. Which number must it beat to be worth anything?

A) The per-product 2024 mean: MAE 13.42

B) Zero, the perfect forecast

C) The training MAE of the same model

D) A shuffled 75/25 split's MAE of 7.36

Why: A buyer can forecast each product's average without any model; that is the bar. The model at 9.1 beats it by about a third; Poisson boosting reaches 8.05.
B is wrong: No forecaster reaches zero; a baseline is what doing nothing clever scores.
C is wrong: Training error is optimistic and measures nothing about 2025.
D is wrong: A shuffled split mixes 2025 weeks into training: a leak, not a bar.

Q3

A learner adds "units sold last week" as a feature to the demand forecaster, which is meant to plan all of 2025 at the start of the year. What is wrong?

A) Nothing: lag features always help

B) Lag features need scaling first

C) The feature makes the model slower

D) On the planning date, last week's sales are unknown for almost every 2025 week: it is a framing leak

Why: A feature is legitimate only if it is known when the prediction is made. A forecast for week 30 made in January cannot use week 29's sales. (Here it does not even help: MAE 8.17 against 8.05.) It would be fine for a one-week-ahead forecaster, which is a different project.

A is wrong: Whether a lag is allowed depends on the forecast horizon, not on the feature.

B is wrong: Trees do not need scaling; the problem is timing.

C is wrong: Speed is irrelevant; validity is the issue.

Q4

How many fits does a GridSearchCV with cv=5 run on this grid, not counting the refit?

```
from sklearn.model_selection import ParameterGrid
grid = {"model__learning_rate": [0.03, 0.1, 0.3],
        "model__max_leaf_nodes": [7, 15, 31, 63]}
print(len(ParameterGrid(grid)) * 5)          # cv=5
```

`60`

Why: 3 × 4 = 12 settings, each cross-validated on 5 folds: 60 fits, plus one refit of the winner. A grid's cost is the product of its list lengths.

Q5

Your search reports best_score_ 0.636 (CV average precision). The tuned model scores 0.614 on the test set, opened once. Which number goes in your report as the model's expected performance?

A) 0.636, because it is the cross-validated score

B) 0.614, the one-time test score (or a nested CV estimate), with best_score_ described only as what chose the settings

C) Their average, 0.625

D) Whichever is higher, since both are honest

Why: best_score_ is the best of many noisy scores (the winner's curse): it chose the model and is biased upward. A score from data that played no part in the choice, the one-time test or a nested estimate, is the one to promise.
A is wrong: Cross-validated, yes, but maximized over 15 settings.
C is wrong: Averaging an optimistic number with an honest one is still optimistic.
D is wrong: Picking the higher one is choosing by the result.

Q6

Two hyperparameters, only the first matters, budget 9 trials. Why does random search usually beat a 3 × 3 grid?

A) Random search uses more folds

B) Random search learns from earlier trials

C) It tries 9 different values of the important hyperparameter instead of 3

D) A grid cannot use log scales

Why: The grid repeats the same three values of the important knob; every random trial draws a fresh value of every knob (Bergstra and Bengio).
A is wrong: Both use whatever cv you pass.
B is wrong: That is Optuna's TPE sampler, not random search.
D is wrong: A grid can list log-spaced values, such as 0.001, 0.01, 0.1.

Q7

For the fraud project, which split is the leak-safe one?

A) Train January to May, test June, never shuffling months together

B) A stratified 70/30 split of all six months, random_state=42

C) Train on June, test on January to May

D) Five-fold cross-validation on all six months, with no test set

> **Why:** The model will be used on future payments, and the fraud rate drifts (1.00% in January, 0.77% in June). Training on the past and testing on a later month is the only split that matches use; cross-validation for choices happens inside January to May.
> **B** is wrong: It lets the model learn from June while being tested on earlier months, and hides the drift.
> **C** is wrong: Backwards in time: the future would teach the past.
> **D** is wrong: Without an untouched test month there is no honest final number.

Q8

What does this print?

```
import numpy as np
print(np.logspace(-3, 1, 5))
```

`[1.e-03 1.e-02 1.e-01 1.e+00 1.e+01]`

> **Why:** Five values evenly spaced in powers of ten from 10^-3 to 10^1: each ten times the last. That is the right spacing for learning rates and regularization strengths.

Q9

A churn offer costs $15 and is worth $60 when it reaches a real churner. If the probabilities are well calibrated, above which churn probability does an offer pay for itself?

A) 0.50, the default of predict

B) 0.75

C) 0.15

D) 0.25

> **Why:** An offer to a customer with churn probability p is worth 60p on average and costs 15: it pays when 60p > 15, so p > 0.25. Tuned on the training customers, TunedThresholdClassifierCV picks a threshold close to that (0.240 in this session's worked example).
> **A** is wrong: 0.5 is a convention, not a business number.
> **B** is wrong: That would be the break-even for a $45 offer.
> **C** is wrong: 15 is the cost in dollars, not a probability; divide by the value.

Q10

Which sentence is an honest report of a tuning result?

A) Tuning made the model 2% better

B) Tuned boosting: test ROC AUC 0.874 against 0.860 for the default, on the same 1,500 test customers, after a search on the training rows only; the nested CV estimate of the procedure is 0.868

C) Our best CV score was 0.8795, so the model will score 0.88 on new customers

D) Tuning proved that small trees cause better predictions

> **Why:** It states the comparison, on which data, how the choice was made, and an estimate that accounts for the search. It promises no more than the evidence shows.
> **A** is wrong: Better on what, measured how, compared with what?
> **C** is wrong: best_score_ is the winner's curse number; the nested estimate is about 0.868.
> **D** is wrong: Tuning finds settings that predict well; it says nothing about causes.

Q11

You choose the churn offer threshold by trying thresholds on the TEST customers and keeping the most profitable (0.26: $5,475). The threshold chosen by TunedThresholdClassifierCV on the training customers (0.240) earns $5,265 on the same test set. Which number goes in the report?

A) $5,475: it is the best result you found

B) The average of the two

C) $5,265: the threshold was chosen without the test set, so its test profit is an honest estimate

D) Neither: profits cannot be reported

> **Why:** Choosing the threshold on the test set makes the test set part of training: $5,475 is the best of many tries on the very data that grades it. The honest number is the one from a threshold chosen on training data, scored once.
> **A** is wrong: Best-of-many on the grading data is optimistic by construction.
> **B** is wrong: Mixing an optimistic number in does not make it honest.
> **D** is wrong: Profit is the business number the decision needs; report it honestly.

Q12

Fraud review cost ($10 per review, $375 per missed fraud). What does this print?

```
from sklearn.metrics import confusion_matrix
y_true = [0, 0, 0, 1, 1, 0, 1, 0]
flag   = [1, 0, 0, 1, 0, 1, 1, 0]
tn, fp, fn, tp = confusion_matrix(y_true, flag).ravel()
print(10 * (tp + fp) + 375 * fn)
```

`415`

> **Why:** Four payments flagged (tp 2, fp 2): 4 × $10 = $40. One fraud missed: $375. Total $415. The same formula the fraud project uses: review nothing costs $24,000 in June.

PREDICT, THEN RUN

P1

Predict the output, then run it.

```
from sklearn.model_selection import ParameterGrid
grid = {"model__learning_rate": [0.05, 0.1], "model__max_leaf_nodes": [7, 15, 31],
        "model__l2_regularization": [0.0]}
print(len(ParameterGrid(grid)), len(ParameterGrid(grid)) * 5)
```

```
6 30
```

Why: 2 × 3 × 1 = 6 settings; with 5 folds, 30 fits (plus one refit). A single-value list still counts as one setting.

P2

Predict the output, then run it.

```
import pandas as pd
demand = pd.read_csv("data/weekly_demand.csv")
print((demand["year"] == 2024).sum(), (demand["year"] == 2025).sum())
```

```
5088 4896
```

Why: 2024 has 53 Mondays in the data (2024-12-30 counts as 2024): 53 × 96 = 5,088 rows; 2025 has 51 × 96 = 4,896. A split by year is exactly these sizes.

P3

Predict the output, then run it.

```
print(round(15 / 60, 2), round(10 / 375, 3))
```

```
0.25 0.027
```

Why: Break-even probabilities: churn offer $15 / $60 = 0.25; fraud review $10 / $375 = 0.027. Act when the probability is above them.

LAB REFERENCE SOLUTIONS

- LAB 6.2-A: LABS/MODULE-11/L6_2-HYPERPARAMETER-TUNING/TUNE_FRAUD_STARTER.PY → TUNE_FRAUD_SOLUTION.PY

The solution prints:

```
default booster (early stopping), 3-fold AP on Jan-May: 0.443 +/- 0.032
8 settings x 3 folds = 24 fits, plus 1 refit
best CV AP: 0.515
{'l2_regularization': 0.0071, 'learning_rate': 0.0186, 'max_leaf_nodes': 24, 'min_sampl
 param_learning_rate  param_max_leaf_nodes  param_min_samples_leaf  mean_test_score  st
               0.019                    24                     170            0.515
               0.012                    27                     167            0.513
               0.031                    43                     161            0.508
               0.017                    22                      84            0.504
               0.060                    47                      98            0.499
               0.080                    45                     197            0.487
               0.047                    50                     199            0.480
               0.254                    46                      81            0.262
settings within one std of the best: 5 of 8
June default  AP 0.477   frauds in top 100: 32 of 64
June tuned    AP 0.544   frauds in top 100: 38 of 64
saved fraud_best_params.json with ['best_params', 'cv', 'cv_score', 'default_cv_score',
```

Reading it: the default (with early stopping) overfits the 374 training frauds (CV AP 0.443); the search finds small learning rates with large leaves (0.515). Five of eight settings are within one std of the winner, and they share that recipe, which matters more than the exact winner. The 0.254 learning rate is unstable (0.262 +/- 0.169): some folds collapse. On June, opened once, tuning lifts AP from 0.477 to 0.544 and puts 38 instead of 32 of the 64 frauds in a 100-payment queue. The JSON record lets anyone rebuild the model.

- LAB 6.2-B: LABS/MODULE-11/L6_2-HYPERPARAMETER-TUNING/WINNERS_CURSE_STARTER.PY → WINNERS_CURSE_SOLUTION.PY

The solution prints:

```
shuffled labels keep the churn rate (0.165) but break every link to the features
  1 settings tried: best_score_ 0.504
 10 settings tried: best_score_ 0.514
 40 settings tried: best_score_ 0.537

shuffled labels: best_score_ 0.537 | nested 0.501 | test 0.524
real labels:     best_score_ 0.848 | nested 0.840 | test 0.859

The number to promise is the nested estimate or the one-time test score, never best_sc
```

Reading it: with shuffled labels the true AUC of any model is 0.5, yet best_score_ rises with the number of settings tried (0.504, 0.514, 0.537): the best of more noisy scores is higher by chance. The nested estimate (0.501) sees through it. The test score on noise (0.524) is a single noisy draw of 1,500 rows. With the real labels the gap between best_score_ (0.848) and nested (0.840) is small because there is real signal and the search space is modest.

- FINAL PROJECT SCAFFOLD: LABS/MODULE-11/L6_2-HYPERPARAMETER-TUNING/FINAL_PROJECT_STARTER.PY → FINAL_PROJECT_SOLUTION.PY

The reference runs milestone 1 for all three problems:

```
CHURN  train 4,500 / test 1,500; first test id C102112
  bar to beat: random-order AP 0.165; contact everyone -7,620 dollars; contact nobody (
  first model: logistic regression, 5-fold CV AP 0.528 +/- 0.042
DEMAND train 5,088 (2024) / test 4,896 (2025)
  bar to beat: overall mean MAE 17.13; per-product mean MAE 13.42
  first model: Poisson boosting, CV on blocks of 2024 weeks, MAE 8.49 +/- 1.34
FRAUD  train 41,738 Jan-May (374 frauds) / test 8,262 June (64 frauds)
  bar to beat: review nothing $24,000; review everything $82,620; break-even probabili
  first model: logistic regression, 5-fold CV AP on Jan-May 0.506 +/- 0.054
```

Use it to check any learner's milestone 1 at a glance: split sizes and the bar to beat must match exactly; first CV scores should match when the learner used the same pipeline and folds.

THE LESSON PAGE'S OWN QUICK QUIZ

LESSON 6.2 · HYPERPARAMETER TUNING

1. **B** 40: 4 × 2 = 8 settings × 5 folds.
2. **A** It is the highest of many noisy scores, so it partly reflects lucky noise.
3. **C** It tries 9 distinct values of the important hyperparameter instead of 3.
4. **C** Every candidate is scored on a small slice of rows, and the best third go on.
5. **A** `"model__learning_rate"`.

STUDIO AND SHOWCASE: WHAT GOOD LOOKS LIKE

- **Milestone 2:** the baseline row is in the leaderboard; one splitter object; the deciding metric is the one in the plan; mean and std side by side; the top two compared fold by fold.
- **Milestone 3:** the space is written before the search; rates and strengths on log scales; the search and any threshold tuning run on the training rows only; the report will quote a nested estimate or the test score.
- **Showcase talk:** leads with the decision and the bar to beat, gives a spread with every score, names one trap. A talk that quotes only accuracy, or only best_score_, gets the private follow-up question.
- Reference numbers for every problem, the traps and what an honest result looks like: `final-assessment.html`.

EXIT TICKET

1. **best_score_:** the maximum of many noisy CV scores, biased upward (winner's curse); promise a nested estimate or the one-time test score.

2. **Three numbers:** for example churn: random-order AP 0.165 → tuned CV AP 0.636 ± 0.04 → test AP 0.614 (once); demand: per-product mean 13.42 → CV 8.49 → test 8.05; fraud: review nothing $24,000 → CV AP about 0.52 → June cost about $9,900 at a threshold tuned on January to May.
3. **What is left:** no right answer; use it to plan follow-ups.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q2 D ("the shuffled split's 7.36").** Thinks a lower error is a higher bar. Ask what the shuffled split let the model see.
- **Q3 A ("lag features always help").** Has not yet separated features from the moment of prediction. Ask: on 2 January, what is week 29's sales?
- **Q5 A / Q10 C.** Treats best_score_ as a forecast. Show the winners-curse lab: 0.537 on pure noise.
- **Q11 A.** Believes the best number found is the result. The test set graded a choice it made.

Complete Answer Key · all 11 sessions · v1.0 · © 2026 Ray de la Paz